MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Capstone Project Document

A PRIVACY-PRESERVING DATA GENERATION FRAMEWORK IN FEDERATED LEARNING FOR ENHANCED IOT SYSTEM ATTACK DETECTION

IAP491_G..		
Group Members	Nguyễn Nam Khánh	
	Nguyễn Sơn Trường Giang	
	Nguyễn Trí Dũng	
	Vũ Đức Lâm	
	Nguyễn Trọng Hoàng Đạt	
Advisor	TS. Nguyễn Anh Nhật	
Co-advisor (if any)	ThS. Lưu Minh Trí	
Capstone Project code		

- 11/2025 -

©IA Dept., FPT University, Hanoi

Acknowledgments

No one achieves something great without the help of those around them, whether directly or indirectly. To complete this capstone project, we were fortunate to receive tremendous support and assistance from our supervisors, lecturers, friends, and family. We would like to dedicate these opening pages to express our sincere gratitude to everyone who has accompanied us throughout this journey.

First and foremost, we would like to express our deepest appreciation to all the lecturers at FPT University Hanoi Campus, particularly those in the Information Assurance Department. The valuable knowledge they have imparted, along with their dedicated support throughout the implementation process, has enabled us to complete this capstone project and achieve noteworthy results.

We would like to extend our special gratitude to our supervisor, Dr. Nguyễn Anh Nhật, and our co-supervisor, MSc. Lưu Minh Trí, who have inspired us, provided dedicated guidance, and offered invaluable support in terms of knowledge. They have created a favorable environment for us to learn and exchange ideas with fellow students in the research group. These are precious knowledge and experiences that will benefit us not only in this capstone project but also in our future careers.

Finally, we wish to express our heartfelt thanks to our families and loved ones, who have always been our solid support and have encouraged every decision we have made.

Despite our best efforts to make this thesis as complete as possible, shortcomings and limitations are inevitable. We sincerely hope to receive your understanding and constructive feedback from our supervisors and readers.

Contents

Acknowledgments ii

Contents iii

List of Figures vi

List of Tables vii

Abstract ix

List of Abbreviations x

1 Introduction 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Research Objectives and Research Questions 3

1.3.1 Objectives 3

1.3.2 Research Questions 4

1.4 Significance of the Study 5

1.5 Scope and Limitations 6

1.6 Thesis Structure 7

2 Literature Review 9

2.1 Review of Previous Studies 9

2.1.1 Overview of Privacy-Preserving Mechanisms in Federated Learning 9

2.1.2 Data Generation 12

2.1.3 Privacy-Preserving Generative Defenses and Research Gap 15
2.2 Summary of the Literature Review 18
2.3 Contribution of Research 20

3 Methodology **23**
3.1 Research Design 23
3.1.1 Research Type and Design Overview 23
3.1.2 System Architecture and the Two Generator Routes 26
3.1.3 Threat Model, Trust Assumptions and Privacy Units 29
3.1.4 Experimental Configurations 31
3.1.5 Requirement-Driven Recommendation Rule 34
3.2 Data Collection Methods 37
3.2.1 Dataset 37
3.2.2 Label Harmonisation and Rare Classes 39
3.2.3 Schema Harmonisation and Feature Selection 39
3.2.4 Group-Based Split and Quota Sampling 40
3.2.5 Preprocessing and Encoding 41
3.2.6 Leakage and Label-Reliability Checks 42
3.3 Sampling Data Analysis Techniques 42
3.3.1 Federated Partitioning 42
3.3.2 FedDP-Marginal 45
3.3.3 Privacy Accounting 51
3.3.4 Federated CVAE, DP-SGD and SecAgg^{+} (Baseline) 56
3.3.5 Hyper-parameter Selection and Search Spaces 58
3.3.6 Evaluation Protocols and Metrics 62
3.3.7 Statistical Decision Rule 66
3.3.8 Experimental Environment and Reproducibility 66
3.4 Limitations of the Methodology 68

4 Experimental and Results **72**
4.1 Introduction 72
4.2 Presentation of Data 73
4.2.1 The Data as Used 73
4.2.2 Leakage and Label-Reliability Checks 73

4.2.3 References on Real Data . . . 75
4.2.4 Settings Selected on Validation . . . 75
4.2.5 Consolidated Scorecard . . . 76
4.3 Analysis of Results . . . 80
4.3.1 RQ1: Utility of the Synthetic Data for the IDS . . . 80
4.3.2 RQ2: FedDP-Marginal Against the Federated CVAE . . . 83
4.3.3 RQ3: Overhead, Granularity and Honest Gateways . . . 87
4.3.4 RQ4: Configuration per Deployment Requirement . . . 93
4.4 Interpretation of Results . . . 94
4.5 Comparison with Literature . . . 96
4.5.1 Published Results on the Same Dataset . . . 96
4.5.2 How Much the Protocol Alone Moves a Score . . . 97
4.5.3 Against Federated and Private Data Generation . . . 98
4.6 Implications of the Results . . . 98

5 Discussion **100**
5.1 Restate the Research Problem or Objectives . . . 100
5.2 Summarize Key Findings . . . 100
5.3 Interpretation in a Broader Context . . . 101
5.4 Working Hypotheses and Research Questions . . . 102
5.5 Why the CVAE Was Studied and Why It Remains the Baseline . . . 103
5.6 Limitations . . . 104

6 Conclusion and Future Work **106**
6.1 Conclusion . . . 106
6.2 Future Work . . . 107

A Complete Scorecard **109**

B Effective Privacy Budget by Honest Gateways **113**

C Reproducing the Results **115**

References **116**

List of Figures

3.1 Overall research workflow of PP-FedData. 26
3.2 System architecture with both generator routes. 28
3.3 Three-release protocol of FedDP-Marginal. The coarse cut-points (after Release 1) and the tree $T^{\star}$ (after Release 2) are sent back to the gateways before the next release. 45

4.1 Non-IID partition of the training rows over the five gateways (Dirichlet $\alpha = 0.5$, seed 0). Each cell gives the number of rows of a sub-class at a gateway; the colour gives the share of that sub-class held by the gateway. 74
4.2 Test macro-F1 by configuration and classifier. Upper row: real plus synthetic data (TAug) and the real-data references. Lower row: synthetic data only (TSTR), with the B0 level as a dashed line. MG-ε denotes the main FedDP-Marginal configurations. 78
4.3 TSTR macro-F1 against the privacy budget ε for the main routes (mean over three seeds, with the seed standard deviation), with the reference levels on pooled real data. 79
4.4 Test F1 per class for the real-data reference, the CVAE without DP, and the two generator routes at $\varepsilon = 5$. 85
4.5 TSTR macro-F1 against the total traffic of a run. Filled markers belong to the Pareto front. 88
4.6 Effective ε as a function of the number of honest gateways h, for $K = 5$ and calibration at $t = K$. 89
4.7 Calibrated membership-inference AUC on the released tables, with the 95% bootstrap interval, against the chance level and the heuristic detection threshold. 91

List of Tables

2.1 Unified comparison of the reviewed studies 19

3.1 Research questions and their experimental design 25
3.2 Comparison of the two generator routes 28
3.3 Comparison of protection mechanisms 30
3.4 Experiment matrix of PP-FedData 32
3.5 Robustness and sensitivity branches 33
3.6 Deployment scenarios (inputs only) 36
3.7 Composition of the DoS/DDoS-MQTT-IoT dataset 37
3.8 Data provenance verification 38
3.9 Final feature schema ($D = 59$) 40
3.10 Class quotas of the training, validation, and test splits 41
3.11 Leakage and label-reliability checks 42
3.12 Federated partition parameters 44
3.13 Variants of FedDP-Marginal 51
3.14 Configuration matrix of the baseline and proposed generators 59
3.15 Hyper-parameter search spaces of the model components 61
3.16 Evaluation protocols 63
3.17 Experimental environment specification 67

4.1 Results of the leakage and label-reliability checks (Random Forest with 100 trees, validation split, three seeds) 74
4.2 References trained on the pooled real training split (TRTR) 75
4.3 Settings selected on the validation split 76
4.4 Main configurations under TSTR. An asterisk marks byte counts estimated from the number of cells, because MGl runs in process. 77

4.5 TAug: difference of test macro-F1 against real data only (B0), with the 95% paired bootstrap interval . . . 81
4.6 Local augmentation at each gateway (RF, 15 client models over 5 clients and 3 seeds, mean ± standard deviation) . . . 82
4.7 The classifier trained directly by federated learning, compared with the generator routes at the same protection (TSTR with the MLP on the generator side) . . . 83
4.8 FedDP-Marginal minus the best CVAE route with DP (TSTR on the real test split). All verdicts are "better" under both bootstraps. . . . 84
4.9 Test F1 per class (mean over three seeds; classifier chosen on validation) . . . 85
4.10 Fidelity of the synthetic sets (mean over classes and seeds) . . . 86
4.11 Measured overhead of secure aggregation . . . 87
4.12 TSTR macro-F1 by protected unit (rows kept in brackets when capped) . . . 88
4.13 Effect of the honest-gateway threshold t ($K = 5$) . . . 90
4.14 Membership inference on the released tables of FedDP-Marginal (calibrated attack, 1,000 bootstrap resamples) . . . 91
4.15 Robustness of FedDP-Marginal at $\varepsilon = 5$, and the eleven-class mode . . . 92
4.16 Sensitivity worlds: test macro-F1 (mean ± standard deviation over three seeds) 92
4.17 Recommended configuration per deployment requirement. The value of ε is the one that holds under the trust assumed by the requirement. . . . 93
4.18 Published results on the DoS/DDoS-MQTT-IoT dataset, with the task, input and split of each study . . . 96
4.19 Binary F1 on the closest tasks (mean of three seeds) . . . 97
4.20 Effect of the split and the identifier columns on the real-data RF (three seeds) . 97

5.1 Status of the working hypotheses and research questions . . . 103

A.1 Scorecard of all configurations (TSTR on the real test split, classifier chosen on validation). Configurations marked with a dagger use statistics outside ε and are excluded from the Pareto front. . . . 110

B.1 Effective ε of the distributed configurations as a function of the number of honest gateways h ($K = 5$) . . . 113

C.1 Commands that produce the results of Chapter 4 . . . 115

Abstract

Gateways that relay Message Queuing Telemetry Transport traffic for the Internet of Things see attacks that few sites observe alone, yet their raw packets often cannot be shared to train an intrusion detection system. This thesis measures how far several gateways can jointly produce synthetic training data under formal privacy protection. A framework was built around a federated generator that releases noisy count tables over three rounds and samples packets from a dependency tree, with the noise split across gateways through secure aggregation. A federated conditional variational autoencoder trained with differentially private gradient descent served as the baseline. The study used a public denial-of-service traffic dataset of 39 million packets, split by capture, with five gateways holding skewed class shares, three random seeds, and a paired bootstrap decision rule. Detectors trained only on synthetic data reached a macro-averaged F1 score of 0.366, 0.425 and 0.433 at privacy budgets of 1, 5 and 10, against 0.448 for a random forest on pooled real data. At every budget the generator exceeded the autoencoder by 0.092 to 0.191, using at most 1.33 megabytes per run against 8.4 to 191 megabytes. When real data could be pooled, synthetic data gave no useful gain and simple class rebalancing performed better. A gateway lacking some classes gained about 0.04 from locally added synthetic rows. Protecting a whole connection cost 0.03 to 0.05, protecting a whole capture was not usable, and a budget of 5 grew to 14.2 with one honest gateway. A membership attack on the released tables showed a measurable benefit of the added noise. Generated data help most where pooling is impossible, and the choice of protection should follow the trust and bandwidth requirements of each deployment.

Keywords: intrusion detection, Internet of Things, federated learning, differential privacy, secure aggregation, synthetic data generation, MQTT

List of Abbreviations

Acronym	**Definition**
AUC	Area Under the ROC Curve
BCF	Basic Connect Flooding
C2ST	Classifier Two-Sample Test
CVAE	Conditional Variational Autoencoder
DDoS	Distributed Denial of Service
DoS	Denial of Service
DP	Differential Privacy
DP-SGD	Differentially Private Stochastic Gradient Descent
FedAvg	Federated Averaging
FL	Federated Learning
IDS	Intrusion Detection System
IoT	Internet of Things
IPF	Iterative Proportional Fitting
MI	Mutual Information
MIA	Membership Inference Attack
MLP	Multilayer Perceptron
MPC	Secure Multi-Party Computation
MQTT	Message Queuing Telemetry Transport
non-IID	Not Independent and Identically Distributed
QoS	Quality of Service

RDP	Rényi Differential Privacy
RF	Random Forest
SecAgg	Secure Aggregation
SMOTE	Synthetic Minority Over-sampling Technique
TAug	Train on Augmented data, Test on Real
TCP	Transmission Control Protocol
TRTR	Train on Real, Test on Real
TSTR	Train on Synthetic, Test on Real
zCDP	Zero-Concentrated Differential Privacy

Chapter 1

Introduction

1.1 Background

The Internet of Things (IoT) connects physical objects, sensors, and various devices through communication networks, enabling data-driven services in domains such as healthcare, industrial automation, and smart environments. Alongside the expansion of IoT deployments, the number of connected components exposed to cybersecurity threats increases correspondingly [1]. The Message Queuing Telemetry Transport (MQTT) protocol is commonly used for lightweight publish-subscribe communication in IoT environments. However, MQTT-based systems can become targets for denial-of-service (DoS) and distributed denial-of-service (DDoS) attacks. The DoS/DDoS-MQTT-IoT dataset, collected from a physical IoT testbed, provides normal MQTT traffic alongside ten DoS attack scenarios to facilitate the study of attacks in this context [2]. Intrusion Detection Systems (IDS) monitor network activity to identify traffic or behavior exhibiting malicious signs. Machine learning and deep learning methods have been utilized and researched for this task due to their ability to learn characteristic patterns directly from network traffic. Their effectiveness depends partly on the availability and representativeness of the training data [3]. Federated Learning (FL) allows multiple clients to collaboratively train a shared model while keeping their data local; a central server distributes the global model and aggregates updates from clients across training rounds [4]. This approach is highly suitable when IoT data is distributed across multiple devices or locations, although communication and computational costs remain critical factors to consider in resource-constrained environments [5, 6]. Keeping data local also does not equate to completely eliminating privacy concerns arising from the exchange of model updates [7].

Privacy-preserving synthetic data generation is a potential solution to address the shortage of training data for rare attack classes in IoT networks while still ensuring strict security

guarantees. Rather than relying on deep neural networks that require substantial computational resources and face convergence difficulties on edge devices [8], the probabilistic graphical model paradigm (Probabilistic Graphical Models) approaches the problem by decomposing tabular data into low-dimensional marginal frequency distribution slices (low-dimensional marginals) [9]. Through the Chow-Liu dependency tree structure (Chow-Liu Tree) [10], this approach effectively captures the core correlations between traffic features and attack labels without being affected by the curse of dimensionality. When combined with Differential Privacy (DP) [9] on contingency tables, the system can reconstruct a valid global distribution to generate conditional traffic data, so that downstream intrusion detection systems (IDS) can be trained where raw traffic cannot be pooled [11, 12].

1.2 Problem Statement

Building an effective IDS for IoT networks faces multiple challenges related to data privacy, the representation of attack classes, and the practical costs of the model training process. These challenges are particularly evident when training data is distributed among clients and synthetic attack samples are considered as a supplementary detection solution. The first problem relates to privacy risks during federated training. Although FL avoids the periodic transfer of raw local data, clients must still exchange model updates with the server. Previous studies have demonstrated that shared gradients can leak information about training data in certain attack scenarios [7]. Therefore, sharing model weights instead of raw data cannot be considered a comprehensive privacy protection measure. Depending on the threat model and the type of information to be protected, supplementary mechanisms may be mandatory. The second problem concerns the representation of attack classes in the training data. Class imbalance can bias the learning process towards majority classes, making it difficult to identify classes that account for a low percentage [3]. For the DoS/DDoS detection problem in MQTT, the impact of this issue depends on the dataset version, label mapping, preprocessing pipeline, and data partitioning strategy for clients. Therefore, the class distribution of the dataset after selection and preprocessing must be empirically verified rather than assumed by default. If some attack classes lack data, synthetic samples can supplement training examples. However, merely increasing the number of samples does not prove that the generated data is useful; their actual contribution must be evaluated through data quality metrics and the detection performance of the downstream IDS model.

The third challenge pertains to the hardware constraints of edge devices, the "curse of dimensionality," and the intricate trade-off among privacy protection, data quality, and operational overhead: on the one hand, deploying traditional distributed deep generative models (such as GANs or CVAEs) requires substantial computational resources (GPU/CPU) and large RAM capacity at client nodes, which is infeasible for hardware-constrained edge IoT devices [8]. On the other hand, tabular MQTT network traffic inherently possesses a high-dimensional structure with numerous interdependent attributes. Applying Differential Privacy (DP) directly to this high-dimensional attribute space readily leads to a state-space explosion—reaching up to 10^{19} cells [11]—causing the injected differential privacy noise to completely overwhelm the original data signal and severely degrade the fidelity of the generated distributions [9].

A direct consequence is a severe tug-of-war among three factors: the degree of privacy protection (privacy budget ε), synthetic data utility for downstream IDS models, and computational/operational overhead across the IoT infrastructure. Specifically, tightening privacy guarantees (reducing the budget ε) injects heavy DP noise that destroys the core feature correlations of rare DoS/DDoS attack classes [9, 12]. Conversely, increasing the privacy budget and expanding model capacity to improve the quality of generated data causes computational costs and system runtime to surge drastically [12]. Consequently, finding a tabular data generation solution that simultaneously ensures rigorous privacy guarantees, remains computationally lightweight for edge devices, and preserves critical feature characteristics for rare attack classes remains a major, unresolved technical bottleneck.

1.3 Research Objectives and Research Questions

1.3.1 Objectives

This study proposes PP-FedData—a privacy-preserving federated data generation framework for intrusion detection systems (IDS) in IoT-MQTT networks. The research aims to achieve four primary objectives (O1–O4):

- **Objective O1 – Federated Architecture Construction:** Model each IoT Gateway device as a client within the Federated Learning network (FL Client). Each gateway stores raw packet traffic data locally and only transmits protected statistics to the Aggregation Server under an honest-but-curious threat model.

- **Objective O2 – Marginal-based Federated Generator Development (FedDP-Marginal):** Extend marginal-based privacy-preserving data synthesis methods (such as MST and PrivBayes) to the federated learning environment: each gateway releases noisy per-class count tables across three communication rounds. Noise can be partitioned among gateways and accumulated via the Secure Aggregation mechanism (SecAgg$^+$) (corresponding to Distributed Differential Privacy using discrete Skellam noise), or fully added locally at each gateway (Local DP). Synthetic data records are subsequently sampled from a Chow-Liu dependency tree structure trained on the released marginal statistics tables.

- **Objective O3 – Deep Generative Baseline Implementation:** Implement a federated Conditional Variational Autoencoder (CVAE) model as a deep generative baseline. The CVAE model is trained using the FedAvg algorithm and protected by DP-SGD, SecAgg$^+$, or a combination of both, under identical data partitioning and evaluation protocols to enable a head-to-head empirical comparison against the FedDP-Marginal generator.

- **Objective O4 – Comprehensive Evaluation and Deployment Guidance Formulation:** Comprehensively evaluate all privacy-preserving configurations on a held-out real test set across five dimensions (utility, fidelity, privacy, overhead, and statistical rigor). From these empirical findings, establish a transparent decision rule to map practical deployment constraints (server/client trust assumptions, privacy budget ε, communication bandwidth, prioritized attack classes) into their corresponding optimal configurations.

1.3.2 Research Questions

To fulfill the four aforementioned objectives, this research focuses on addressing four core research questions:

- **RQ1:** When raw network traffic data cannot be centralized, how much IDS utility does the synthetic data generated by the federated generator retain compared to an IDS trained on centralized real data? Does augmenting training sets with synthetic data improve detection performance when real data is already available (either centrally or at individual gateways)?

- **RQ2:** Under the same privacy budget (ε, δ), how does the federated marginal-based generator (FedDP-Marginal) compare to the federated CVAE model trained with DP-SGD regarding downstream utility, detection capabilities on rare attack classes, and network communication overhead?
- **RQ3:** What overhead does each protection mechanism incur under DP, SecAgg$^+$, or their combination, and at what granularity does it provide privacy protection? How do the unit of privacy protection (packet, TCP stream, or capture session) and the number of honest gateways alter privacy guarantees and operational costs?
- **RQ4:** Which protection configuration (combining DP, SecAgg$^+$, noise distribution, and privacy protection unit) should be selected for specific real-world deployment requirements?

1.4 Significance of the Study

This study addresses two fundamental bottlenecks in collaborative intrusion detection across IoT-MQTT networks: (1) rare attack classes are sparsely and non-identically distributed across IoT Gateway devices (Non-IID), and (2) gateways cannot share raw packet traffic data due to privacy concerns.

The study offers three primary scientific and practical contributions:

- **Marginal-Based Distributed Generator (FedDP-Marginal):** Proposes a distributed synthetic data generation solution in which marginal statistics released by individual gateways are simultaneously protected by formal Differential Privacy (DP) and Secure Aggregation (SecAgg$^+$).
- **Controlled Benchmark:** Conducts a direct, head-to-head empirical comparison between the marginal-based generator (FedDP-Marginal) and the deep generative baseline (Federated CVAE) under identical data partitioning schemes, consistent privacy-preserving mechanisms, and a rigorous evaluation protocol.
- **Requirement-Driven Configuration Guide:** Proposes a transparent decision guideline to assist IoT system administrators in selecting the most suitable privacy configuration based on their specific threat model, mutual trust assumptions, and practical resource budgets.

Explicitly Stated Limitations and Privacy Guarantees:

The privacy guarantees in this study are precisely and transparently defined:

- **Differential Privacy (DP)** provides mathematical upper bounds limiting the amount of information that released statistical tables or models can leak regarding a single data record (by default at the packet level, or optionally at the TCP stream or capture session level under specific assumptions). The DP mechanism does not anonymize class labels and does not enforce privacy guarantees on the hyperparameter tuning process conducted on validation data.
- **Secure Aggregation (SecAgg$^+$)** only guarantees the confidentiality of individual gateway updates/statistics with respect to the Aggregation Server; it does not prevent what the final model or generated synthetic data might leak about the aggregate dataset.

1.5 Scope and Limitations

This research is scoped within the following specific experimental and technical boundaries:

- **Dataset and Data Granularity:** Utilizes the DoS/DDoS-MQTT-IoT dataset at the packet level (where each record corresponds to a single network packet and includes six traffic class labels: one Normal class and five DoS/DDoS attack families).
- **Federated Simulation Architecture:** A network of five simulated IoT Gateway devices is deployed under an uneven data distribution across nodes (Dirichlet non-IID partitioning).
- **Generative Models and Privacy Mechanisms:** Evaluates and compares two generators: the primary marginal-based federated generator (FedDP-Marginal) and the deep generative baseline (Federated CVAE). Enforces Differential Privacy (DP) across privacy budget levels $\varepsilon \in \{1, 5, 10\}$ with $\delta = 10^{-5}$, Secure Aggregation (SecAgg$^+$), as well as a hybrid configuration combining both mechanisms.
- **Experimental Infrastructure:** The entire federated training workflow is simulated on a single physical host machine using the Flower framework.

Despite achieving rigorous empirical findings, this study is subject to several scientific limitations:

- **Single-Dataset Reliance:** All empirical evaluation results and configuration selection guidelines are derived exclusively from the DoS/DDoS-MQTT-IoT dataset; therefore, generalizability to other IoT application protocols (such as CoAP or HTTP/2) requires further empirical validation.
- **Packet-Level Independence Assumption:** Packets are modeled independently as tabular records, which does not fully capture the temporal/sequential dependencies inherent in an end-to-end communication session or stream.
- **Default Record-Level Privacy Scope:** The Differential Privacy (DP) guarantee applies by default at the individual record level (a single packet). Extending protection to an entire TCP stream or a full packet capture session represents an advanced configuration option that necessitates a trade-off resulting in degraded synthetic data utility.
- **Privacy Budget ε Exclusions:** The privacy budget ε covers only the released count statistics and trained models. Auxiliary metadata such as attack class labels, global preprocessing statistics, and validation-based hyperparameter tuning fall outside the theoretical guarantee of ε.
- **Simulation Environment:** Conducting the simulation on a centralized physical machine does not fully capture real-world network latency, packet loss rates, or the physical hardware constraints of resource-limited edge IoT devices.
- **Constrained Threat Model:** The study operates under an honest-but-curious threat model across all gateways. Sophisticated adversarial threat scenarios such as data/model poisoning attacks or Byzantine clients are outside the scope of this investigation.

1.6 Thesis Structure

The remainder of this thesis is structured into six chapters as follows:

- **Chapter 1: Introduction:** Outlines the background, problem, objectives, significance, scope, and structure of the study.

- **Chapter 2: Literature Review:** Reviews relevant studies and identifies the research gap and the position of this study in the existing literature.
- **Chapter 3: Methodology:** Details the research design, the dataset and its preparation, the two generators, the privacy mechanisms and accounting, and the evaluation protocol.
- **Chapter 4: Experimental and Results:** Describes the experimental setup and reports synthetic-data, IDS, and system overhead results.
- **Chapter 5: Discussion:** Interprets the findings, examines the observed trade-offs, and discusses the limitations and implications of the study.
- **Chapter 6: Conclusion and Future Work:** Summarizes the conclusions and outlines directions for further research.

Chapter 2

Literature Review

This chapter reviews the research on intrusion detection for IoT and MQTT traffic, federated and differentially private data generation, and secure aggregation, and it positions the present study against that work.

2.1 Review of Previous Studies

2.1.1 Overview of Privacy-Preserving Mechanisms in Federated Learning

Overall Landscape and Scope of Research

The integration of Federated Learning (FL) into Intrusion Detection Systems (IDS) for IoT environments has resolved the issue of raw data centralization, yet the model weight sharing process itself still harbors data leakage risks [13]. To counter this, current studies classify Privacy-Preserving Federated Learning (PPFL) techniques into five main categories: Differential Privacy (DP), Homomorphic Encryption (HE), Secure Multi-Party Computation (SMPC), Trusted Execution Environments (TEE), and Blockchain [13]. Although TEEs provide a secure execution environment and Blockchain ensures transparency, they require specialized hardware or complex consensus mechanisms, which are unsuitable for the constrained resources of edge/IoT devices [14]. Therefore, this study implements DP and SMPC-based secure aggregation (SecAgg^{+}); HE is reviewed as an alternative but not implemented, because of its computational cost on constrained devices [15].

Trusted Execution Environment (TEE)

The Trusted Execution Environment (TEE) provides a completely isolated and secure hardware computing area for processing sensitive data, effectively preserving privacy.

However, this technology is heavily dependent on specialized hardware security mechanisms and verifiable components. This strict hardware requirement makes TEE cumbersome and unsuitable for deployment across lightweight IoT networks comprising massively resource-constrained devices [13].

Blockchain

Blockchain preserves data privacy through decentralized ledgers and cryptographic algorithms that ensure tamper-resistance and eliminate the need for intermediaries. However, this system suffers from fatal performance weaknesses due to low throughput, high latency, and expensive verification costs stemming from continuously growing immutable storage. These severe latency and massive storage overheads make blockchain highly impractical for lightweight, multi-device IoT environments that demand rapid response times [13].

Homomorphic Encryption (HE)

HE enables computations to be performed directly on encrypted data, ensuring absolute security for model updates from clients before server aggregation without the need for decryption [15]. However, the computational cost and communication requirements of HE algorithms are extremely immense and heavy, leading to a surge in system latency [15]. This renders HE less feasible for systems requiring real-time responses and operating under resource constraints, such as IoT-based IDS [15].

Differential Privacy (DP)

Unlike the cumbersomeness of HE, Differential Privacy (DP) offers excellent scalability. The core principle of DP is to mask the contribution of any single data point by adding random noise to the model update at each client [16]. DP is exceptionally lightweight, consumes no extra bandwidth, and easily scales to tens of thousands of participating clients [16]. The tradeoff of this method is that injecting noise into the model inevitably degrades data accuracy, forcing engineers to constantly seek an optimal balance for the privacy budget [16].

Secure Multi-Party Computation (SMPC/SecAgg)

Conversely, Secure Aggregation (SecAgg) protocols based on SMPC comprehensively resolve the weakness of DP. SMPC prevents the server from directly observing individual client updates by ensuring that the server only receives the aggregated model update [17]. Secure aggregation adds no statistical noise; with the fixed-point quantisation used by SecAgg$^+$, the aggregate differs from the plain sum only by a bounded quantisation error [18]. As a trade-off, the original Secure Aggregation protocol requires multiple rounds of complex interactions between clients to exchange secret shares, causing severe bandwidth bottlenecks as the client population grows [19]; however, the subsequent SecAgg$^+$ protocol successfully resolves these overhead issues by replacing the complete communication graph with a k-regular graph, thereby reducing the per-client communication and computation complexity to a polylogarithmic scale [18].

Hybrid Privacy-Preserving Systems

The study by Gutti et al. [20] (integrating DP and SecAgg via the HADA algorithm) demonstrates that combining DP (to prevent inference attacks) and SecAgg (to protect the transmission channel) is a promising trajectory. However, stacking these security layers generates immense complexity in computation and hyperparameter tuning [20]. Therefore, before advancing to hybrid architectures, independently measuring the performance trade-offs between the DP pipeline and the SecAgg pipeline serves as a mandatory stepping stone to evaluate the endurance of the network system.

Distributed Differential Privacy

Distributed DP combines the two families: each client adds only a share of the noise, and secure aggregation reveals only the sum, which carries the full calibrated noise [21, 22]. Because secure aggregation operates on integers modulo a large number, discrete noise is preferred; the sum of independent Skellam variables is again Skellam, which permits exact privacy accounting after modular aggregation [22]. The guarantee assumes that enough clients add their share honestly; with fewer honest clients the effective noise is smaller. Distributed DP fits releases that are a single sum, such as count tables, but not multi-step local DP-SGD, where each client's noise passes through non-linear local training before aggregation.

2.1.2 Data Generation

Overview of Data Generation Techniques

In the field of cybersecurity and Intrusion Detection Systems (IDS) for Internet of Things (IoT) environments, Generative AI (GenAI) techniques for data generation are defined as deep generative paradigms capable of learning the underlying multivariate probability distribution of network traffic, thereby automatically synthesizing novel data samples whose structural attributes and statistical properties closely match genuine traffic distributions [23].

Core problems addressed by data generation techniques:

- **Overcoming Extreme Class Imbalance:** Real-world IoT network traffic inherently exhibits extreme class skewness, where over 90–99% of collected traffic consists of benign activity, while malicious behaviors—especially Low-Rate Stealth Attacks (LRSA) or rare DoS/DDoS variants—represent a negligible fraction. This acute shortage of attack samples severely biases downstream IDS classifiers toward majority classes, resulting in unacceptable false-negative rates on critical threats [23].
- **Beyond Interpolation-Based Oversampling:** Traditional statistical oversampling algorithms such as SMOTE [24] operate by performing linear interpolation between existing minority instances within a localized convex hull. As highlighted by Deng et al. [23], deep generative models surpass these rigid boundaries by capturing the global multivariate distribution, enabling the synthesis of samples in unexplored regions of the feature space to cover edge cases and zero-day attack variants, thereby strengthening model robustness. Furthermore, because SMOTE requires simultaneous access to neighboring minority instances in a unified feature space, it is fundamentally ill-suited for federated environments where an individual edge gateway may hold few or no records of a rare attack class [24].
- **Supporting Data Augmentation and Restructuring:** Systematic survey statistics indicate that up to 89% of studies leverage GenAI specifically for data augmentation and class rebalancing, substantially improving the detection performance and generalization of IDS without exposing sensitive raw telemetry [23].

Primary taxonomies of generative architectures in IDS:

- **Generative Adversarial Networks (GANs):** Architectures structured around an adversarial minimax game between a Generator network that creates synthetic samples

from latent noise and a Discriminator network that evaluates sample authenticity. Due to their capacity to model sharp, high-dimensional distributions, GANs account for an overwhelming 58% of research in generative IoT-IDS [23].

- **Autoencoders and Variational Autoencoders (AEs/VAEs):** Frameworks comprising an Encoder that maps high-dimensional traffic into a low-dimensional latent manifold and a Decoder that reconstructs the input. VAEs enforce continuous probability distribution priors over the latent space, permitting stable random sampling to synthesize realistic minority samples [23].

- **Transformer-Based Generative Models:** Architectures leveraging multi-head self-attention mechanisms to capture complex contextual correlations and sequential packet dependencies across temporal network traffic flows [23].

Analysis of Strengths and Weaknesses of Techniques in Existing Studies

Although data generation techniques provide substantial potential for IoT-IDS, critical examination of existing literature reveals distinct computational bottlenecks, training instabilities, and privacy trade-offs when deployed in decentralized Federated Learning (FL) settings [23]:

- **Heavy Generative Denoising Techniques (Tabular Diffusion / Tab-DDPM):** Manzano et al. [14] integrated tabular diffusion models (Tab-DDPM) directly at local FL clients to synthesize minority samples on the BoT-IoT benchmark. While Tab-DDPM achieves exceptional sample fidelity and distribution alignment, the reverse diffusion process requires iterative denoising over hundreds of sequential steps. On edge-tier testbeds (1 CPU core, 2 GB RAM), generating a single synthetic sample requires approximately 32 seconds [14]. Such extreme computational latency is completely unviable for line-rate packet inspection in edge IoT environments.

- **Improved Policy-Guided GANs:** Yang et al. [25] employed an improved GAN guided by reinforcement learning to augment training sets for DDoS detection. While ablation experiments confirm that removing synthetic augmentation causes a noticeable drop in detection performance, the multi-component architecture requires up to 2.726 seconds per batch and consumes 19.6 GB of GPU memory on the KDDcup99 dataset [25].

Furthermore, the study did not isolate standalone generator inference latency, obscuring the true computational efficiency of the generator on edge hardware.

- **Survey Insights on Pure GAN Instability:** In their survey, Deng et al. [23] highlight that while GANs produce high-fidelity samples, the adversarial training dynamics are notoriously brittle and susceptible to mode collapse—wherein the generator repeatedly outputs a narrow set of duplicate patterns, wasting node compute without contributing meaningful diversity.

- **Random Resampling Techniques (Bootstrapping / Oversampling):** Jacob and Sultana [26] utilized bootstrapping (random resampling with replacement) at local nodes to balance the BoT-IoT and CSE-CIC-IDS2018 datasets. While computationally trivial and easily executable on microcontrollers, bootstrapping merely duplicates existing data points without expanding feature diversity. This encourages classifier overfitting and fails to generalize against evolving attack variants or concept drift [23, 26].

- **Cloud-Based Centralized Data Generation Techniques (Cloud-Based VAE-GAN):** Anandhi and Srikanth [27] proposed the SSAIM framework utilizing a hybrid VAE-GAN at the cloud coordinator to generate 51,200 Low-Rate Stealth Attack (LRSA) samples for the IoTID20 dataset, raising attack recall from 62% to 98%. However, placing data synthesis in the cloud violates the decentralized privacy principle of FL. Additionally, compressing the heavy VAE-GAN model via fixed-point quantization for transmission back to edge devices demonstrated that quantization below 6-bit (e.g., 4-bit) caused a severe degradation in detection sensitivity [27].

- **Federated Lightweight Edge GANs (TinyGAN in FL):** Durga et al. [28] deployed TinyGAN within a federated framework for IoMT malware detection, reporting 99.30% accuracy after 20 rounds. However, to stabilize GAN training, all malware classes representing less than 0.5% of the dataset were discarded, thereby sidestepping the core challenge of extreme class imbalance. Moreover, the study omitted standalone sample generation throughput measurements, leaving its actual computational advantage on constrained edge hardware unproven [28].

2.1.3 Privacy-Preserving Generative Defenses and Research Gap

Privacy-Utility Dilemma in Federated Generative IDS

Federated intrusion detection systems (IDS) on edge networks must protect client telemetry while coping with severe minority-attack-class imbalance and non-IID traffic distributions. Differential privacy (DP) and cryptographic protocols bound gradient inversion and membership inference risks [29] but do not correct class skew. Augmenting under-represented attack classes with federated generative models mitigates this imbalance, yet generative architectures may memorize training records and expose sensitive network traces through parameter exchanges or synthesized flows [29]. Edge defenses must therefore limit the utility loss of synthetic data while providing formal client-side privacy guarantees.

Existing privacy-preserving generative IDS frameworks use cryptographic primitives, DP, or both. Cryptographic mechanisms, such as secure multi-party computation (MPC) and threshold secret sharing, preserve numerical precision without perturbation error [29]. However, secure aggregation of high-dimensional neural parameters requires iterative multi-round communication, which burdens resource-constrained IoT edge devices [29].

Empirical Generative PPFL-IDS and Cryptographic Limitations

A literature search of IEEE Xplore and SpringerLink (2022–2026) for studies combining federated learning, intrusion detection, and generative models (GAN or VAE) identified five empirical implementations, all from IEEE [30–34]. None applies cryptographic secure aggregation to uplink generator updates. Salim et al. [32] (global VAE aggregation), Jiao et al. [33] (local CGAN gradients), and Shettar et al. [34] (central aggregator with moments-accountant clipping at $\varepsilon = 4.8$ and $\delta = 10^{-5}$) rely on DP alone. He et al. [31] combine local gradient noise with blockchain Proof-of-Authority consensus across edge servers, which ensures aggregation integrity and auditability rather than parameter confidentiality.

Yao et al. [30] use a hybrid design: proactive dynamic (t,n)-threshold secret sharing with $t = 4$ and $n = 5$ protects only downlink synthetic broadcasts, whereas uplink client gradients rely on Gaussian DP. They report execution measurements for key distribution and sample reconstruction (Table VI), but secret sharing covers downlink tabular samples, not full model-parameter aggregation. Outside the primary search, Hamouda et al. [35] likewise omit secure aggregation and identify cryptographic model protection as an open challenge. Because

none of these works benchmarks cryptographic aggregation against DP on model weights, their use of DP reflects a design choice rather than a measured trade-off.

The effect of DP perturbation on downstream detection depends on the privacy budget ε, gradient clipping threshold C, and minority-class sample density, yet existing studies do not isolate these factors. Utility can remain high at a moderate budget or with large synthetic volumes: Shettar et al. [34] report a 96.8% detection F1-score under central DP ($\varepsilon = 4.8$), and Hamouda et al. [35] observe a 0.25-percentage-point accuracy reduction at $\varepsilon = 1.0$ when applying DP-SGD to the downstream global classifier with 50,000 synthetic samples per class. Generative augmentation can also raise rare-class detection from sparse baselines: He et al. [31] report that CGAN augmentation improved Infiltration F1-score from 33.33% to as high as 98.01%, starting from 29 baseline training samples. However, Hamouda et al. [35] applied DP to the classifier rather than the generator, and neither He et al. [31] nor Jiao et al. [33] report formal numerical (ε, δ) budgets. Because generator-side DP must spread privacy noise across iterative training steps, how strict bounds ($\varepsilon \leq 1.0$) interact with extreme class scarcity in federated generative architectures remains an open empirical question.

Marginal-Based Tabular Synthesis vs. Deep Generative Models

Marginal-based tabular synthesis offers an alternative to deep generative models. Instead of optimizing non-convex neural network weights by backpropagation, marginal synthesizers select and measure low-order contingency tables and reconstruct the data distribution with probabilistic graphical models. Beyond network IDS research, a cross-domain search of the ACM Digital Library, PMLR, and arXiv identified distributed extensions: Maddock et al. [36] introduce FLAIM, which adapts the Adaptive and Iterative Mechanism (AIM) to distributed clients under zero-concentrated differential privacy (ρ-zCDP) and compares DP with secure multi-party computation (DistAIM); Pentyala et al. [37] present CaPS, a "DP-in-MPC" framework in which secret sharing protects marginal selection and measurement; and Li et al. [38] propose HeteroFedSyn, which compresses two-way marginal dependencies by random projection under distributed perturbation.

Marginal synthesizers avoid the adversarial minimax training instability of GANs but carry their own privacy-utility trade-offs. Iterative selection-and-measurement algorithms (e.g., AIM and Private-PGM) divide the total privacy budget ε across rounds, and distributing measurements across federated clients compounds perturbation noise [36, 37]. In sparse discrete categories with small contingency cell counts, additive Laplace or Gaussian noise

yields a low signal-to-noise ratio. Restricting measurements to low-order marginals (typically two-way dependencies [38]) truncates higher-order multivariate correlations, and continuous flow variables require discretization, which introduces binning artifacts.

Federated marginal generators have so far been evaluated mainly on general-purpose tabular benchmarks from demographic, financial, and biomedical domains (e.g., Adult, Census, Credit Card Fraud, COMPAS, Diabetes) [36–38]. FLAIM [36] also used the legacy DARPA Intrusion Detection benchmark but evaluated only workload error and negative log-likelihood, not downstream minority-class intrusion classification. Decentralized tabular GANs with cryptographic primitives also exist: Duan et al. [39] combine Paillier additively homomorphic encryption for global multimodal distribution encoding with DP during GAN training, but evaluate statistical utility and membership inference resistance on legacy tabular data (KDD Cup 99), not modern protocol telemetry.

IoT-MQTT Protocol Telemetry and Working Hypotheses

IoT-MQTT telemetry combines structured, low-cardinality discrete protocol fields (e.g., message types, QoS levels, retain flags) with continuous flow metrics (e.g., payload lengths, burst durations). Sparse discrete feature combinations form low-count contingency cells, where both generative families risk degradation under DP noise, making this a suitable setting for examining these trade-offs. Two working hypotheses follow. **H1:** under generator-side DP at a strict budget ($\varepsilon = 1$), a federated marginal synthesizer reaches a higher class-level F1 than a federated deep generative model for the attack classes that are defined by low-order discrete header fields (Invalid Subscription Flooding and Connect Flooding with WILL payload). **H2:** the reverse holds for the volumetric attacks (Basic Connect Flooding and SYN TCP Flooding), whose behaviour depends on several continuous timing and length attributes together. Both are tested over three random seeds at matched (ε, δ) budgets. Each mechanism is accounted through Rényi or zero-concentrated differential privacy and converted to (ε, δ)-DP (Section 3.3.3). Differences are read with the decision rule of Section 3.3.7, a paired bootstrap interval that must exclude zero together with a margin above the seed-to-seed spread.

Identified Research Gaps

To the best of our knowledge, the IEEE, Springer, ACM, PMLR, and arXiv searches described above indicate three distinct research gaps:

- **Marginal Synthesis for Protocol Telemetry:** Deep generative models have been evaluated on MQTT-bearing datasets (e.g., Hamouda et al. [35], federated conditional GAN on Edge-IIoTset), but no prior study evaluates a federated marginal-based generator on modern IoT-MQTT telemetry or examines how mixed discrete-continuous features affect marginal synthesis relative to deep models.
- **Controlled Architectural Comparison:** No empirical study compares a federated marginal-based synthesizer with a federated deep generative model (e.g., CGAN, VAE) under identical non-IID client partitions, minority-attack imbalance ratios, and calibrated generator-side privacy budgets (ε, δ).
- **Downstream Detection Trade-offs:** Prior marginal studies compared DP with cryptographic aggregation on workload error, negative log-likelihood, runtime, and communication bandwidth [36, 37], and decentralized GANs were evaluated for statistical utility and membership inference resistance [39]. No study quantifies how choosing between DP and cryptographic secure aggregation affects downstream minority-class intrusion detection under matched federated edge conditions.

This study addresses these gaps by evaluating communication overhead, synthetic sample fidelity, and downstream minority-class intrusion detection for marginal and deep generative models under both DP and cryptographic secure aggregation on edge IoT networks.

2.2 Summary of the Literature Review

The reviewed literature establishes federated learning as a promising approach to IoT intrusion detection while identifying persistent limitations in data representation, privacy protection, and empirical evaluation. As summarised in Table 2.1, several studies address non-IID data distributions or incorporate data augmentation, but these features alone do not demonstrate effective detection of under-represented attack classes. Class imbalance and inter-client heterogeneity therefore remain central concerns. Two generator families address them under privacy constraints, deep generative models trained with DP-SGD and marginal-based synthesis, but they have been evaluated separately and rarely on IoT traffic; whether either adds detection value must be verified through controlled experiments.

The comparative synthesis in Table 2.1 shows that privacy preservation and synthetic data generation have increasingly converged across distributed and federated environments.

Table 2.1: Unified comparison of the reviewed studies

Reference	Ref.	FL	IoT Dataset	Non-IID	Data Synthesis	DP	SMPC	DP vs. SecAgg Trade-off	Marginal-based DP synthesis	Distributed DP
Vyas et al.	[13]	✓	×	×	×	✓	×	×	×	×
Manzano et al.	[14]	✓	✓	✓	✓	×	×	×	×	×
Nakayiza et al.	[15]	✓	✓	×	×	×	×	×	×	×
Ruzafa-Alcázar et al.	[16]	✓	✓	✓	×	✓	×	×	×	×
Bessaa et al.	[17]	✓	✓	✓	×	×	✓	×	×	×
Bonawitz et al.	[19]	✓	×	×	×	×	✓	×	×	×
Bell et al.	[18]	✓	×	×	×	×	✓	×	×	×
Gutti et al.	[20]	✓	✓	✓	×	✓	✓	×	×	×
Kairouz et al.	[21]	✓	×	✓	×	✓	✓	×	×	✓
Agarwal et al.	[22]	✓	×	✓	×	✓	✓	×	×	✓
Deng et al.	[23]	×	×	×	×	×	×	×	×	×
Chawla et al.	[24]	×	×	×	✓	×	×	×	×	×
Yang et al.	[25]	×	✓	×	✓	×	×	×	×	×
Jacob & Sultana	[26]	✓	✓	✓	✓	×	×	×	×	×
Anandhi & Srikanth	[27]	✓	✓	✓	✓	×	✓	×	×	×
Durga et al.	[28]	✓	✓	✓	✓	×	×	×	×	×
Cunha Neto et al.	[29]	✓	×	×	×	×	×	×	×	×
Yao et al.	[30]	✓	✓	✓	✓	✓	✓	×	×	×
He et al.	[31]	✓	×	×	✓	✓	×	×	×	×
Salim et al.	[32]	✓	✓	✓	✓	✓	×	×	×	×
Jiao et al.	[33]	✓	✓	✓	✓	✓	×	×	×	×
Shettar et al.	[34]	✓	✓	✓	✓	✓	×	×	×	×
Hamouda et al.	[35]	✓	✓	✓	✓	✓	×	×	×	×
Maddock et al.	[36]	✓	×	✓	✓	✓	✓	×	✓	×
Pentyala et al.	[37]	×	×	×	✓	✓	✓	×	✓	×
Li et al.	[38]	✓	×	✓	✓	✓	×	×	✓	×
Duan et al.	[39]	✓	×	✓	✓	✓	×	×	×	×
Ours (PP-FedData)	–	✓	✓	✓	✓	✓	✓	✓	✓	✓

On the one hand, several intrusion detection frameworks employ deep generative models, such as generative adversarial networks and variational autoencoders, to address severe class imbalance and zero-day threat variants in IoT, UAV, and satellite networks (e.g., [30–35]). On the other hand, recent advances in tabular data synthesis, including FLAIM [36], CaPS [37], and HeteroFedSyn [38], have extended marginal-based synthesis to distributed or federated settings using differential privacy (DP) and secure multi-party computation (SMPC). However, these marginal-based synthesisers have been validated almost exclusively on general-purpose tabular benchmarks (such as Adult, Census, and Credit Card Fraud) rather than on streaming network telemetry, leaving their efficacy in packet-level IoT intrusion detection unexamined.

More critically, an inspection of Table 2.1 highlights two persistent research gaps in the literature:

- **Absence of a controlled DP versus SecAgg trade-off analysis:** Cryptographic protection (SMPC and SecAgg/SecAgg$^+$) preserves exact parameter sums without statistical noise but incurs communication and round complexity. Conversely, perturbation-based protection (DP) offers formal mathematical bounds against inference attacks at the expense of utility degradation. While several hybrid systems incorporate aspects of both paradigms (e.g., [20, 30, 36]), none provides an explicit, head-to-head empirical trade-off analysis comparing a purely cryptographic pipeline against a perturbation-based pipeline under identical IoT network conditions, non-IID partitions, and threat models.

- **Untapped distributed DP for federated marginal synthesis in IoT:** Although discrete distributed DP mechanisms, such as discrete Gaussian [21] and Skellam [22] noise, permit noise shares to be split across clients and reconstructed exactly over modular secure aggregation, existing federated intrusion detection architectures continue to rely on centralised DP noise injection or local DP-SGD. Furthermore, deep generative approaches and marginal-based mechanisms have developed in isolation, without a unified benchmark to determine whether lightweight count-table synthesis retains sufficient utility compared with complex deep generative baselines.

To the best of our knowledge, based on a non-systematic search of the literature, no prior study has jointly addressed these limitations within a unified federated framework. This observation motivates PP-FedData, which addresses the gap by implementing: (i) a federated marginal-based generator (FedDP-Marginal) tailored to IoT-MQTT network traffic; (ii) an end-to-end distributed differential privacy mechanism using integer Skellam noise aggregated through Flower's SecAgg$^+$ protocol, alongside a federated CVAE deep generative baseline; and (iii) a controlled evaluation across five dimensions: downstream IDS utility, synthetic-data fidelity, formal privacy accounting, operational overhead, and statistical rigour.

2.3 Contribution of Research

To address the limitations and research gaps identified in Section 2.2 and synthesised in Table 2.1, namely the lack of marginal-based private synthesis for IoT telemetry, the absence of distributed differential privacy for tabular count data, and the lack of an explicit empirical

DP-versus-SecAgg trade-off benchmark, this research proposes PP-FedData and makes three primary contributions.

G1. A Federated Marginal-Based Generator for Non-IID IoT-MQTT Traffic (FedDP-Marginal), with a Deep Generative Baseline

Table 2.1 shows that existing marginal-based synthesisers (e.g., FLAIM [36], CaPS [37], HeteroFedSyn [38]) have only been evaluated on generic tabular datasets. This study develops FedDP-Marginal, tailored to packet-level IoT-MQTT network traffic. Across three communication rounds, the framework releases noisy per-class 1-way marginal tables, global 2-way tables from which a single Chow–Liu dependency tree is learned [10], and class-conditional tables along that tree; synthetic packets are then sampled class by class. It adapts central differential privacy synthesis mechanisms (MST [40], PrivBayes [9]) to a federation of distributed gateways and adds post-processing heuristics (fusion of independent marginal estimates and iterative proportional fitting) that consume no additional privacy budget. A federated conditional variational autoencoder (CVAE) [41] trained with FedAvg [4] serves as the deep generative baseline under identical data partitions, downstream classifiers, and evaluation criteria, which establishes whether structural marginal modelling offers operational advantages over deep generative alternatives.

G2. Distributed Differential Privacy through a Real Secure Aggregation Implementation, with Explicit Trust Assumptions

In contrast to the federated intrusion detection architectures in Table 2.1, which inject heuristic local perturbation without cryptographic aggregation or rely solely on centralised server-side noise, PP-FedData integrates formal distributed DP with a real cryptographic protocol. For FedDP-Marginal, integer Skellam noise [22] is split across the edge gateways, and the noisy tables are summed with Flower's SecAgg^{+} protocol [18], so that the honest-but-curious aggregator observes only the noisy sum. The cumulative privacy budget ε is accounted for with the Rényi differential privacy (RDP) [42] bound established by Agarwal et al. [22] and is reported explicitly as a function of the number of honest gateways. The framework is configurable in two respects: (i) a threshold t of required honest gateways, and (ii) the protection unit, which is record-level (one packet) by default and optionally stream-level or capture-level when each unit is held by a single gateway and capped at m rows.

A local-DP variant is provided for clients that trust no other party. For the deep CVAE baseline, differentially private stochastic gradient descent (DP-SGD) [43] with per-example gradient clipping and Rényi accounting is combined with fixed-point quantised SecAgg^{+}.

G3. A Controlled Five-Dimensional Empirical Evaluation and a Requirement-Driven Configuration Guide

To resolve the fragmentation observed in prior studies, all configurations (FedDP-Marginal, the CVAE baseline, and non-private variants) are evaluated against the same held-out test split of real IoT-MQTT traffic along five dimensions:

- **Downstream IDS utility:** macro-F1, binary F1, per-class precision and recall, and minority-attack recall, for IDSs trained on purely synthetic data (TSTR), on real data augmented with synthetic data (TAug), and with per-gateway local augmentation.
- **Fidelity:** Wasserstein distance, Jensen–Shannon divergence, correlation distance, and classifier two-sample tests (C2ST), which assess the preservation of multi-attribute dependencies.
- **Privacy:** formal (ε, δ) budgets recomputed independently with the privacy accountants, together with empirical membership inference attacks (MIA) that use positive controls.
- **Operational overhead:** communication volume counted on the federated grid (bytes per round and per run), number of rounds, and compute time per round.
- **Statistical rigour:** three random seeds, paired bootstrap confidence intervals, a fixed decision rule, and stream-level bootstrap sensitivity checks.

Finally, building on these empirical findings, the research derives a transparent decision rule that filters valid configurations by operational requirements (server trust, gateway collusion risk, bandwidth limits, and target attack priority) and selects the most suitable operating point from their empirical Pareto front.

Chapter 3

Methodology

This chapter sets out how the study was designed and carried out: the design and threat model, the data and how they were prepared, the two generator routes with their privacy accounting, the evaluation protocols, and the decision rule used to read every difference. It holds design values only; every measured score appears in Chapter 4.

3.1 Research Design

3.1.1 Research Type and Design Overview

This study is a quantitative, controlled experimental study carried out on one published network-traffic dataset, DoS/DDoS-MQTT-IoT [2]. In the terminology of the FPT thesis guide, it combines Experiments with Archival Research: the independent variables (generator, privacy mechanism, privacy budget) are manipulated by the researchers, while the traffic itself is existing recorded data rather than data collected specifically for this study.

Research Object

The object of study is a framework, PP-FedData, that lets several IoT/MQTT gateways jointly produce synthetic network-traffic records without any gateway releasing its raw packets, so that an intrusion detection system (IDS) can be trained on data that no single party could pool. The framework is evaluated along four dimensions: the utility of the synthetic data for the IDS, the privacy guarantee attached to the released artefacts, the communication and computation overhead, and the fit of each configuration to a stated deployment requirement.

Experimental Control

Internal validity rests on holding everything except the studied factors fixed. Every configuration in the experiment matrix (Section 3.1.4) is run with:

- the same train/validation/test split, formed by capture group, with a real test split that is locked and never used for tuning (train 88,500, validation 12,000, test 30,000 rows; class quotas fixed in the configuration);
- the same non-IID partition of the training rows over $K = 5$ clients, drawn from a Dirichlet distribution with concentration $\alpha = 0.5$ [44] and fixed by the seed;
- the same preprocessing ($\log(1+x)$ transformation of skewed columns, standardisation, clipping at $\pm 5\sigma$, one-hot encoding of categorical columns), which turns the harmonised packet attributes into 59 model input columns;
- the same downstream classifiers: a Random Forest [45] with 200 trees and a multilayer perceptron with hidden layers $[128, 64]$;
- the same three random seeds $\{0, 1, 2\}$;
- the same real test split for every reported score.

Under this design, differences between configurations are attributable to the generator and to the protection mechanism, subject to the limits listed in Section 3.4. The design does not claim that extraneous influences are eliminated: hyper-parameter selection, the choice of capture groups, and the number of seeds all remain sources of variation, and Section 3.4 states how far each is covered.

Research Questions and Design Mapping

Table 3.1 maps each research question to the comparison, the evaluation protocol, and the measures used to answer it. The protocols are defined in Section 3.3.6. Training-and-testing notation follows common usage: TRTR trains and tests on real data; TSTR trains on synthetic data and tests on real data; TAug trains on real plus synthetic data; TAugR adds synthetic rows only for the rare classes; local augmentation supplements each client's own classifier with synthetic rows for classes it lacks.

Table 3.1: Research questions and their experimental design

RQ	Question (short form)	Compared configurations	Protocols	Primary measures
RQ1	Does synthetic data improve an IDS relative to limited real data?	B0, B1a, B1b (pooled real references) against B3, M2 and MG (synthetic)	TRTR, TSTR, TAug, TAugR, local augmentation	Macro-F1 over six classes; mean recall of the rare classes
RQ2	How do the marginal-based and deep generator routes compare under the same DP budget?	M1, M1o, M3, M3o (federated CVAE with DP) against MG, at $\varepsilon \in \{1, 5, 10\}$	TSTR	Macro-F1; rare-class recall; MB per round; number of rounds
RQ3	What does privacy cost in utility and in resources?	B3 against M1 (cost of DP); B3 against M2 (cost of SecAgg); MGs against MGl; suffixes `-strm`, `-cap`, `-t`	TSTR; privacy measurements; cost monitoring	ε as a function of honest clients h; membership-inference AUC; bytes; time
RQ4	Which configuration suits which deployment requirement?	All valid rows of the scorecard	Requirement-driven rule (Section 3.1.5)	Satisfaction of the requirement vector

The rare classes are DELAYED, SYN, INVALID, and WILL, the four attack families whose training quota is at most 5,000 rows. The macro-F1 is the unweighted mean of the per-class F1 over the six classes; binary F1 separates NORMAL from the merged attack classes.

Overall Workflow

The research proceeds in five stages (Figure 3.1): data preparation with leakage checks; partitioning over clients; training of the generator routes; multi-protocol evaluation; and aggregation into a scorecard from which recommendations are drawn.

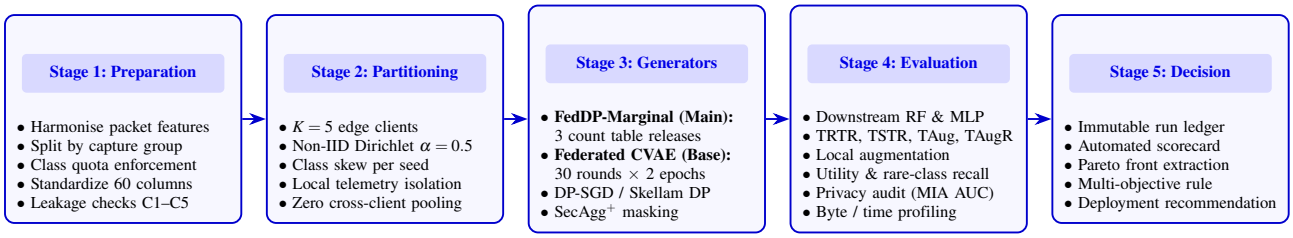


Figure 3.1: Overall research workflow of PP-FedData.

Evolution of the Design

The study was originally designed around a federated conditional variational autoencoder (CVAE) [41], which is the premise of the project. After a first evaluation round, the framework adopted a marginal-based generator, FedDP-Marginal, as its primary generator and retained the CVAE as a baseline. The decision is recorded with its date and rationale in the project's deviation log (`docs/SPEC_DEVIATIONS.md`, entries O1.9 and O2.3); Chapter 4 presents the evidence and Chapter 5 discusses it. The CVAE configurations remain in the matrix so that the comparison in RQ2 is made under identical federation, privacy, and secure-aggregation conditions.

3.1.2 System Architecture and the Two Generator Routes

Roles

The system involves three primary roles (Figure 3.2):

- **Gateways (federated learning clients):** The main study uses $K = 5$ gateways. Each gateway holds the traffic it observed and is trusted with its own data. Because the partition is non-IID, some gateways have few or no examples of some attack classes.
- **Aggregation server:** The server coordinates the rounds, combines client contributions and, after training or table release, produces the synthetic data. It is modelled as honest-but-curious (Section 3.1.3).
- **Downstream IDS:** A Random Forest and a multilayer perceptron are trained on synthetic data (TSTR) or on real plus synthetic data (TAug, TAugR, local augmentation) and are evaluated on the real test split.

Where Synthetic Data is Generated

Synthetic records are generated at the server, from the global model (CVAE decoder) or from the released global tables (marginal counts and dependency tree), by sampling conditionally on the class label. They are not generated at the gateways. The sampling functions are implemented in `models/generate.py` (CVAE) and `models/marginal.py` (FedDP-Marginal).

What the Server Sees

The visibility of client contributions depends on the configuration and must not be stated in general terms:

- M2, M3, MGs, and MGr use Flower's SecAgg$^+$ protocol [18, 46]: the server receives masked vectors and can recover only their sum.
- B3 and M1 use plain FedAvg [4]: the server sees each client's model update; in M1 the update already carries noise added by DP-SGD.
- MGl (local DP): each client adds the full noise to its own tables and sends them without secure aggregation, so the server sees each client's noisy tables.
- MGd and MGb simulate the distributed sum inside one process; they do not pass through the SecAgg$^+$ protocol.

The Two Generator Routes

Table 3.2 contrasts the two routes. FedDP-Marginal exchanges counts of discrete attribute combinations in three releases; the CVAE exchanges continuous model weights over many FedAvg rounds. The difference matters for privacy accounting. Each marginal release is a single additive aggregate with bounded sensitivity, so the calibrated noise can be divided among the clients and the sum computed through secure aggregation. DP-SGD [43], by contrast, performs many local non-linear steps per round, and noise added by one client passes through the other clients' training before aggregation; the project therefore does not claim distributed DP for the CVAE route (`SPEC_DEVIATIONS.md`, O2.1).

For the FedDP-Marginal route, the sum is carried by a patched client path: stock Flower SecAgg$^+$ sends masked vectors as 64-bit integers, so the project casts the masked vector to

Table 3.2: Comparison of the two generator routes

Property	FedDP-Marginal (main generator)	Federated CVAE (baseline)
Quantity shared	Count tables (summed over clients) in three releases: class-conditional 1-way marginals, pairwise tables, tables for dependency tree edges	Model weights, 30 FedAvg rounds with 2 local epochs (default configuration)
Model structure	Chow–Liu dependency tree [10]; optional second-order Bayesian network (PrivBayes-style [9], MGb)	Conditional VAE [41] with an encoder and a block-wise decoder
Where DP noise is added	On each client's tables, once per release (Skellam [22] or Gaussian share)	On each client's per-example gradients at every local step (DP-SGD via Opacus [47])
Secure aggregation	Flower SecAgg$^+$ on exact integers (uint32, arithmetic modulo 2^{32})	Flower SecAgg$^+$ on fixed-point quantised weights (2^{22} quantisation levels, clipping range 16)
Distributed DP	Valid: each release is one additive aggregate	Not claimed: multiple non-linear local steps
Main code	`models/marginal.py`, `fl/mg_app.py`, `group_dp.py`	`models/cvae.py`, `fl/core.py`, `fl/app.py`, `fl/dp_utils.py`

uint32 after the final modular reduction and lets the server's unsigned addition wrap, which equals addition modulo 2^{32} (`SPEC_DEVIATIONS.md`, O2.2). The installed Flower package is not modified; the patch lives in the project process.

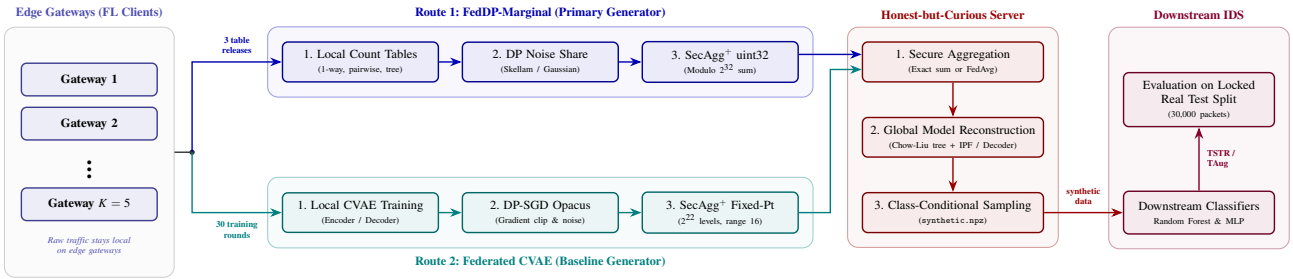


Figure 3.2: System architecture with both generator routes.

3.1.3 Threat Model, Trust Assumptions and Privacy Units

Adversary and Trust Assumptions

The aggregation server is assumed to be honest-but-curious, also called semi-honest [48]: it follows the protocol but may analyse every message it receives. Gateways hold their own traffic and are trusted with it. The study considers four trust settings, which differ in how much the privacy guarantee depends on other gateways:

- **Base setting:** All K gateways follow the protocol and each adds its designated share of the noise ($1/K$ of the variance in the distributed variants).
- h **honest gateways:** The guarantee is also reported when only $h \in \{1, \ldots, K\}$ gateways add their share; the remaining gateways may collude with the server or drop out. The reported ε then grows as h decreases. For the distributed FedDP-Marginal variant, the project reports ε for every h; Chapter 4 gives the values.
- **Threshold** t**:** The per-client noise is calibrated so that the target (ε, δ) holds whenever at least t gateways contribute their share. Each client then adds $1/t$ of the calibrated variance. The default is $t = K$.
- **Local DP (MGl):** Each gateway adds the full noise itself, so no other gateway needs to be trusted [49]; the total noise variance in the aggregate grows with the number of clients.

Out of Scope

Active attacks by malicious gateways, namely data poisoning, model poisoning, and Byzantine behaviour, are outside the threat model. Statements in this thesis about privacy do not extend to them.

Roles of Secure Aggregation and Differential Privacy

The two mechanisms address different threats and are not interchangeable (Table 3.3).

Table 3.3: Comparison of protection mechanisms

Aspect	Secure Aggregation (SecAgg$^+$)	Differential Privacy (DP)
Threat addressed	Server inspecting an individual client's update or table	Inference about one data unit from what is released (aggregate, model or synthetic data)
Guarantee	The server recovers only the sum of masked contributions [18, 19]	(ε, δ)-DP [50] with $\delta = 10^{-5}$: the release distribution changes by at most a factor e^{ε}, up to δ, when one unit is added or removed
What it does not do	It does not limit what the aggregate itself reveals, so it receives no differential-privacy credit	It does not hide the individual update from the server unless combined with secure aggregation or applied locally

Privacy Unit

The default unit is the record, that is, one network packet. An ε at record level protects the presence of one packet. For FedDP-Marginal, two larger units are available: the TCP stream (suffix `-strm{m}`) and the capture (suffix `-cap{m}`). Their guarantee holds only under two conditions:

1. each unit is held entirely by one client (the partition is made by unit, not by row);
2. each client uses at most m rows of each unit.

When both hold, a release of T tables has sensitivities $\Delta_1 = m \cdot T$ in ℓ_1 and $\Delta_2 = m\sqrt{T}$ in ℓ_2 (`group_dp.py`), and the noise is calibrated to these values. The CVAE routes keep record-level ε.

Why the Unit Must Be Stated

Packets of one stream, and packets of one capture, are correlated. Under record-level ε, a set of k correlated records from the same source is protected only through group privacy, which weakens the guarantee to roughly $k\varepsilon$ with a correspondingly larger δ [51]. A statement of ε without its unit therefore says little about whether an attack session as a whole is concealed. The unit statistics of the data are: the training split contains 77,106 TCP streams and 147 capture groups, and a capture contains up to 1,277 rows.

Public Side Information (Not Covered by ε)

The following are treated as known to every party and are outside the accounting:

- the class labels and the number of examples per class;
- client sizes n_i, where they are transmitted in clear (Flower's `num_examples` in the CVAE routes);
- the normalisation statistics, the $\log(1+x)$ choices, the category lists, and the mask of unused categories, computed centrally on the pooled training split before partitioning;
- the hyper-parameter selection, performed on the real validation split.

In the FedDP-Marginal route the client reports `num_examples` as a fixed public constant, so client sizes do not travel through that field (`SPEC_DEVIATIONS.md`, O2.3).

Standard Formulation for the Thesis

We assume an honest-but-curious aggregation server that follows the protocol but may analyse every message it receives. Gateways hold their own traffic and are trusted with it. In the base setting every gateway follows the protocol and adds its share of the noise; we also report the guarantee when only h of the K gateways do so, and an optional threshold t calibrates the noise so that the target ε holds whenever at least t gateways add their share (the others may collude with the server or drop out). In the local-DP variant each gateway adds the full noise, so no other gateway needs to be trusted. Malicious (poisoning or Byzantine) gateways are out of scope.

Secure aggregation hides each gateway's contribution from the server, which learns only the sum; it does not limit what the aggregate reveals and receives no privacy credit. Differential privacy bounds what the released tables, model, or synthetic data reveal about one record (one packet) with $\delta = 10^{-5}$. Class labels, class counts, and client sizes are treated as public side information and are not protected.

3.1.4 Experimental Configurations

Experiment Matrix

Table 3.4 lists the configurations. Every configuration is run with seeds $\{0,1,2\}$. The code of each configuration in the ledger carries the suffixes defined at the end of the table.

Table 3.4: Experiment matrix of PP-FedData

Code	Description	Shared / Released	Protection	Role
B0	IDS trained on pooled real data (TRTR)	None	None	Reference
B1a	B0 with balanced class weights (`class_weight="balanced"`, RF only)	None	None	Reference for class imbalance
B1b	B0 with SMOTE [24] oversampling to class quota	None	None	Reference for classical interpolation
B2	CVAE trained centrally on pooled data	Pooled data	None	Upper generation reference (pools real data)
B3	CVAE with FedAvg, 5 non-IID clients, 30 rounds $\times$ 2 epochs	Model weights	None	Federated baseline ($\varepsilon = \infty$)
M1-eps{1,5,10}	B3 with DP-SGD at each client	Noisy weights	DP (local noise)	DP cost of CVAE route
M2	B3 with Flower SecAgg$^+$	Masked weights	SecAgg$^+$	SecAgg cost of CVAE route
M3-eps5	DP-SGD and SecAgg$^+$ together	Noisy, masked weights	DP and SecAgg$^+$	Combined protection
M1o / M3o-eps{1,5,10}	M1 and M3 tuned per ε at full federated scale	As M1, M3	As M1, M3	Optimisation round O1
MGs-eps{1,5,10}	FedDP-Marginal with Skellam noise, summed via SecAgg$^+$	3 releases of integer count tables	Distributed DP and SecAgg$^+$	Main generator
MGr-eps{1,5,10}	MGs with server post-processing (fusion, IPF [52], smoothing)	As MGs	As MGs	Post-processing (spends no extra ε)
MGd-eps{1,5,10}	Gaussian noise, simulated inside one process	Noisy count tables	Distributed DP (zCDP [53])	Accounting variant
MGl-eps{1,5,10}	Each client adds full noise; no secure aggregation	Noisy count tables	Local DP [49]	No trust in other clients
MGb-eps10	FedDP-Marginal with 2nd-order Bayesian net (PrivBayes-style [9])	Pairwise count tables	Distributed DP (simulated)	Dependency extension
FedMLP / FedMLPcw	MLP classifier trained directly with FedAvg, no generator	Classifier weights	None, DP or SecAgg	Reference (releases no data)

Suffixes: `-strm{m}` protects a whole TCP stream with at most m rows per stream; `-cap{m}` protects a whole capture with at most m rows per capture; `-t{t}` calibrates the noise for t honest clients (default $t = K = 5$; $t \in \{2, 3\}$ are additional settings).

Main Configurations of the Framework

Three configurations are designated the framework's main configurations, one per privacy budget: MG-eps1 = MGs-eps1, MG-eps5 = MGr-eps5, MG-eps10 = MGs-eps10, all with $\delta = 10^{-5}$. For each budget, the variant is the one with the highest validation macro-F1 over three seeds (Section 3.3.5).

Hyper-parameter Selection

Hyper-parameters are tuned on the real validation split with Optuna [54] using the Tree-structured Parzen Estimator (TPE) [55]. The search spaces differ by route (CVAE without DP, CVAE with DP at proxy scale, CVAE with DP at full scale, FedDP-Marginal, IDS side) and are defined in Section 3.3.5.

Robustness and Sensitivity Branches

To assess how far the design conclusions depend on settings fixed in the main study, the matrix includes the branches in Table 3.5. The run status is stated explicitly; branches without a runner are not part of the study.

Table 3.5: Robustness and sensitivity branches

Branch	Change relative to main study	Status
Heterogeneity	Dirichlet $\alpha \in \{0.1, 10.0\}$ (main $\alpha = 0.5$)	Run for MGs (one seed per setting)
Federation size	$K \in \{10, 20\}$ (main $K = 5$)	Run for MGs (one seed) and for MGl (three seeds)
Label resolution	11-class mode separating DoS and DDoS	Run through robustness module for reference and FedDP-Marginal
A4	`split_seed` changed so other capture groups form validation/test	Run with two alternative choices
A5	At most 20 packets per TCP stream in training pool (`max_rows_per_stream` = 20)	Run
A2	Rare-class training quotas scaled by 6 (NORMAL:WILL = 10:1) and by 0.3 (NORMAL:WILL = 200:1)	Run for the CVAE and FedDP-Marginal
A3	11-class mode (the Label resolution branch above)	Run for B0, B3 and FedDP-Marginal; the CVAE with DP and SecAgg$^+$ are not run in this mode

Configurations Written but Not Run

Two components exist as code and tests but were not executed after the change of primary generator (Section 3.1.1): M3f, a distributed DP-FedSGD variant of the CVAE, and a sweep of the SecAgg^{+} quantisation width for the CVAE weights. Both are listed again among the limitations (Section 3.4).

3.1.5 Requirement-Driven Recommendation Rule

Motivation and Relation to Rule R6

The original specification fixed a single rule, R6: among the configurations whose membership-inference AUC is at most 0.55, whose ε (maximum over clients and worst seed) is at most 5 when DP is used, and whose time per round is at most three times that of B3, choose the one with the highest TAug macro-F1 of the Random Forest. Ties within the seed noise are broken by the strength of the formal protection. The thresholds are heuristics fixed by the research team (Section 3.4). R6 is retained as a reference rule. A single fixed criterion cannot express deployments that differ in trust, bandwidth, and privacy unit, so the framework adds a requirement-driven rule, implemented as `ppfeddata recommend` (`recommend.py`), which takes a deployment requirement as input.

Requirement Formulation

A requirement is the tuple:

$$\begin{aligned}\mathscr{R} = \langle &\mathrm{Trust}_{\mathrm{server}}, \mathrm{Trust}_{\mathrm{clients}}, \varepsilon_{\max}, \mathrm{BW}_{\mathrm{round}}, \mathrm{BW}_{\mathrm{total}}, \\ &\mathrm{Rounds}_{\max}, \mathrm{Unit}, \mathrm{Priority}, \mathrm{Recall}_{\mathrm{rare,min}}, \mathrm{MIA}_{\max}, h\rangle\end{aligned} \tag{3.1}$$

where $\mathrm{Trust}_{\mathrm{server}}$ and $\mathrm{Trust}_{\mathrm{clients}}$ state whether the server and the other clients are trusted; $\varepsilon_{\max}$ is the record-level (or unit-level) budget, or none if no DP is required; $\mathrm{BW}_{\mathrm{round}}$, $\mathrm{BW}_{\mathrm{total}}$, and $\mathrm{Rounds}_{\max}$ bound the gateway bandwidth per round, over the whole run, and in rounds; $\mathrm{Unit} \in \{\mathrm{packet}, \mathrm{stream}, \mathrm{capture}\}$ is the unit ε must cover; $\mathrm{Priority} \in \{\mathrm{macro}, \mathrm{binary}, \mathrm{rare}\}$ selects TSTR macro-F1, binary F1, or mean rare-class recall; $\mathrm{Recall}_{\mathrm{rare,min}}$ is a floor on the rare-class recall; $\mathrm{MIA}_{\max}$ is the maximum acceptable membership-inference AUC; and h is the number of clients assumed to add their noise share.

Procedure

Inputs are the rows of the scorecard, which are produced from the run ledger; no figure is typed by hand. The rule applies five steps:

1. **Validity filter:** A row is considered only if its reported quantities correspond to what its name claims. For example, a DP configuration with a component computed from data outside the ε accounting (residual noise computed from real training data) is not valid; a directly federated classifier is a reference and is excluded from the generator front.

2. **Trust filter:** If the server is not trusted, only rows with secure aggregation or local DP remain. If clients may collude, the ε used for a distributed-DP row is the one that holds with the assumed number of honest clients (one honest client by default, or h when it is specified); local-DP rows keep their ε because each client adds the full noise.

3. **Constraint filter:** Rows that exceed the ε budget (with a 2% tolerance), the per-round or total bandwidth, the number of rounds, or the MIA bound, that do not protect the requested unit (a row protecting a packet does not satisfy a stream or capture requirement), or whose rare-class recall is below the floor, are removed. Missing cost figures do not exclude a row; they are reported as not available.

4. **Pareto front:** Among the remaining rows, configuration A dominates B if A is at least as good on every axis and better on at least one. The axes are the three utility measures (TSTR macro-F1, binary F1, rare-class recall), ε under the assumed trust, the presence of secure aggregation, MB per round, and total MB. A utility difference counts only if it exceeds the seed standard deviation of both configurations; ε values within 2% and cost values within 5% are treated as equal; a configuration with secure aggregation is preferred to one without when the other axes are equal.

5. **Selection and ties:** The configuration with the best value on the requested priority is selected, and every configuration within the seed standard deviation of it is listed as tied.

Deployment Scenarios

Table 3.6 defines eleven illustrative requirements. Only the inputs are specified here; the configuration chosen for each is a result and is reported in Chapter 4.

Table 3.6: Deployment scenarios (inputs only)

Scenario	Setting	Requirement
`lab`	One operator owns server and all gateways	Server and clients trusted; no DP required; priority macro
`cloud-aggregator`	Gateways of one company, 3rd-party cloud	Server not trusted; clients trusted; $\varepsilon \leq 5$; priority macro
`hidden-tables`	As `cloud-aggregator`, but the synthetic data stay inside the company	Server must not see any gateway's tables; clients trusted; no DP required; priority macro
`consortium`	Brokers of different companies may collude	Server not trusted; clients may collude; $\varepsilon \leq 10$; priority macro
`strict-privacy`	Sensitive deployments (e.g., healthcare IoT)	Server not trusted; clients trusted; $\varepsilon \leq 1$; priority macro
`low-bandwidth`	NB-IoT, LoRa or metered cellular	Server not trusted; $\varepsilon \leq 10$; ≤ 2 MB/round; $\leq$ 100 MB total; priority macro
`rare-attacks`	Rare attacks (DELAYED, SYN, INVALID, WILL)	Server not trusted; clients trusted; $\varepsilon \leq 5$; priority rare
`honest-majority`	Five operators, at most 2 may collude	Server not trusted; ≥ 3 honest clients; $\varepsilon \leq 10$; MIA AUC < 0.55; priority macro
`gateway-dropout`	Up to 2 of 5 gateways drop out / leak noise	Server not trusted; ≥ 3 honest clients; $\varepsilon \leq 5$; priority macro
`session-privacy`	ε must cover a whole TCP session	Server not trusted; $\varepsilon \leq 5$; unit = stream; priority macro
`capture-privacy`	ε must cover a whole capture	Server not trusted; $\varepsilon \leq 10$; unit = capture; priority macro

3.2 Data Collection Methods

3.2.1 Dataset

The data source is the DoS/DDoS-MQTT-IoT dataset of Alatram et al. [2], obtained from the authors' official Kaggle release [56]. It contains normal IoT traffic and several denial-of-service (DoS) and distributed denial-of-service (DDoS) attacks that abuse the MQTT protocol [57]. The release is distributed in several formats; this study works on CSV exports that total 39,000,601 rows.

Each row is one network packet described by 33 feature columns. The Normal traffic has two additional columns, `Label` and `Capture_ID`; `Capture_ID` assigns the `group_id` used for splitting, whereas attack rows take their `group_id` from the name of the source file. An inventory step confirms that the row counts match the expected totals. Table 3.7 gives the composition by class.

Table 3.7: Composition of the DoS/DDoS-MQTT-IoT dataset

Class	DoS	DDoS	Total rows	Share
NORMAL	—	—	24,955,877	63.99%
BCF (Basic Connect Flooding)	5,825,026	6,455,848	12,280,874	31.49%
DELAYED (Delayed Connect Flooding)	291,988	309,554	601,542	1.54%
SYN (SYN TCP Flooding)	103,828	497,611	601,439	1.54%
INVALID (Invalid Subscription Flooding)	109,150	181,190	290,340	0.74%
WILL (Connect Flooding with WILL payload)	114,474	156,055	270,529	0.69%
Total			**39,000,601**	**100.00%**

Data Provenance and Verification

The attack traffic is used as published: the 48 attack CSV files exported by the dataset authors [2], whose paths and sizes match the Kaggle release [56]. The release provides the Normal traffic only as 49 PCAP files, so the Normal traffic was extracted with TShark 4.4.7 [58] into the same 33 columns, with the export settings of the dataset authors and a script kept in the project repository.

The settings were recovered from the attack captures, for which both the PCAP and the authors' CSV exist. Two options reproduce the frame numbers of the official CSV exactly: two-pass dissection (`-2`) and the display filter `ip.dst==192.168.80.49`, which keeps the packets sent to the broker. They match 47 of the 48 captures. The remaining capture, SYN_DDoS-AD_1, was exported by the authors with the filter `ip`, which keeps both directions; it is used as published, so the SYN class also holds packets sent by the broker. Over the 14,044,724 rows of these 48 captures, 5,860 rows (0.042%) differ in at least one column used by the models, and 3,156 of those are TCP retransmissions.

The same two options were then applied to the 49 Normal PCAP files and yield 24,955,877 rows. The extraction is complete in the sense that all 49 captures have one row per kept frame, with no frame missing from a full extraction of the same PCAP, and the time gap of every row is the gap to the previous kept packet. Table 3.8 summarises the checks.

Table 3.8: Data provenance verification

Question	**Method**	**Result**
Are the files those of the release?	File paths and sizes compared with the public Kaggle manifest [56]	411/411 files match (49 attack CSVs, 313 attack PCAPs, 49 Normal PCAPs)
Which settings did the authors use?	Script with `-2` and a display filter run on the attack PCAPs; frame numbers compared with the official CSVs	`ip.dst==192.168.80.49` in 47/48 captures; `ip` in SYN_DDoS-AD_1
Do those settings give the authors' feature values?	Output compared row by row with the official CSVs (14,044,724 rows)	0.042% of rows differ in a column used by the models; 53.9% of these are TCP retransmissions
Is the Normal extraction complete?	One row per kept frame; every kept frame also present in a full extraction of the same PCAP	49/49 captures (24,955,877 rows)
Is the time-gap column consistent with the filter?	Time delta compared with the gap to the previous kept packet	Equal in 100.00% of the rows of each capture

3.2.2 Label Harmonisation and Rare Classes

The labels are harmonised into a six-class mode: NORMAL, BCF, DELAYED, SYN, INVALID, and WILL. In this mode, the DoS and DDoS variants of the same attack family are merged into one class. The merge keeps the main matrix at a size where every class has enough rows for the generators; leakage check C5 (Section 3.2.6) measures how far the two variants of each family can be told apart at packet level. The original eleven-class mode remains available in the pipeline but is used only for the robustness branch (Table 3.5).

In this study, the rare classes are the attack classes whose training quota is at most 5,000 rows, namely DELAYED, SYN, INVALID, and WILL. Their rarity in training is a deliberate consequence of the quota design rather than a lack of raw data: as Table 3.7 shows, the raw dataset contains between 270,529 and 601,542 packets for each of them.

The quotas create an imbalance of 60:1 between the majority class (NORMAL, 60,000 rows) and the smallest class (WILL, 1,000 rows), which makes the evaluation of the generators demanding [3]. The validation and test splits, in contrast, are class-balanced (2,000 and 5,000 rows per class respectively), so that the evaluation measures are not dominated by the majority class.

3.2.3 Schema Harmonisation and Feature Selection

Preprocessing starts by harmonising the schema of the raw packet exports. Column names are converted to snake case. Wireshark text labels (such as `message_type`, `qos_level_*`, and `requested_qos`) are normalised to canonical codes, and boolean indicators (True/False, Set/Not set) are mapped to 0/1. Redundant columns with the suffix `_1` are removed. For multi-valued cells that contain several MQTT messages, only the first element is kept, in line with the Normal export, which used the TShark option `occurrence=f`; the `n_mqtt_msgs` column is therefore dropped. Rows are then restricted to those whose protocol is TCP or MQTT, followed by a format audit of the time-based tokens.

To prevent leakage [59, 60], identifier and metadata columns are removed:

- `no` (row sequence), `epoch_time` (absolute timestamp), and `time_since_reference_or_first_frame` (which reveals the start time of the capture);
- `source`, because sender IP addresses would let a model memorise machine identities instead of learning traffic patterns;

- the free-text payload in `info`, and the `label` and `capture_id` columns, which exist only in the Normal traffic.

`stream_index` is kept only as grouping metadata and is not an input feature. The raw dataset contains no MAC address or port columns. After the feature selection decisions, the final schema has $D = 59$ input columns (Table 3.9).

Table 3.9: Final feature schema ($D = 59$)

Feature type	**Columns**	**Dim.**
Numeric	`time_delta_from_previous_displayed_frame`, `frame_length_on_wire_2`, `irtt`, `time_since_first_frame_in_this_tcp_stream`, `tcp_segment_len`, `calculated_window_size`, `keep_alive`, `user_name_length`, `password_length`, `will_message_length`, `will_topic_length`, `topic_length`, `msg_len`	13
Binary	`syn`, `reset`, `acknowledgment`, `retain`, `clean_session_flag_2`, `will_retain`, `will_flag`	7
Missingness flags	12 `_is_na` indicator columns	12
Categorical (one-hot)	`message_type` (11), `qos_level_1` (5), `qos_level_2` (4), `requested_qos` (3), `protocol` (4)	27
Total		**59**

3.2.4 Group-Based Split and Quota Sampling

The eleven sub-classes (NORMAL and the DoS and DDoS variants of the five attack families) are split by group: whole capture groups are assigned to the training, validation, or test split according to a fixed `split_seed`. This keeps highly correlated packets of the same TCP stream or capture session from appearing in more than one split [60].

The source data limits this approach: 8 of the 11 sub-classes come from a single capture file. For these sub-classes, the validation and test rows are taken as consecutive blocks of the same capture used for training (block split), separated by buffers of 1,000 rows to reduce adjacency leakage. TCP streams that straddle a block boundary are dropped entirely. This

removes between 1.2% and 9.9% of the eligible rows of each affected sub-class and biases the sample slightly toward shorter connections.

After the split, rows are drawn at random without replacement according to the class quotas in Table 3.10. The training quotas impose a strong imbalance, whereas the validation and test quotas are balanced.

Table 3.10: Class quotas of the training, validation, and test splits

Class	**Train**	**Validation**	**Test**
NORMAL	60,000	2,000	5,000
BCF	20,000	2,000	5,000
DELAYED	3,000	2,000	5,000
SYN	3,000	2,000	5,000
INVALID	1,500	2,000	5,000
WILL	1,000	2,000	5,000
Total	**88,500**	**12,000**	**30,000**

The 30,000-row test split is locked and used only for the final report. All hyper-parameter tuning, design choices, and threshold calibration use the validation split only.

3.2.5 Preprocessing and Encoding

The preprocessing pipeline is fitted on the training split only; the validation and test splits are transformed with the stored parameters. Missing or non-applicable numeric values are imputed with 0, and an indicator column `_is_na` is added for every feature whose missing rate in the training split lies strictly between 0.005 and 0.995.

Numeric features with a skewness greater than 2 are transformed with $\log(1+x)$; time-delta features are first multiplied by 1,000 (seconds to milliseconds). Numeric columns are then standardised to zero mean and unit variance and clipped at $\pm 5\sigma$ to limit the influence of extreme values. Categorical features are one-hot encoded. To bound the dimension, only the ten most frequent training tokens of each categorical column are kept; other valid values are grouped into an `OTHER` token and missing values are mapped to a `NONE` token.

The normalisation statistics, category lists, and imputation parameters are fitted centrally on the pooled training split. This is a limitation of the method: it assumes a centralised view of the data before federation, and these statistics lie outside the privacy budget ε (Section 3.1.3).

3.2.6 Leakage and Label-Reliability Checks

Before any synthetic data is generated, a set of diagnostic checks verifies that the learning task is not made artificially easy by the schema and that the attack labels are reliable. Each check uses a Random Forest [45] with 100 trees, evaluated on the validation split with seeds $\{0,1,2\}$.

The suite consists of five checks (C1–C5). Each has a heuristic threshold that flags an anomaly for manual inspection or a change of schema (Table 3.11).

Table 3.11: Leakage and label-reliability checks

Code	Check	Flag threshold
C1	Presence of each column across classes, to find attributes structurally bound to particular attacks	A column is empty in one class but populated in another
C2	Random Forest trained only on the `_is_na` missingness indicators	Macro-F1 > 0.5
C3	Class-balanced Random Forest trained on single raw columns, and ablations that omit given features	Macro-F1 or best per-class F1 > 0.9 on any single feature
C4	Gap between a random row split and the group-based stream split	Difference (random minus group) > 0.05
C5	Binary task separating the DoS and DDoS variants of the same attack family	F1 < 0.6

3.3 Sampling Data Analysis Techniques

3.3.1 Federated Partitioning

Dirichlet Partition of the Training Data

To model non-identically distributed (non-IID) traffic across edge gateways, the training split is partitioned over a federation of gateways $\mathscr{K} = \{1,2,\ldots,K\}$, with $K = 5$ in the main study. The partition follows the label-skew model of Hsu et al. [44], which uses a symmetric Dirichlet distribution with concentration parameter α_{dir}. For each class $c \in \{1,\ldots,C\}$ ($C = 6$

in the six-class mode), the proportions assigned to the K gateways are drawn independently:

$$\mathbf{p}_c = (p_{c,1}, p_{c,2}, \ldots, p_{c,K}) \sim \mathrm{Dir}(\alpha_{\mathrm{dir}} \mathbf{1}_K), \qquad \mathbf{1}_K = (1, \ldots, 1) \in \mathbb{R}^K. \tag{3.2}$$

The concentration parameter controls the heterogeneity between gateways [44]: small values ($\alpha_{\mathrm{dir}} \to 0$) concentrate each class on few gateways, while large values ($\alpha_{\mathrm{dir}} \to \infty$) approach an IID allocation. The main study uses $\alpha_{\mathrm{dir}} = 0.5$. Under this allocation, some gateways receive few or no examples of the rare attack classes.

Record-Level and Unit-Level Partition

Two partition modes are distinguished:

- **Record-level partition (default):** Each packet record (x, y) of class c is assigned to gateway k with probability $p_{c,k}$. Packets of the same TCP stream or capture group can therefore reach several gateways. This models an open edge network in which the packets of one flow pass through different regional routers.
- **Unit-level partition (group DP):** Used only for the stream- and capture-level privacy evaluations (Section 3.3.3). Capture groups or TCP streams are treated as indivisible units: the Dirichlet proportions are applied to the majority class of each unit, and all records of a unit go to one gateway. No protected unit is split across gateways, which gives a clear boundary for bounding each unit's contribution.

Constraints and Safeguards

The partition enforces the following constraints:

- **Resampling floor:** If any gateway receives fewer than 500 records in total ($\sum_c n_{k,c} < 500$), the proportions are rejected and drawn again from $\mathrm{Dir}(\alpha_{\mathrm{dir}} \mathbf{1}_K)$ until every gateway reaches the floor.
- **Unit-level floor:** In the unit-level partition, each gateway must hold at least 200 rows, so that every gateway has enough local samples for aggregation.
- **One partition per seed:** Each seed $s \in \{0, 1, 2\}$ draws an independent partition $\mathbf{p}_c^{(s)}$, and every model evaluated with seed s uses that same partition, so the partition does not confound the comparison of models.

- **Split before partition:** The global train/validation/test split is made before the allocation to gateways, so the validation and test rows never enter the federated training pool. Full session independence between splits is still limited by the data: for 8 of the 11 sub-classes, the validation and test rows are consecutive blocks of the capture used for training (Section 3.2.4 and Section 3.4).

Table 3.12 summarises the partition parameters.

Table 3.12: Federated partition parameters

Parameter	Value	Role
Number of gateways K	5 (main); $\{10, 20\}$	Federation size; scalability branch
Concentration α_{dir}	0.5 (main); $\{0.1, 10\}$	Non-IID setting; heterogeneity branch
Partition mechanism	Per-class Dirichlet allocation [44]	By row (record DP); by unit (group DP)
Resampling floor	$n_k \geq 500$ records	Rejection threshold, record-level partition
Unit floor	$n_k \geq 200$ records	Rejection threshold, unit-level partition
Seeds	$\{0, 1, 2\}$	One partition per seed, shared by all models
Training quota ($N =$ 88,500)	NORMAL 60,000; BCF 20,000; DELAYED 3,000; SYN 3,000; INVALID 1,500; WILL 1,000	Class distribution partitioned over the gateways (rare classes: quota $\leq$ 5,000)

Invariance of the Summed Marginals to the Partition

For any count query summed through SecAgg$^+$ [18], the global result is the exact, unweighted sum of the local integer count vectors:

$$\mathbf{h}_{\text{global}} = \sum_{k=1}^{K} \mathbf{h}_k = \sum_{k=1}^{K} \sum_{i \in \mathscr{D}_k} \mathbf{e}(x_i) = \sum_{i \in \bigcup_k \mathscr{D}_k} \mathbf{e}(x_i), \tag{3.3}$$

where $\mathbf{e}(x_i)$ is the indicator vector of the bin that contains record x_i. Because integer addition is commutative and associative, the summed table without noise is identical to the table computed centrally, whatever the label skew $P_k(Y)$ or covariate shift $P_k(X \mid Y)$.

Running FedDP-Marginal with $\alpha_{\text{dir}} \in \{0.1, 0.5, 10\}$ is therefore a null prediction, or sanity check: without client dropout or unit capping, its summed marginals and synthesis are in theory unaffected by the partition skew. In a real federation, heterogeneity affects FedDP-Marginal only in two cases:

1. **Client dropout:** The counts of a dropped gateway are missing from the sum, and under small α_{dir} that gateway may hold class samples that no surviving gateway has.
2. **Record-level partition with a unit cap m:** A unit spread over several gateways is capped at m rows by each of them, whereas on a single gateway it would be capped only once.

3.3.2 FedDP-Marginal

Positioning and Overview

FedDP-Marginal adapts private marginal-based synthesis of tabular data, namely the MST method of McKenna et al. [40] and the graphical-model principles of PrivBayes [9], to a federated setting. It is not a new generative method; it adapts marginal-based dependency modelling to federation through secure integer aggregation. Unlike deep federated models, which need many rounds of weight exchange, FedDP-Marginal uses three query releases coordinated through SecAgg$^+$ [18] in Flower [46] (Figure 3.3).

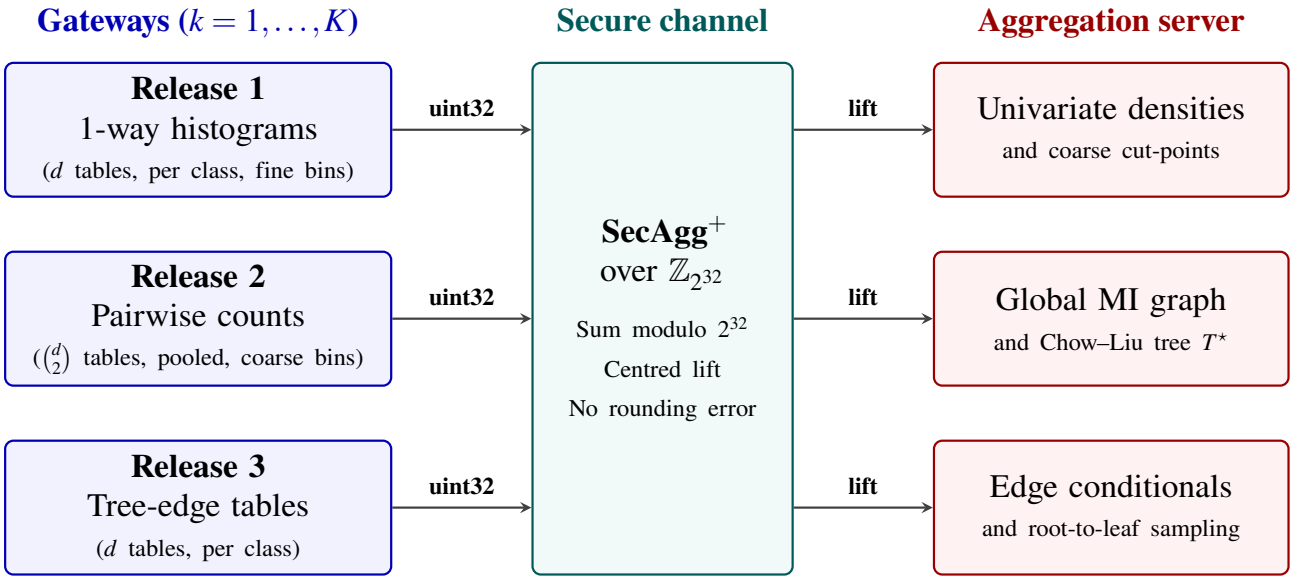


Figure 3.3: Three-release protocol of FedDP-Marginal. The coarse cut-points (after Release 1) and the tree $T^\star$ (after Release 2) are sent back to the gateways before the next release.

Attribute Schema ($d = 37$) and One-Hot Representation ($D = 59$)

The marginal generator and the neural generator represent features differently:

- **Tabular schema ($d = 37$ attributes):** FedDP-Marginal queries count tables over 37 discrete or discretised attributes:
  - 13 numeric attributes (for example inter-arrival times, segment lengths, TCP window sizes), discretised into $B_{\text{fine}} = 64$ uniform bins over the normalised interval $[-5, 5]$, which matches the $\pm 5\sigma$ clipping of preprocessing;
  - 19 binary attributes: 7 protocol flags (`syn`, `reset`, `acknowledgment`, `retain`, `clean_session_flag_2`, `will_retain`, `will_flag`) and 12 `_is_na` indicators;
  - 5 categorical attributes (`message_type`, `qos_level_1`, `qos_level_2`, `requested_qos`, `protocol`), kept as discrete variables in the tables.
- **One-hot representation ($D = 59$):** The CVAE baseline works on $\mathbb{R}^D$ with $D = 59$, obtained by expanding the 5 categorical attributes into 27 one-hot columns ($13 + 19 + 27 = 59$; Table 3.9).

Three-Release Protocol

The protocol issues three releases in sequence and divides the privacy budget between them:

- **Release 1 (fine univariate marginals per class):** Each gateway counts, for every attribute X_i ($i = 1, \ldots, d$) and class c, the rows in each of the $B_{\text{fine}} = 64$ fine bins, adds its share of discrete noise, and sends the result through SecAgg$^+$ [18]. From the summed noisy histograms, the server sets data-dependent coarse cut-points for the numeric attributes at empirical quantiles, with $B_{\text{coarse}} \in \{4, 8, 16, 32\}$ (default 16). The coarse cut-points coincide with edges of the fine bins.
- **Release 2 (coarse pairwise marginals, pooled over classes):** Gateways count all $\binom{d}{2} = 666$ attribute pairs (X_i, X_j) over the coarse bins, pooled over all classes. Pooling maximises the count per cell and bounds the table size to B_{coarse}^2, which limits the effect of the privacy noise on dependency estimation.

- **Release 3 (tree-edge marginals per class):** Using the structure derived from Release 2, the server requests the joint tables of the $d-1$ edges of the global Chow–Liu tree, conditioned on the class $y=c$, over the coarse bins (at most $B_{\text{coarse}}^2 \cdot C$ cells, so that the noise does not swamp the rare classes), together with the class-conditional table of the root attribute over the coarse bins.

One Global Chow–Liu Tree

FedDP-Marginal builds a single global Chow–Liu dependency tree [10], pooled over all classes, rather than one tree per class. From the post-processed tables of Release 2, the server computes the empirical mutual information (MI) between attributes and extracts the maximum spanning tree $T^\star$ with Kruskal's algorithm [61]:

$$T^\star = \arg\max_T \sum_{(i,j)\in T} \hat{I}(X_i;X_j), \qquad \hat{I}(X_i;X_j) = \sum_{a,b} \hat{p}(a,b) \log \frac{\hat{p}(a,b)}{\hat{p}(a)\,\hat{p}(b)}. \tag{3.4}$$

Following the post-processing of McKenna et al. [40], the probabilities are obtained from the noisy pairwise counts $c_{i,j}$ by truncation at zero, additive (Laplace) smoothing α_{smooth}, and normalisation:

$$\hat{p}(a,b) = \frac{\max(0, c_{i,j}(a,b)) + \alpha_{\text{smooth}}}{\sum_{a',b'} [\max(0, c_{i,j}(a',b')) + \alpha_{\text{smooth}}]}. \tag{3.5}$$

The univariate terms $\hat{p}(a)$ and $\hat{p}(b)$ are obtained by summing out the other dimension. By the post-processing property of differential privacy [51], deriving $T^\star$ spends no additional privacy budget.

Trade-off of pooling. The class-conditional MI $\hat{I}(X_i;X_j \mid Y=c)$ would match class-conditional synthesis more closely. However, splitting Release 2 by class leaves the sensitivity per cell unchanged while dividing the signal per cell by about C, and the total noise over the $C\binom{d}{2}$ tables grows C-fold. The signal-to-noise ratio of each cell therefore drops by a factor of C, which corrupts the choice of edges. FedDP-Marginal deliberately pools the pairwise counts over classes to maximise the signal-to-noise ratio of the skeleton $T^\star$. The pooled MI then reflects the overall dependency structure, which is dominated by NORMAL traffic (67.8% of the training quota); class-specific relations are captured afterwards by the class-conditional edge tables of Release 3.

Cardinality bias. The plug-in MI on noisy tables has a finite-sample bias proportional to the degrees of freedom of the table, $(|X_i|-1)(|X_j|-1)$. Numeric attributes have a higher coarse cardinality ($B_{\text{coarse}}=16$) than binary flags ($|\cdot|=2$), so a pair of numeric attributes has

up to $(16-1)^2 = 225$ degrees of freedom against $(2-1)^2 = 1$ for a pair of flags. Kruskal's algorithm on unnormalised plug-in MI therefore tends to favour edges between high-cardinality numeric attributes. Laplace smoothing adds a uniform pseudo-mass $\alpha_{\text{smooth}}|X_i||X_j|$, which dilutes the probabilities more in larger tables. The Miller–Madow correction [62]

$$\hat{I}_{\text{MM}}(X_i;X_j) = \hat{I}(X_i;X_j) - \frac{(|X_i|-1)(|X_j|-1)}{2\hat{N}}, \tag{3.6}$$

where $\hat{N} = \sum_{a,b} \max(0, c_{i,j}(a,b))$ is the total truncated count, corrects the asymptotic multinomial sampling bias but not the additive Skellam noise or the non-linear truncation $\max(0,\cdot)$. The main pipeline keeps the uncorrected plug-in MI with coarse bins ($B_{\text{coarse}} \leq 16$) as the baseline heuristic. To measure the effect of the cardinality bias on the structure, separately from downstream utility, search space (d) (Section 3.3.5) includes an ablation that compares plug-in MI, Miller–Madow MI, and normalised MI ($\hat{I}(X_i;X_j)/\sqrt{H(X_i)H(X_j)}$), together with a check of the stability of the tree edges across partition seeds.

Class-Conditional Sampling

Synthetic records of class c are generated by traversing $T^\star$ from the root to the leaves [10]:

$$\hat{P}(\mathbf{x} \mid y = c) = \hat{P}(x_r \mid y = c) \prod_{(u,v)\in T^\star} \hat{P}(x_v \mid x_u, y = c), \tag{3.7}$$

where r is the root attribute. The steps are:

1. **Root selection and sampling:** For MGs, the root is the first attribute of the schema, the tree orientation used at release time. MGr re-orients the same undirected tree to the attribute with the largest sum of noisy MI, $\sum_{j\neq r} \hat{I}(X_r;X_j)$. The root is sampled from its class-conditional table on the coarse bins, $x_r \sim \hat{P}(X_r \mid y = c)$.

2. **Conditional propagation:** Each child is sampled given its parent, $x_v \sim \hat{P}(X_v \mid X_u = x_u, y = c)$. The noisy counts are truncated at zero and smoothed with α_{smooth}.

3. **Refinement of numeric attributes:** The sampled coarse bin is refined to a fine bin in proportion to the fine class histogram of Release 1 (whose fine bins nest inside the coarse bins), and a value is drawn uniformly within the fine bin $[b_{\text{low}}, b_{\text{high}}]$.

4. **Inverse transformation:** The vectors pass through the same inverse pipeline as the CVAE baseline (inverse standardisation and inverse $\log(1+x)$).

Sensitivity, Budget Split and Consistency Refinement (MGr)

Neighbouring relation and sensitivity. The accounting uses unbounded adjacency: two datasets $D \sim D'$ differ by the addition or removal of one packet record. One record increments exactly one cell in each of the d tables of Release 1, the $\binom{d}{2}$ tables of Release 2, and the d tables of Release 3 ($d-1$ edge tables and one root table). For a release of T tables, the global sensitivities are $\Delta_1 = T$ and $\Delta_2 = \sqrt{T}$. Under bounded DP (replacement of a record within a class), they become $\Delta_1^{\text{repl}} = 2T$ and $\Delta_2^{\text{repl}} = \sqrt{2T}$.

Budget split. The budget is divided between releases by a split vector $\mathbf{s} = (s_1, s_2, s_3)$ with $s_1 + s_2 + s_3 = 1$, by default $(0.3, 0.1, 0.6)$:

- For the Gaussian variant (MGd), the zCDP parameter is split as $\rho_r = s_r \rho$, which gives the noise standard deviation $\sigma_r = \Delta_{2,r}/\sqrt{2 s_r \rho}$ for release r.
- For the Skellam variant (MGs), the noise parameter of release r is $\mu_{\text{sk},r} = \Delta_{2,r}^2/(2 s_r \kappa)$, where the global factor $\kappa > 0$ is found by binary search.
- With the default split, Release 2 has $\Delta_{2,2}^2 = \binom{37}{2} = 666$ with $s_2 = 0.1$, and Release 1 has $\Delta_{2,1}^2 = 37$ with $s_1 = 0.3$, so $\mu_{\text{sk},2}/\mu_{\text{sk},1} = (666/0.1)/(37/0.3) = 18 \times 3 = 54$. Release 2 thus adds about 54 times the noise variance per cell of Release 1, which makes coarse binning ($B_{\text{coarse}} \leq 16$) necessary to keep a usable signal-to-noise ratio.

Consistency refinement (MGr). To make the tables consistent without spending budget, MGr applies the following post-processing:

1. **Bin-wise minimum-variance merging:** Merging is done on counts, per class, before normalisation. The fine counts of Release 1 are collapsed to coarse bins; a coarse bin b of X_u that contains k_b fine bins has variance $k_b \sigma_1^2$. Projecting an incident Release 3 edge table (X_u, X_v) onto X_u, by summing over the $|X_v|$ coarse bins of X_v, gives an independent estimate with variance $|X_v|\sigma_3^2$. A non-root attribute with degree $d(u)$ in $T^\star$ therefore has $1 + d(u)$ independent estimators. For the root r, Release 3 also provides the class-conditional fine root table, a second univariate estimator with variance $k_b \sigma_3^2$, so the root merges $2 + d(r)$ estimators. Following McKenna et al. [40], the estimators are merged bin by bin with inverse-variance weights:

$$\hat{C}_{\text{merged}}(X_u = b \mid y = c) = \frac{\sum_j w_{u,j}(b)\, \hat{C}_{u,j}(b \mid y = c)}{\sum_j w_{u,j}(b)}, \tag{3.8}$$

with weights $w_{u,j}(b) = 1/\mathrm{Var}\big(\hat{C}_{u,j}(b \mid y = c)\big)$. The merged counts are then normalised, $\hat{P}_{\text{merged}}(b) = \hat{C}_{\text{merged}}(b)/\sum_{b'} \hat{C}_{\text{merged}}(b')$.

2. **Truncation:** The merged marginals and the noisy Release 3 tables are truncated at zero and normalised to sum to 1.

3. **Iterative proportional fitting (IPF):** Each edge table $(X_u, X_v \mid y = c)$ is adjusted by IPF [52, 63] to match the merged marginals of X_u and X_v, which makes the margins consistent.

4. **Smoothing and root selection:** Laplace smoothing $\alpha_{\text{smooth}} \in \{0.1, 1, 10\}$ is applied, and the root is the attribute with the largest sum of noisy MI.

Second-order Bayesian network (MGb). This ablation adapts PrivBayes [9]: pairs of parents (u_1, u_2) are chosen greedily from the MI of Release 2 while keeping the graph acyclic. Each record still touches d tables, so the record sensitivity is unchanged, while each table grows to accommodate two parents.

Transport through SecAgg$^+$

For MGs and MGr, the tables are sent through Flower SecAgg$^+$ [18] over the integer ring $\mathbb{Z}_M$ with $M = 2^{32}$ (uint32):

- Client updates are scaled with `max_weight` (unit factor 1), and the quantisation range is set to twice the clipping range, so integers are transmitted exactly, without floating-point rounding.

- Under record-level DP, `clipping_range` is 100,000, which exceeds the largest possible cell count ($N = 88{,}500$). Under group DP, it is set to $C_{\text{clip}} = N + 6\sqrt{\mu_{\text{sk}}^{\text{group}}/K}$, which covers both the data mass and 6σ of the discrete noise, so the probability that a noisy count exceeds C_{clip} is below 10^{-9} by the Skellam tail bound.

- The noisy counts $c_{k,j} = x_{k,j} + \xi_{k,j} \pmod{2^{32}}$ are sent as uint32 vectors and recovered at the server by a centred lift: $\mathrm{lift}(S) = S$ if $S < 2^{31}$, otherwise $S - 2^{32}$. The cell totals satisfy $\sum_k x_{k,j} \le N = 88{,}500$. Under record-level DP the aggregate noise has $\sigma_{\text{agg}} = \sqrt{\mu_{\text{sk}}} \le 400$. Under group DP with cap $m = 100$, the variance grows quadratically to $\mu_{\text{sk}}^{\text{group}} \le 1.6 \times 10^9$, so $\sigma_{\text{agg}} \le 4 \times 10^4$ (and $\sigma_k = \sqrt{\mu_{\text{sk}}/K} \le 1.8 \times 10^4$ per client share).

The noisy total therefore satisfies $|S| \leq N + 6\sigma_{\text{agg}} \leq 88{,}500 + 2.4 \times 10^5 \approx 3.3 \times 10^5 \ll 2^{31} \approx 2.15 \times 10^9$; the largest sum lies tens of thousands of standard deviations below the overflow threshold, and the probability of a modular wrap-around is below 10^{-50}.

- The Gaussian variant MGd is simulated in one process over $\mathbb{R}$, and the local-DP variant MGl sends its noisy tables directly without SecAgg$^+$.

Table 3.13 lists the variants of FedDP-Marginal.

Table 3.13: Variants of FedDP-Marginal

Code	Noise	Privacy target	Aggregation channel	Role
MGs	Skellam	Distributed (ε, δ)	SecAgg$^+$ over $\mathbb{Z}_{2^{32}}$	Main generator (noise shared under SecAgg$^+$)
MGr	Skellam	Distributed (ε, δ)	SecAgg$^+$ over $\mathbb{Z}_{2^{32}}$	MGs with merging and IPF consistency refinement
MGd	Gaussian	Distributed (ε, δ)	Simulated in one process over $\mathbb{R}$	Continuous Gaussian comparator (analytical zCDP)
MGl	Skellam or Gaussian	Local DP (ε, δ)	Direct sum, no SecAgg$^+$	Local-DP baseline (sum of K unshared noises)
MGb	Skellam	Distributed (ε, δ)	SecAgg$^+$ over $\mathbb{Z}_{2^{32}}$	Second-order Bayesian network (PrivBayes-style)
MGs-nodp	None ($\varepsilon = \infty$)	None	SecAgg$^+$ over $\mathbb{Z}_{2^{32}}$	Non-private reference with SecAgg$^+$ (counterpart of M2)
MGp-nodp	None ($\varepsilon = \infty$)	None	Plain Flower FedAvg	Non-private reference in clear (counterpart of B3)

3.3.3 Privacy Accounting

Analytical zCDP Accounting for the Gaussian Comparator (MGd)

To provide a continuous comparator for the discrete mechanism, MGd is accounted with analytical zero-concentrated differential privacy (zCDP) [53, Proposition 1.3]. For each release $r \in \{1,2,3\}$ with noise standard deviation σ_r and ℓ_2 sensitivity $\Delta_{2,r}$, the accumulated parameter ρ follows from the Gaussian mechanism [53, Proposition 1.6] and additive composition [53, Lemma 1.7], and the resulting (ε, δ) guarantee follows from the

conversion [53, Proposition 1.3]:

$$\rho = \sum_{r=1}^{3} \frac{\Delta_{2,r}^2}{2\sigma_r^2}, \qquad \varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{3.9}$$

ρ is divided between the three releases by the split $\mathbf{s} = (s_1, s_2, s_3)$, by default $(0.3, 0.1, 0.6)$. The analytical conversion from zCDP to (ε, δ) is looser than the numerical conversion from RDP [64]. At the same nominal ε, part of the utility difference between MGd and MGs therefore comes from the tightness of the conversion and not from the noise mechanism itself.

Skellam Mechanism and Accounting (MGs, MGr)

The nominal guarantees are $\varepsilon \in \{1, 5, 10\}$ with $\delta = 10^{-5}$. The Skellam distribution $\mathrm{Sk}(\mu_{\mathrm{sk}}) = \mathrm{Pois}(\mu_{\mathrm{sk}}/2) - \mathrm{Pois}(\mu_{\mathrm{sk}}/2)$ is the difference of two independent Poisson variables with parameter $\mu_{\mathrm{sk}}/2$; it has zero mean and variance μ_{sk} [65], [22, Definition 3.1]. Because it is closed under summation, the integer noise shares drawn by the gateways add up exactly under addition modulo 2^{32} [22].

The accounting uses the Rényi differential privacy (RDP) bound of Agarwal et al. [22, Corollary 3.6, eq. (3.2)]. For an integer order $\lambda > 1$:

$$\varepsilon(\lambda) \leq \frac{\lambda \Delta_2^2}{2\mu_{\mathrm{sk}}} + \min\left(\frac{(2\lambda - 1)\Delta_2^2 + 6\Delta_1}{4\mu_{\mathrm{sk}}^2}, \frac{3\Delta_1}{2\mu_{\mathrm{sk}}} \right). \tag{3.10}$$

Over the three releases, RDP composes additively [42, Proposition 1]: $\varepsilon_{\mathrm{total}}(\lambda) = \sum_{r=1}^{3} \varepsilon_r(\lambda)$. The composed RDP curve is converted to (ε, δ) with the bound of Balle et al. [64, Th. 21], the conversion used by the RDP accountant of Opacus [47]:

$$\varepsilon = \min_{\lambda} \left[\varepsilon_{\mathrm{total}}(\lambda) + \ln \frac{\lambda - 1}{\lambda} - \frac{\ln \delta + \ln \lambda}{\lambda - 1} \right]. \tag{3.11}$$

Because Equation (3.10) holds only for integer orders, the grid is restricted to $\lambda \in \{2, 3, \ldots, 63\} \cup \{64, 72, \ldots, 1024\}$. A global calibration factor for μ_{sk} is found by binary search so that the composed guarantee matches the target (ε, δ).

Trust Models and Division of the Noise

The noise is calibrated under three trust assumptions:

- **Distributed DP ($t = K$, main setting):** Each gateway k adds $1/K$ of the total noise variance, $\mu_{\mathrm{sk},k} = \mu_{\mathrm{sk}}/K$. The nominal guarantee holds when all K gateways are honest. When only $h < K$ remain honest, the effective $\varepsilon(h)$ is recomputed with the Opacus RDP accountant from the reduced aggregate variance $\frac{h}{K}\mu_{\mathrm{sk}}$, for $h \in \{1,\ldots,K\}$.
- **Threshold DP ($t \leq K$):** Each gateway adds $\mu_{\mathrm{sk},k} = \mu_{\mathrm{sk}}/t$, so the target guarantee holds whenever at least t gateways are honest. In SecAgg$^+$ [18], the masks are shared by Shamir secret sharing with reconstruction threshold T_{recon}. To stop colluding gateways from reconstructing the masking seeds of honest gateways and exposing their individual updates to the server, the number of colluders $c = K - t$ must satisfy $c < T_{\mathrm{recon}}$. The evaluated settings are:
  - $t = 5$: distributed DP ($c = 0$, $T_{\mathrm{recon}} = 3$), assuming full honest participation;
  - $t = 3$: threshold DP ($c \leq 2 < T_{\mathrm{recon}} = 3$), tolerating up to two dropouts or two semi-honest colluders while the server still sees only the aggregate;
  - $t = 2$ (hardened threshold DP): to tolerate $c = 3$ colluders, T_{recon} must be raised to at least 4 (leaving one dropout of tolerance). With $T_{\mathrm{recon}} = 3$, three colluders would hold three shares, enough to reconstruct the honest gateways' masks and reduce the distributed guarantee to local DP. Evaluating $t = 2$ therefore requires $T_{\mathrm{recon}} = 4$.
- **Local DP (MGl):** Each gateway adds the full noise variance, $\mu_{\mathrm{sk},k} = \mu_{\mathrm{sk}}$. No other gateway and not the server need to be trusted, and SecAgg$^+$ is not used, but the aggregate carries K times the noise variance ($K\mu_{\mathrm{sk}}$).

Group Privacy and Contribution Bounding

To protect against the tracing of a session or the re-identification of a capture, group differential privacy is enforced. The sensitivities are scaled by the cap m on the records of one unit [51]:

$$\Delta_1^{\mathrm{group}} = m \cdot \Delta_1, \qquad \Delta_2^{\mathrm{group}} = m \cdot \Delta_2. \tag{3.12}$$

With Gaussian noise the standard deviation scales linearly, $\sigma^{\mathrm{group}} = m\sigma$; with Skellam noise the parameter scales quadratically, $\mu_{\mathrm{sk}}^{\mathrm{group}} \approx m^2\mu_{\mathrm{sk}}$.

Group DP holds when (1) each protected unit (TCP stream or capture group) is held entirely by one gateway, through the unit-level partition, and (2) each gateway keeps at most m records

of each unit. Under the default record-level DP ($m = 1$), a unit of ℓ records receives only an inflated group bound. With approximate (ε, δ) composition, the group relaxation $\ell e^{(\ell-1)\varepsilon}\delta$ becomes vacuous for $\ell \geq 3$ at $\varepsilon = 10$, so the group guarantee is computed by zCDP scaling for Gaussian mechanisms [53, Proposition 1.9],

$$\rho_{\text{group}} = \ell^2 \rho, \tag{3.13}$$

converted to $(\varepsilon_{\text{group}}, \delta)$ with Equation (3.9), or by recomputing Equation (3.10) with $\Delta \to \ell\Delta$ for Skellam mechanisms. The caps $m \in \{1, 2, 4\}$ for streams and $m \in \{25, 50, 100\}$ for capture groups are target privacy policies, not tuned model parameters.

Validity Constraints and Verification

- **Linear aggregation:** Distributed DP under secure addition is valid only when each release is a single additive sum. It cannot be applied to the multi-epoch DP-SGD of the CVAE.
- **Uniform protection:** The noise is never reduced selectively for the rare classes, as that would weaken the protection of minority-class records below the announced ε.
- **Values sent in clear:** Flower sends the client sizes n_i (`num_examples`) in clear, outside the DP perimeter.
- **Independent verification:** Every reported (ε, δ) is recomputed independently by `eval/dp_check.py`. An audit flag (F5) is raised when the recomputed value differs from the reported one by more than 10% ($|\varepsilon_{\text{recomp}} - \varepsilon|/\varepsilon > 0.10$).

DP-SGD Accounting for the Federated CVAE

The deep baseline is trained with differentially private stochastic gradient descent (DP-SGD) [43] adapted to federated learning. At each local step, each client clips the per-example gradients to norm C and adds Gaussian noise to the mini-batch sum (Equation (3.17) in Section 3.3.4), using Opacus 1.6.0 [47]. Mini-batches are formed by Poisson sampling with rate $q_k = B_k/n_k$, where $B_k = \min(B_{\text{batch}}, n_k)$ keeps $q_k \leq 1$. The local privacy loss ε_k is accumulated over R rounds and E local epochs with the RDP accountant of the sampled Gaussian mechanism [66].

- **Per-client calibration and amplification by subsampling:** The noise multiplier σ_k is calibrated per client from its size n_k and its number of steps. Under a skewed partition, clients with very few rows ($n_k \approx 500 \leq B_{\text{batch}}$) have $q_k = 1$ and lose the amplification by subsampling, so their σ_k must be much larger to reach the target. Because steps are discrete and the calibration grid is finite, the realised ε_k differ slightly between clients. The federation is reported conservatively as the worst client [66],

$$\varepsilon_{\text{effective}} = \max_{k \in \mathcal{K}} \varepsilon_k, \tag{3.14}$$

  together with the median over clients; $\delta < 1/n_k$ holds for every client.

- **Asymmetry between the routes:** In M3, SecAgg$^+$ [18] hides the updates from the server in transit, but the noise is still added locally by each client without sharing across clients. The CVAE therefore works under local DP-SGD, whereas FedDP-Marginal benefits from distributed DP under secure aggregation.

Formal Privacy Perimeter

The formal DP perimeter is as follows (see also Section 3.1.3):

- **Protected by ε:** All 37 attributes (and their 64-bin discrete states) of the clients' training records. The coarse cut-points are derived from the noisy histograms of Release 1 and are therefore covered by post-processing [51].

- **Public side information outside ε:** Class labels y and the global class quotas; client sizes n_i (`num_examples` sent in clear by Flower); and the global preprocessing parameters (means, standard deviations, $\log(1+x)$ choices, and $\pm 5\sigma$ clipping bounds) computed centrally on the training split. As stated in Section 3.4, treating normalisation parameters as public is a common convention in tabular benchmarks but lies outside the formal accounting.

- **Leakage through selection:** Hyper-parameters are selected on the real validation split without a private selection mechanism (Section 3.3.5). The reported ε is the nominal guarantee of training, not an end-to-end guarantee that includes hyper-parameter selection.

- **Relaxation parameter:** $\delta = 10^{-5}$ in all evaluations ($1/n \approx 1.13 \times 10^{-5}$ for $n = 88{,}500$).

3.3.4 Federated CVAE, DP-SGD and SecAgg$^+$ (Baseline)

CVAE Architecture and Loss

The deep baseline is a label-conditional variational autoencoder (CVAE) [41]. The encoder $q_\phi(z \mid x,y)$ maps the features $x \in \mathbb{R}^D$ ($D = 59$) and the one-hot label $y \in \{0,1\}^C$ to latent parameters $(\mu_z, \log \sigma_z^2) \in \mathbb{R}^{2d_z}$, and the decoder $p_\theta(x \mid z,y)$ reconstructs the input.

- **No batch normalisation:** The CVAE has no batch-normalisation layers. This is required for per-example gradients to be independent, so that Opacus can clip each example's gradient without coupling between examples [47]. The parameters form a fixed-size vector, compatible with FedAvg and SecAgg$^+$.
- **Loss:** The model minimises the negative variational lower bound [67, eq. (3)] in its label-conditional form [41], with a weight β on the Kullback–Leibler term [68]:

$$\begin{aligned}\mathscr{L}_{\mathrm{CVAE}}(\phi,\theta) = &\sum_{b\in\mathrm{num}} \|x_b - \hat{x}_b\|_2^2 + \sum_{b\in\mathrm{bin}} \mathrm{BCE}(x_b,\hat{x}_b) + \sum_{g\in\mathrm{cat}} \mathrm{CE}(x_g,\hat{x}_g) \\ &+ \beta \cdot D_{\mathrm{KL}}\big(q_\phi(z \mid x,y) \,\|\, \mathscr{N}(0,\mathbf{I})\big),\end{aligned} \tag{3.15}$$

  where num, bin, and cat denote the numeric, binary, and categorical feature blocks.
- **Warm-up and early stopping:** β rises linearly from 0 to its target 0.5 over the first 10 local epochs (the first 5 global rounds with $E = 2$). Training is monitored by early stopping on the validation loss.
- **Default parameters:** latent dimension $d_z = 16$, hidden layers $[128, 64]$, learning rate $\eta = 10^{-3}$, batch size $B_{\mathrm{batch}} = 256$, and $\beta = 0.5$.

Federated Optimisation

Training uses Federated Averaging (FedAvg) [4, Algorithm 1] over $R = 30$ global rounds with $E = 2$ local epochs per round:

$$w^{r+1} = \sum_{k=1}^{K} \frac{n_k}{\sum_j n_j} w_k^{r+1}. \tag{3.16}$$

- **Optimiser state:** The Adam states are re-initialised at the start of each global round.
- **Deterministic order:** Client updates are aggregated in a fixed order of client identifiers, so that runs are exactly reproducible.

- **Execution:** The federation is simulated with Flower 1.39.0 [46] and Ray 2.59.0 actors that share the CPU of one machine.

DP-SGD at the Clients

When DP is used (M1 and M3), clients run DP-SGD with Opacus 1.6.0 [47]. At each local step, the per-example gradients are clipped to norm C and Gaussian noise is added before the update [43, Algorithm 1]:

$$\bar{g}_i = \frac{g_i}{\max(1, \|g_i\|_2 / C)}, \qquad \tilde{g} = \frac{1}{B_k}\left(\sum_{i=1}^{B_k} \bar{g}_i + \mathcal{N}(0, \sigma_k^2 C^2 \mathbf{I})\right), \tag{3.17}$$

where $B_k = \min(B_{\text{batch}}, n_k)$ gives a valid normalising constant for small clients.

SecAgg$^+$ Configuration and Bounds on the Weights

For M2 and M3, the updates are aggregated with Flower's `SecAggPlusWorkflow` and `secaggplus_mod` [18]:

- **Protocol topology:** `num_shares` $= 5$ and `reconstruction_threshold` $= 3$, which tolerates up to two dropped clients per round.
- **Bounds:** `clipping_range` $= 16$ (checked against the largest absolute parameter value $|w|$), uniform quantisation over $Q = 2^{22}$ levels, `max_weight` $= 100{,}000$ (above the largest client size, $n_k \leq N = 88{,}500$), and modulus $M = 2^{32}$ (uint32).
- **Weight scaling and modular bound:** To compute the exact weighted average of Equation (3.16) without overflow in $\mathbb{Z}_{2^{32}}$, each client scales its update by its relative weight before quantisation. Let $\mathcal{K}_{\text{active}}$ be the clients active in the round, whose sizes n_k are sent in clear (Section 3.3.3). Each client computes $p_k = n_k / \sum_{j \in \mathcal{K}_{\text{active}}} n_j \in (0, 1]$ and scales its clipped update $\bar{\mathbf{w}}_k$ ($\|\bar{\mathbf{w}}_k\|_\infty \leq 16$) to $\mathbf{u}_k = p_k \bar{\mathbf{w}}_k$, which still satisfies $\|\mathbf{u}_k\|_\infty \leq 16$ because $p_k \leq 1$. It then quantises $\mathbf{u}_k$ uniformly into integers $\tilde{\mathbf{u}}_k \in \{0, 1, \ldots, Q-1\}$. Through SecAgg$^+$, the server receives the modular sum [18]

$$\mathbf{S} = \sum_{k \in \mathcal{K}_{\text{active}}} \tilde{\mathbf{u}}_k \pmod{2^{32}}. \tag{3.18}$$

Since each entry is at most $Q-1$, the unmasked sum over $K = 5$ clients satisfies

$$\sum_{k \in \mathcal{K}_{\text{active}}} \tilde{u}_k \leq K(Q-1) = 5 \times (2^{22} - 1) \approx 2.097 \times 10^7 \ll 2^{31} < 2^{32}. \tag{3.19}$$

The server applies the centred lift and the affine dequantisation to recover $\sum_k p_k \bar{\mathbf{w}}_k$ directly. Because $\sum_k p_k = 1$, the server does not divide by $\sum_k n_k$. This gives the exact weighted FedAvg while avoiding 32-bit wrap-around, which would occur if the updates were scaled by the raw counts n_k: $\sum_k n_k(Q-1) \approx 88{,}500 \times 2^{22} \approx 3.71 \times 10^{11} > 2^{32}$.

- **Protocol tests:** T-SA1 (deviation from plain FedAvg $\leq 10^{-4}$), T-SA2 (masked vectors uncorrelated with the true weights and uniformly distributed), and T-SA3 (robustness to simulated client dropouts). SecAgg$^+$ provides transport privacy only and receives no credit in the DP accounting.

Generation of Synthetic Samples

After training, the encoder q_ϕ is discarded. Synthetic samples are generated by drawing $z \sim \mathcal{N}(0, \mathbf{I})$ and decoding with $p_\theta(x \mid z, y = c)$ [41]:

- **Sampling by block:** Bernoulli sampling for binary features, softmax sampling for categorical blocks, and inverse standardisation for numeric features.
- **Plain decoder:** Uses the final decoder outputs directly. It stays within the formal DP guarantee and is the format used in all DP comparisons.
- **Residual-noise decoder:** Restores class-conditional feature variances estimated on the centralised training data (outside ε). It is used only as a non-private reference.
- **Options of round O1:** DP-computed residual statistics (budget fraction `stat_frac` $\times \varepsilon$) and client class weighting (`cw_power`).

Table 3.14 compares the CVAE configurations with the main marginal generator.

3.3.5 Hyper-parameter Selection and Search Spaces

Optimisation Protocol and Integrity of Selection

Hyper-parameters are optimised with Optuna [54] using the Tree-structured Parzen Estimator (TPE) [55]. All searches use the validation split only, so the test split is never touched. The objective is the utility for the downstream IDS: the validation macro-F1 of a Random Forest [45] (100 trees during tuning to save time, 200 trees in the final protocols)

Table 3.14: Configuration matrix of the baseline and proposed generators

Code	Architecture	Privacy guarantee	Transport privacy	Rounds	Payload per round
B3	CVAE	None ($\varepsilon = \infty$)	Plain FedAvg [4]	30	Dense weights θ
M1	CVAE	DP-SGD (ε) [43]	Plain FedAvg [4]	30	Dense weights θ
M2	CVAE	None ($\varepsilon = \infty$)	SecAgg$^+$ over $\mathbb{Z}_{2^{32}}$ [18]	30	Ring tensors (uint32)
M3	CVAE	DP-SGD (ε) [43]	SecAgg$^+$ over $\mathbb{Z}_{2^{32}}$ [18]	30	Ring tensors (uint32)
MGs	FedDP-Marginal	Distributed DP (ε, δ) [22]	SecAgg$^+$ over $\mathbb{Z}_{2^{32}}$ [18]	3	Count vectors (uint32)

trained on synthetic records (5,000 rows per class) and evaluated on the real validation traffic (TSTR). The test split stays locked and is evaluated once, on the final configurations.

Selecting hyper-parameters on the real validation split leaks information about the training data through the trial models. Private selection algorithms [69–71] bound this extra privacy cost; this study reports the nominal ε of training and documents the number of trials of each search so that the selection can be accounted for.

Separate Search Spaces

Tuning is split into five search spaces (Table 3.15):

1. **Space (a), centralised CVAE without DP (phase 7):** 30 TPE trials on centralised data over 30 epochs, exploring $d_z \in \{8, 16, 32\}$, hidden width $\in \{64, 128, 256\}$, $\beta \in [0.1, 2.0]$ (log-uniform), and $\eta \in [10^{-4}, 3 \times 10^{-3}]$ (log-uniform). Output: `configs/best_cvae.yaml`.

2. **Space (b), CVAE with DP at proxy scale (`tune-dp`):** To limit the cost, initial DP-SGD tuning runs 30 trials on a single proxy client with 20% of the data at $\varepsilon = 5$ (one seed per trial). The best candidates are verified in true federated training (`verify-dp`), selected by validation macro-F1, with ties within 0.01 broken by the lowest validation loss. It explores $d_z \in \{8, 16, 32\}$, hidden width $\in \{32, 64, 128, 256\}$, $B_{\text{batch}} \in \{128, 256, 512, 1024\}$, $\beta \in [0.1, 2.0]$, $\eta \in [2 \times 10^{-4}, 5 \times 10^{-3}]$ (log-uniform), and

clipping norm $C \in [0.1, 20.0]$ (log-uniform). Output: `configs/best_cvae_dp.yaml` (family M1d-t18).

3. **Space (c), CVAE with DP at full federated scale (`tune-dp-full`, round O1):** Full federated optimisation over 5 clients with seed 0, with 30 trials for $\varepsilon = 5$, 28 for $\varepsilon = 1$, and 12 for $\varepsilon = 10$. It explores $d_z \in \{8, 16, 32\}$, hidden width $\in \{64, 128\}$ (256 removed for memory reasons), $B_{\text{batch}} \in \{256, 512, 1024, 2048\}$, $\beta \in [0.1, 2.0]$, $\eta \in [5 \times 10^{-4}, 2 \times 10^{-2}]$ (log-uniform), $C \in [0.5, 50.0]$ (log-uniform), rounds $R \in \{15, 30, 50\}$, local epochs $E \in \{1, 2, 4\}$, client class weighting `cw_power` $\in \{0, 0.5, 1.0\}$, DP residual budget fraction `stat_frac` $\in \{0.02, 0.05, 0.10\}$, and residual clipping `res_clip` $\in \{0.5, 1.0, 2.0, 4.0\}$. Output: `configs/best_cvae_dp_full.yaml` (M1o, M3o).

4. **Space (d), FedDP-Marginal:** Discrete grid search scored by the mean over three seeds on the validation split. It explores the coarse bins $B_{\text{coarse}} \in \{4, 8, 16, 32\}$ crossed with the budget splits $\mathbf{s} \in \{(0.3, 0.1, 0.6), (0.5, 0.1, 0.4), (0.15, 0.05, 0.8)\}$; the dependency measure (plug-in MI, Miller–Madow MI $\hat{I}_{\text{MM}}$ [62], or normalised MI), with a stability check that measures the Jaccard similarity [72] of the tree edges across partition seeds,

$$J(T_a^\star, T_b^\star) = \frac{|E(T_a^\star) \cap E(T_b^\star)|}{|E(T_a^\star) \cup E(T_b^\star)|}; \tag{3.20}$$

the second-order structure of MGb (parent search, bins specific to rare classes, number of fine bins); and the refinement options (inverse-variance merging, Laplace smoothing $\alpha_{\text{smooth}} \in \{0.1, 1.0, 10.0\}$, root chosen by noisy MI). The policy settings for unit caps ($m_{\text{stream}} \in \{1, 2, 4\}$, $m_{\text{capture}} \in \{25, 50, 100\}$) and honest thresholds ($t \in \{5, 3\}$, and $t = 2$ with $T_{\text{recon}} = 4$) are evaluated in `best_group_dp.yaml`.

5. **Space (e), downstream IDS:** Choice between Random Forest (200 trees) and MLP (hidden layers $[128, 64]$, early stopping) for each protocol by validation macro-F1; the rare-class augmentation ratio of TAugR $\in \{0.25, 0.5, 1.0, 2.0\}$ (seed 0); and class-probability weights $w_c \in \{0.25, 0.5, 1.0, 2.0, 4.0\}$, by a two-round coordinate search averaged over three seeds.

Table 3.15: Hyper-parameter search spaces of the model components

Search space	Component	Method	Ranges
(a) Centralised CVAE (phase 7)	CVAE without DP	30 TPE trials, centralised data	$d_z \in \{8,16,32\}$; hidden $\in \{64,128,256\}$; $\beta \in [0.1,2.0]$ (log); $\eta \in [10^{-4}, 3\times10^{-3}]$ (log)
(b) CVAE DP proxy (`tune-dp`)	CVAE with DP-SGD	1 proxy client (20% of data), 30 trials, $\varepsilon = 5$, 1 seed; checked by `verify-dp`	$d_z \in \{8,16,32\}$; hidden $\in \{32,64,128,256\}$; $B_{\text{batch}} \in \{128,256,512,1024\}$; $\beta \in [0.1,2.0]$; $\eta \in [2\times10^{-4}, 5\times10^{-3}]$ (log); $C \in [0.1,20.0]$ (log)
(c) Full federated DP CVAE (`tune-dp-full`, O1)	Federated CVAE with DP-SGD	True federation (5 clients, seed 0); 30/28/12 trials for $\varepsilon \in \{5,1,10\}$	$d_z \in \{8,16,32\}$; hidden $\in \{64,128\}$; $B_{\text{batch}} \in \{256,512,1024,2048\}$; $\beta \in [0.1,2.0]$; $\eta \in [5\times10^{-4}, 2\times10^{-2}]$ (log); $C \in [0.5,50.0]$ (log); $R \in \{15,30,50\}$; $E \in \{1,2,4\}$; `res_clip` $\in \{0.5,1,2,4\}$; `cw_power` $\in \{0,0.5,1\}$; `stat_frac` $\in \{0.02,0.05,0.1\}$
(d) FedDP-Marginal (grid on validation)	Marginal generator	Discrete grid; mean of 3 seeds on validation	$B_{\text{coarse}} \in \{4,8,16,32\}$; $\mathbf{s} \in \{(0.3,0.1,0.6), (0.5,0.1,0.4), (0.15,0.05,0.8)\}$; MI: plug-in, Miller–Madow [62], NMI; tree stability (Jaccard [72]); MGb order 2; refinement $\alpha_{\text{smooth}} \in \{0.1,1,10\}$; policy: $m_{\text{strm}} \in \{1,2,4\}$, $m_{\text{cap}} \in \{25,50,100\}$, $t \in \{5,3\}$ (or $t = 2$, $T_{\text{recon}} = 4$)
(e) Downstream IDS (IDS tuning)	Classifiers and balancing	Grid search on validation; mean of 3 seeds	Model: Random Forest (200 trees) or MLP; TAugR ratio $\in \{0.25,0.5,1.0,2.0\}$; class weights $w_c \in \{0.25,0.5,1.0,2.0,4.0\}$

Multi-Seed Screening Rule and Asymmetries

For the ablations within space (d), candidates are screened with an adaptation of the one-standard-error rule [73]:

$$\text{Macro-F1}_{\text{candidate}} > \text{Macro-F1}_{\text{baseline}} + \text{std}_{\text{seeds}}(\text{baseline}), \tag{3.21}$$

where the means and sample standard deviations (ddof $= 1$) are computed over the validation seeds $\{0,1,2\}$, and the baseline is the main MGs generator at the same ε.

- **A coarse filter, not a test:** This condition only prunes unpromising configurations during the validation search; it is not a formal hypothesis test. Formal comparisons between configurations use the paired bootstrap decision rule [74] defined in Section 3.3.7.

- **Unpaired comparison and expected false passes:** The rule compares candidate and baseline means over independent seeds rather than paired differences per seed. Under the null hypothesis of equal performance and equal variance ($\sigma_{\text{cand}} = \sigma_{\text{base}} = \sigma$), the difference of two three-seed means has variance $\text{Var}(\bar{X}_{\text{cand}} - \bar{X}_{\text{base}}) = 2\sigma^2/3$. The offset $\text{std}_{\text{seeds}}(\text{baseline}) \approx \sigma$ thus corresponds to a standard normal score $z = \sigma/(\sqrt{2/3}\,\sigma) = \sqrt{1.5} \approx 1.22$, that is $p = 1 - \Phi(1.22) \approx 0.11$. In a grid of at least 12 candidates, about 11% of null candidates (about 1.3 candidates) are expected to pass by chance alone. Equation (3.21) is therefore only a high-throughput sieve; the final statistical decision requires the paired bootstrap rule over shared partition seeds (Section 3.3.7).

- **Asymmetry of the tuning protocols:** Because iterative DP-SGD is expensive, the CVAE searches used one seed per trial (space (c) was tuned on seed 0 and then evaluated on all seeds), whereas FedDP-Marginal was selected on the mean of three seeds on the validation split.

- **Reporting:** The final test evaluations are run on the locked test split with seeds $\{0,1,2\}$. Validation and test scores are results and are reported in Chapter 4.

3.3.6 Evaluation Protocols and Metrics

The study never reads a generator through a single number. Each trained generator is judged on what an IDS can do with its data, on how close the data are to the real traffic, on what an

adversary can learn from what is released, and on what the training costs. All scores come from the same locked real test split of 30,000 rows (5,000 per class), which plays no part in training or tuning.

Downstream Classifiers

Two classifiers stand in for the IDS: a Random Forest (RF) [45] with 200 trees and a multilayer perceptron (MLP) with hidden layers $[128, 64]$ and early stopping. Each model classifies one packet at a time from its 59 encoded features. When a single figure per configuration is needed, as in the scorecard, the classifier is the one with the higher macro-F1 on the real validation split. The test split does not influence this choice.

Training Protocols

Table 3.16 lists the protocols. They differ only in what the classifier is trained on.

Table 3.16: Evaluation protocols

Protocol	Training data of the classifier	Question it answers
TRTR	The pooled real training split (B0); B0 with class weights (B1a, RF only); B0 with SMOTE [24] up to 20,000 rows per class (B1b)	What can be reached when raw data can be pooled?
TSTR	Synthetic rows only, 5,000 per class	What are the synthetic data worth when raw data cannot be pooled?
TAug	All real rows, plus synthetic rows that bring every class to at least 20,000 rows	Do synthetic rows add anything to pooled real data?
TAugR	All real rows, plus synthetic rows for the rare classes only, in a ratio $\in \{0.25, 0.5, 1, 2\}$ of the real count chosen on validation	Does a smaller, targeted top-up behave differently?
Local augmentation	The rows of one client, plus synthetic rows of the federation's generator for every class of which the client holds fewer than 5,000 rows	What does a single gateway gain from the federation?

TRTR, TAug and TAugR all need the pooled real training split, so they describe a setting that the threat model of this study excludes. They are kept as references. TSTR is the primary

protocol for the protected configurations. Local augmentation trains one RF per client, giving 15 client models over 5 clients and 3 seeds, and each model is scored on the same real test split.

Utility Metrics

For class c with true positives TP_c, false positives FP_c and false negatives FN_c on the test split, the per-class F1 is

$$\mathrm{F1}_c = \frac{2\,\mathrm{TP}_c}{2\,\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \qquad \text{Macro-F1} = \frac{1}{|\mathscr{C}|} \sum_{c \in \mathscr{C}} \mathrm{F1}_c, \tag{3.22}$$

where $\mathscr{C}$ is the set of six classes. Macro-F1 gives every class the same weight, so a model that ignores the rare classes cannot score well. Three further measures are reported:

- **Binary F1:** the same predictions read as attack against normal, with every attack class counted as "attack" and no retraining. This is the score closest to the binary tasks common in the literature.
- **Rare-class recall:** the mean recall over DELAYED, SYN, INVALID and WILL.
- **Balanced accuracy:** the mean recall over all six classes.

Fidelity Metrics

Fidelity is computed per class in the encoded feature space and averaged over the classes.

- **Wasserstein distance:** the mean one-dimensional Wasserstein distance over the numeric columns, in units of the training standard deviation.
- **Jensen–Shannon divergence:** the mean divergence (base 2) over the binary and missingness columns and the categorical groups, where 0 means identical and 1 means disjoint.
- **Correlation distance:** the distance between the correlation matrices of the real and synthetic rows.
- **Classifier two-sample test (C2ST):** the five-fold cross-validated AUC of a Random Forest asked to tell real rows from synthetic ones. An AUC of 0.5 is the ideal; the heuristic threshold of the configuration is 0.95.

Empirical Privacy Checks

The formal guarantee is ε. The checks below test what an adversary can do in practice and do not replace it. An attack that cannot detect a deliberately exposed target says nothing, so every attack family is paired with a positive control.

- **Duplicate rate:** the share of synthetic rows equal to a training row after rounding to three decimals (threshold ≤ 0.01).
- **Distance-to-closest-record ratio:** the median distance from synthetic rows to training rows divided by the median distance from synthetic rows to validation rows. A value near 1 means that the synthetic rows are no closer to the training data than to unseen data (threshold ≥ 0.5).
- **Membership inference on the synthetic set:** the score of a record is its negative distance to the synthetic set. Members are training rows, non-members are validation rows, and the area under the ROC curve (AUC) is computed per class and averaged.
- **Membership inference with access to the released CVAE:** the score is the fit of the record under the class-conditioned model (negative variational bound, 8 posterior samples), calibrated by subtracting the fit under a reference model trained with the same recipe on the other half of the training pool.
- **Membership inference on the released tables:** the score is the log-likelihood of the record under the released marginal tables of FedDP-Marginal, calibrated in the same way by a reference model. The true-positive rate at 1% false positives is reported alongside the AUC.

A membership-inference AUC above 0.55 is treated as a detection. This threshold is a heuristic of the configuration.

Cost Metrics

Bytes per round are counted on the Flower grid for all stages of the protocol. For plain FL with a dense model they equal the size of the float32 model times the number of clients times two directions. Total megabytes are the bytes per round times the number of rounds. Time per round is the median of rounds 2 and later, because round 1 includes the start-up of Ray; it is compute time on the simulation machine.

3.3.7 Statistical Decision Rule

Let Δ be the difference of a test metric between configurations A and B, averaged over the seeds $s \in \{0,1,2\}$. A difference is said to have an effect only if both conditions hold:

$$0 \notin \mathrm{CI}_{95\%}(\Delta) \quad \text{and} \quad |\Delta| > \max\left(\mathrm{sd}_s(A), \mathrm{sd}_s(B)\right), \tag{3.23}$$

where sd_s is the standard deviation over the seeds with ddof $= 0$, as in every table of this thesis. The verdict is *better* or *worse* according to the sign of Δ, and *no effect* otherwise.

The interval comes from a paired percentile bootstrap [74]. The 30,000 real test rows are resampled with replacement, stratified by class, with the same 1,000 resamples for every run, so that the two configurations of a comparison are evaluated on identical resamples. Packets of one TCP stream are correlated, which makes this interval too narrow. Every verdict is therefore repeated with a bootstrap over whole streams (20,485 streams in the test split), and any verdict that changes is marked with a dagger (†).

The rule has known limits, stated here so that the results are read with them. The interval reflects the sampling of the test rows and three training seeds. It does not reflect which capture groups form the test split, the sampling of the training rows, or the choice of hyper-parameters. Branch A4 covers the first of these only in part. The rule uses no p-values and no correction for multiple comparisons, so a verdict is a statement about this test split and these seeds.

3.3.8 Experimental Environment and Reproducibility

Environment

All experiments ran on one Windows 11 machine (build 10.0.26200) with an Intel x86-64 CPU and 16 GB of RAM, using the CPU only. Federated learning is simulated with Flower and Ray; each client is a Ray actor sharing the machine's CPU, so no network latency is measured (see Section 3.4). The values come from the environment record `results/repro/environment.json` and the project's deviation log (Table 3.17).

Random Seeds

Every experiment is run with seeds $\{0,1,2\}$. The seed controls weight initialisation, the client partition, and the sampling of synthetic data. `utils.set_seed` seeds Python's `random`, NumPy, and PyTorch; the federated and DP code additionally derives per-client and per-round generators from the seed.

Table 3.17: Experimental environment specification

Component	Value
Operating system	Windows 11, build 10.0.26200 (AMD64)
Processor	Intel64 Family 6 Model 186 Stepping 2 (GenuineIntel); CPU only, 10 threads in benchmark
Memory	16 GB
Language	Python 3.13.5
Federated simulation	Flower 1.39.0 [46]; Ray 2.59.0
Deep learning and DP	PyTorch 2.14.1; Opacus 1.6.0 [47]
Machine learning and data	scikit-learn 1.9.1; imbalanced-learn 0.14.2; NumPy 2.3.2; pandas 2.3.1; SciPy 1.18.1; PyArrow 25.0.1
Hyper-parameter search	Optuna 5.0.0 [54] (TPE sampler [55])
Other	Joblib 1.6.0; Matplotlib 3.10.5; PyYAML 6.0.3; Streamlit 1.65.0; psutil 7.2.2

Measures Supporting Reliability and Reproducibility

- **Central run ledger:** Each run appends one row to `results/runs.csv` with a configuration hash and the Git commit of the code that produced it. The ledger rows of the main study carry commit `a4c4601` and, for 407 of the 431 configurations, configuration hash `8c86c0b143d4`.
- **Checkpoint and resume:** Long training jobs save state; re-running a command compares against the ledger and skips completed configurations.
- **Automated reporting:** The tables, intervals, and figures of the final report are generated by code (`ppfeddata aggregate`, `ppfeddata interpret`); no result is typed by hand. A check recomputes macro-F1 from the saved prediction files and compares it with the ledger.
- **Reproducibility bundle:** `results/repro/` holds the exact package list (`pip_freeze.txt`, identical to `requirements-lock.txt`), the configuration files, the environment record, and a manifest with SHA-256 digests of the ledger, the split and schema manifests, and the report.
- **Acceptance command:** `python -m ppfeddata.cli accept` checks the repository

against the Definition of Done and writes a checklist of twelve items (D1–D10, with D6 split into three). This command does not run the test suite; the suite is run separately.

- **Clean-copy re-execution:** Phases 1–5 (inventory, harmonisation, sampling, preprocessing, leakage check) were re-run from the raw data in a clean copy of the repository with a fresh virtual environment, and the processed train, validation, and test arrays were compared with those of the main checkout. Phases 6–14 were not re-run in the clean copy.
- **Controlled trial:** A trial run of all ten configurations of the base matrix with seed 0 was compared row by row with the ledger of the main checkout (`SPEC_DEVIATIONS.md`, 11.6).
- **Leakage checks:** The data pipeline is guarded by five checks (C1–C5, Section 3.2.6), and the real test split is verified against a stored hash.

3.4 Limitations of the Methodology

The limitations below arise from the design of the study and the scope of the techniques. The full list is maintained in `limitations.py`, which feeds the final report and the README; this section groups them. None of the statements below reports a result.

Data and Features

- Only one dataset is used; the study offers no evidence about other traffic or networks.
- The release has no CSV of the Normal traffic, so it was extracted from the original PCAP files with the authors' export settings. The one attack capture exported by the authors with another filter, SYN_DDoS-AD_1, is used as published, so the SYN class also holds packets sent by the broker (Section 3.2.1).
- Records are single packets, and the generators produce packets independently. Session-level temporal structure is therefore not reproduced, and the time-based features depend on how the captures were exported.
- Eight of the eleven sub-classes originate from a single capture file, so their validation and test rows are consecutive blocks of the same capture and not other captures. Removing

TCP streams that straddle two blocks (`drop_cross_block_streams`) reduces leakage between splits but biases the sample toward short connections; the share removed per sub-class is documented in the split manifest.

- The pipeline keeps only rows with protocol TCP or MQTT, keeps the first element of multi-valued cells, clips the tail of the time-gap feature at 5σ, and treats nearly empty columns with the statistics of the rows where they apply plus an indicator.
- DoS and DDoS are merged in the six-class mode used for the experiment matrix.
- The validation and test splits are class-balanced (5,000 test rows per class) and do not follow the class proportions of real traffic.

Privacy Scope

- ε is stated at record level (one packet) by default. Stream- and capture-level protection is optional for FedDP-Marginal and requires each unit to be held by one client and capped at m rows. Under record-level ε, an attack session or capture is protected only through the weaker group bound [51].
- Class labels, class counts, and client sizes are not protected (Section 3.1.3).
- Normalisation statistics are computed centrally on the pooled training split, as if each client had shared its data once. A real federation would have to compute them securely or fix them beforehand.
- Hyper-parameter selection on the real validation split is not private and lies outside ε.
- Distributed DP assumes that enough clients add their noise share; the guarantee degrades as the number of honest clients falls, which is why ε is reported as a function of h.
- Accounting differs between variants: the Skellam variant uses the RDP bound of Agarwal et al. [22] and the RDP-to-(ε, δ) conversion [64, Th. 21] used by Opacus [47], while the Gaussian variant uses zCDP [53] with a looser conversion, so part of the difference between the two at the same ε comes from the conversion and not from the mechanism.
- Malicious gateways (poisoning, Byzantine behaviour) are outside the threat model.

Generators

- FedDP-Marginal represents dependencies with a Chow–Liu tree [10], so each attribute depends on one parent; a second-order structure (MGb) [9] is only an option.
- Only one CVAE architecture is studied, and its hyper-parameter search used one seed per trial.

Evaluation and Statistics

- Three seeds are used for every configuration. Federated training can be sensitive to very small numerical perturbations, so a standard deviation over three seeds is a rough estimate.
- The bootstrap intervals and decision rules reflect the sampling of test rows (or whole streams) and of training seeds; they do not cover the choice of capture groups, the hyper-parameters, or the data sampling. Sensitivity A4 explores only two alternative choices of validation and test groups.
- The decision thresholds (for example the membership-inference AUC bound of 0.55, the ε bound of 5, and the overhead ratio of 3) are heuristics fixed by the research team and not derived from data.
- Each membership-inference family tests one attack at record level and does not test group-level inference; each attack must pass a positive control to be informative.
- The decision rule does not use p-values or a multiple-comparison correction.

Execution Platform

- Federated learning is simulated on one CPU machine with Ray actors; no network latency, packet loss, or constrained-device behaviour is measured, and the reported times per round are compute times on this machine.

Baselines Not Compared and Parts Not Run

- **Not compared:** FLAIM and AIM (the Private-PGM dependency was not installed), DP-CTGAN or other deep generators in the federation, and a discrete (Skellam) mechanism for the classifier trained directly with distributed DP.
- **Written but not run after the change of primary generator:** M3f (distributed DP-FedSGD for the CVAE) and the SecAgg$^+$ quantisation-width sweep for the CVAE.

Chapter 4

Experimental and Results

This chapter reports the results of the experiments designed in Chapter 3 and answers the four research questions of Section 1.3 in order. Every number comes from the run ledger and the reports generated by the project code. All results use the data of Section 3.2, with the Normal traffic extracted under the export settings of the dataset authors.

4.1 Introduction

The run ledger holds 1,237 rows for 431 configurations, most of them run with three seeds. A one-seed trial run re-executed the base matrix from scratch and reproduced 53 of 53 ledger rows exactly; only the byte counts of secure aggregation runs differ, by up to 31 bytes, because they depend on random keys. The macro-F1 recomputed from the saved test predictions equals the ledger value to within 8×10^{-17}.

Four conventions apply to every table and figure below.

- **Scores:** utility is measured on the locked real test split of 30,000 rows (5,000 per class) and is written as mean $\pm$ standard deviation over the seeds $\{0, 1, 2\}$, with ddof $= 0$.
- **Classifier:** a row labelled with a generator reports the classifier, RF or MLP, that had the higher macro-F1 on the real validation split. Where both are shown, the classifier is named.
- **Privacy budget:** ε is stated at record level (one packet) with $\delta = 10^{-5}$ unless a unit is named. For distributed noise it is the value when every gateway adds its share; the value for fewer honest gateways is given separately.
- **Verdicts:** "better", "worse" and "no effect" have the meaning of Equation (3.23). A dagger (†) marks a verdict that changes under the bootstrap over whole TCP streams.

Names Used in This Chapter

Three configurations are the main ones of the framework: MG-eps1 is MGs-eps1, MG-eps5 is MGr-eps5, and MG-eps10 is MGs-eps10. MGs, MGr, MGd and MGl are the variants of Table 3.13. On the CVAE side, B3 has no protection, M2 adds SecAgg$^+$, M1 adds DP-SGD, and M3 combines both. The suffix "o" (M1o, M3o) marks the settings searched at full federated scale in round O1. For each ε, the CVAE row used against FedDP-Marginal is the best valid CVAE route on validation: M1-eps1, M3o-eps5 and M1o-eps10.

Organisation

Section 4.2 presents the data as they were used, the leakage checks, the settings selected on validation, and the consolidated scorecard. Section 4.3 analyses the results per research question. Section 4.4 explains the patterns, Section 4.5 compares them with published work, and Section 4.6 draws the practical consequences.

4.2 Presentation of Data

4.2.1 The Data as Used

The dataset holds 39,000,601 packets (Table 3.7), of which 130,500 are sampled into the training, validation and test splits under the quotas of Table 3.10. The real test split comes from 11 capture groups and 20,485 TCP streams. The training split has 77,106 TCP streams and 147 capture groups, so a stream contributes 1.15 rows on average, while one capture contributes up to 1,277 rows. No TCP stream appears in more than one split.

Figure 4.1 shows the non-IID partition of the first seed. With Dirichlet $\alpha = 0.5$, the five gateways hold between 3,666 and 33,618 rows in this seed, and over the three seeds the client sizes range from 2,862 to 69,578 rows. Several gateways hold almost no rows of some rare sub-classes. Client 1, for example, has no SYN_DoS row. This is the situation the federation is meant to address.

4.2.2 Leakage and Label-Reliability Checks

Table 4.1 gives the outcome of the five checks of Table 3.11. Only one flag was raised.

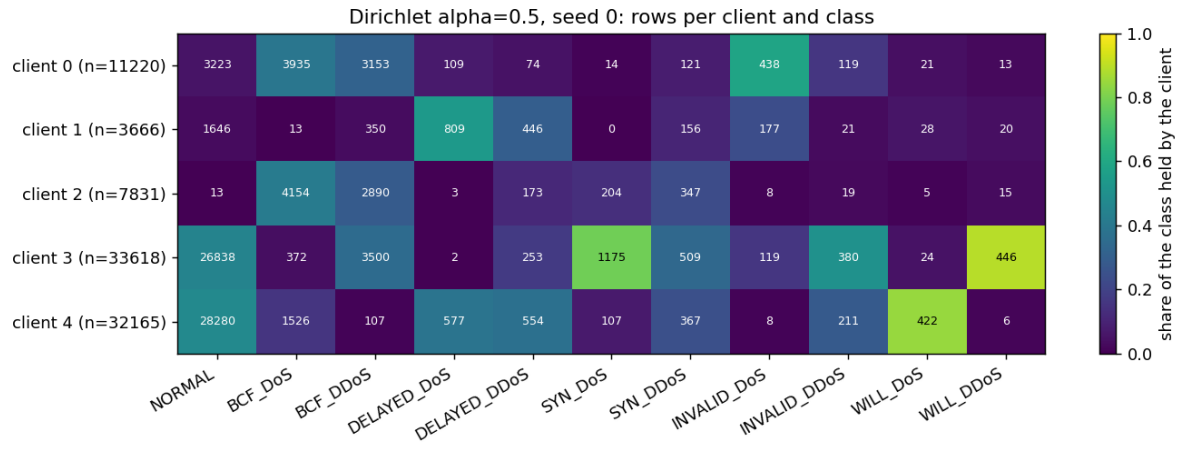


Figure 4.1: Non-IID partition of the training rows over the five gateways (Dirichlet $\alpha = 0.5$, seed 0). Each cell gives the number of rows of a sub-class at a gateway; the colour gives the share of that sub-class held by the gateway.

Table 4.1: Results of the leakage and label-reliability checks (Random Forest with 100 trees, validation split, three seeds)

Check	Result	Decision
C1	Five columns are empty in some sub-classes and populated in others. Of these, only `requested_qos`, `will_message_length` and `will_topic_length` are model features, with a presence of at most 0.166	Kept: they follow from the protocol
C2	Macro-F1 from the 12 `_is_na` flags alone: 0.1775 ± 0.0000, against 0.167 for chance	Not flagged (threshold 0.5)
C3	Best single column: `frame_length_on_wire_2`, macro-F1 0.2972. Best per-class F1 from one column: 0.4134 (`tcp_segment_len`, INVALID)	Not flagged (threshold 0.9)
C4	Five-fold macro-F1 by random row 0.5394, by stream 0.5365, by capture group 0.5308. Gaps: random minus group $+0.0086$, random minus stream $+0.0029$	Not flagged (threshold 0.05)
C5	DoS against DDoS, macro-F1: BCF 0.5915, DELAYED 0.6151, WILL 0.6841, INVALID 0.7871, SYN 0.9630	BCF flagged (threshold 0.6); the others are separable

Three facts from these checks shape the rest of the chapter. First, no column gives the class away, so the task is not artificially easy. Second, the detection task is hard: an RF on real data reaches a validation macro-F1 of 0.4482 ± 0.0009, or 0.5002 ± 0.0002 with class-balanced weights. Third, the time columns carry much of the signal. Without `irtt`, `time_delta_from_previous_displayed_frame` and `time_since_first_frame_in_this_tcp_stream`, the class-balanced macro-F1 falls to 0.3464, and the F1 of DELAYED falls from 0.418 to 0.001.

Check C5 also bears on the six-class merge. Only BCF falls below the threshold, so the merge does not rest on the two variants being inseparable. It is a design choice that keeps enough rows per class, and Section 4.3.3 reports the eleven-class results for reference.

4.2.3 References on Real Data

Table 4.2 gives the references. They train on the pooled real split and therefore need what the threat model forbids. They mark how far a detector can go on these features when pooling is allowed.

Table 4.2: References trained on the pooled real training split (TRTR)

Reference	Macro-F1	Balanced accuracy	Rare-class recall
B0, real data only, RF	0.448 ± 0.000	0.481	0.285
B0, real data only, MLP	0.343 ± 0.006	0.418	0.190
B1a, class weights, RF	0.496 ± 0.000	0.518	0.355
B1b, SMOTE, RF	0.505 ± 0.002	0.516	0.381
B1b, SMOTE, MLP	0.477 ± 0.014	0.485	0.365

The B0 RF recalls NORMAL at 0.964 and BCF at 0.780, but only DELAYED at 0.231, SYN at 0.472, INVALID at 0.161 and WILL at 0.276. Each of the rare classes is hard to separate packet by packet, even for a model trained on all the real rows.

4.2.4 Settings Selected on Validation

Table 4.3 lists the settings selected for FedDP-Marginal over three seeds on the validation split, together with the CVAE routes tuned in round O1 on one seed. The two generator families were not tuned with the same effort per candidate, an asymmetry stated in Section 3.3.5.

Table 4.3: Settings selected on the validation split

ε	**FedDP-Marginal setting**	**Coarse bins**	**Budget split s**
1	MGs	16	$(0.3, 0.1, 0.6)$
5	MGr: inverse-variance fusion, IPF, smoothing $\alpha = 0.1$, root by noisy MI	16	$(0.15, 0.05, 0.8)$
10	MGs	8	$(0.5, 0.1, 0.4)$

At $\varepsilon = 5$ and $\varepsilon = 10$, the two best FedDP-Marginal candidates differ by only 0.000016 and 0.000008 on validation. Both gaps are below the weight of one validation packet in the macro-F1, which is $1/(6 \times 2{,}000) = 0.00008$. The selected settings at these budgets should therefore be read as members of a tied group, not as clear winners.

4.2.5 Consolidated Scorecard

Table 4.4 gathers the main configurations. Appendix A holds all rows of the scorecard. Figure 4.2 shows the matrix of the two classifiers under TAug and TSTR.

Figure 4.3 draws the TSTR macro-F1 of the main routes against the privacy budget. The curves of FedDP-Marginal lie above those of the CVAE at every budget.

Table 4.4: Main configurations under TSTR. An asterisk marks byte counts estimated from the number of cells, because MGl runs in process.

Config.	Clf.	ε	Macro-F1	Binary F1	Rare recall	MB/round	Rounds	MB/run
B3 (CVAE, no protection)	MLP	∞	0.413 ± 0.005	0.904	0.361	4.20	30	126.1
M2 (CVAE, SecAgg^{+})	MLP	∞	0.419 ± 0.008	0.896	0.361	6.37	30	191.1
M1-eps1	MLP	1	0.176 ± 0.008	0.750	0.068	0.56	30	16.9
M3o-eps5	MLP	5	0.254 ± 0.013	0.814	0.132	0.91	15	13.6
M1o-eps10	MLP	10	0.275 ± 0.015	0.799	0.146	1.45	50	72.4
MGl-eps1 (local DP)	RF	1	0.267 ± 0.021	0.774	0.117	0.38*	3	1.13*
MGl-eps5 (local DP)	RF	5	0.397 ± 0.006	0.825	0.279	0.35*	3	1.05*
MGl-eps10 (local DP)	MLP	10	0.406 ± 0.015	0.828	0.277	0.31*	3	0.94*
MGs-eps1 (distributed)	RF	1	0.366 ± 0.018	0.813	0.231	0.18	3	0.53
MGr-eps5 (distributed)	MLP	5	0.425 ± 0.004	0.824	0.301	0.17	3	0.51
MGs-eps10 (distributed)	MLP	10	0.433 ± 0.004	0.843	0.301	0.16	3	0.48
MGp-nodp (plain Flower)	MLP	∞	0.459 ± 0.007	0.885	0.367	0.11	3	0.34
MGs-nodp (SecAgg^{+})	MLP	∞	0.459 ± 0.007	0.885	0.367	0.16	3	0.47

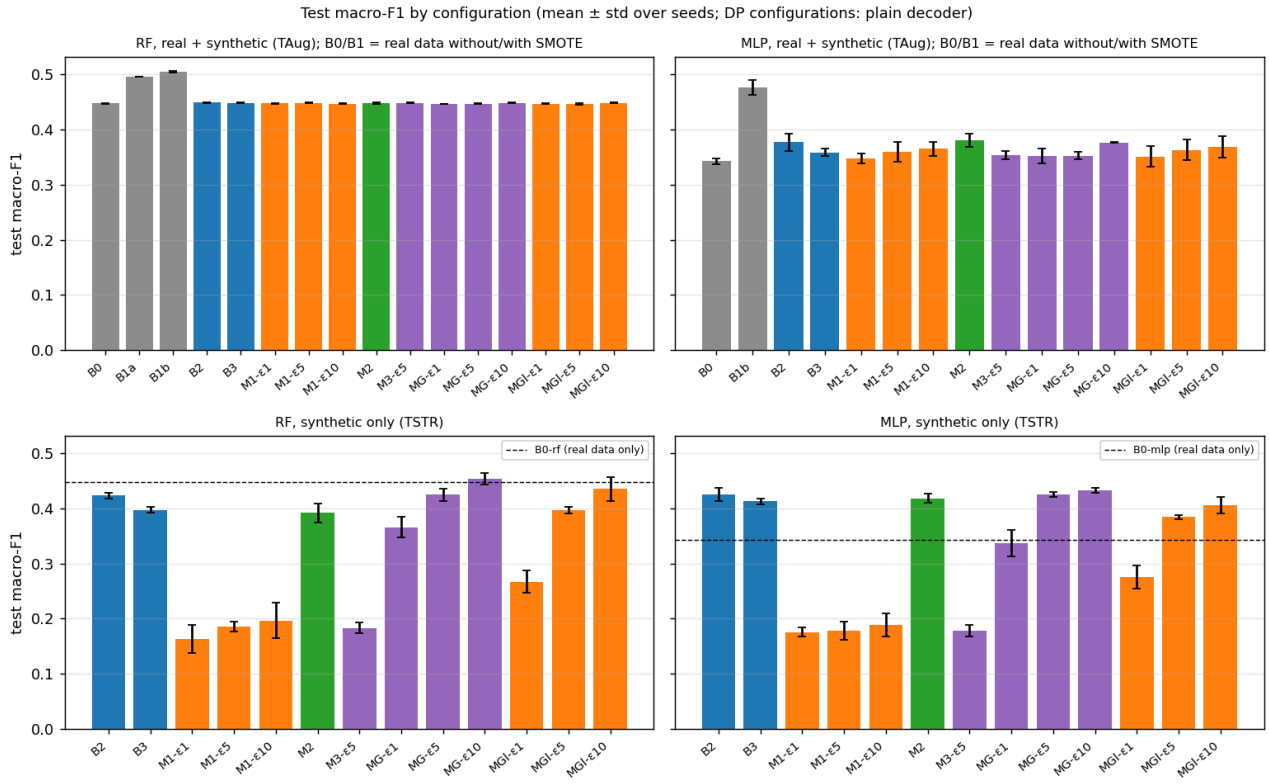


Figure 4.2: Test macro-F1 by configuration and classifier. Upper row: real plus synthetic data (TAug) and the real-data references. Lower row: synthetic data only (TSTR), with the B0 level as a dashed line. MG-ε denotes the main FedDP-Marginal configurations.

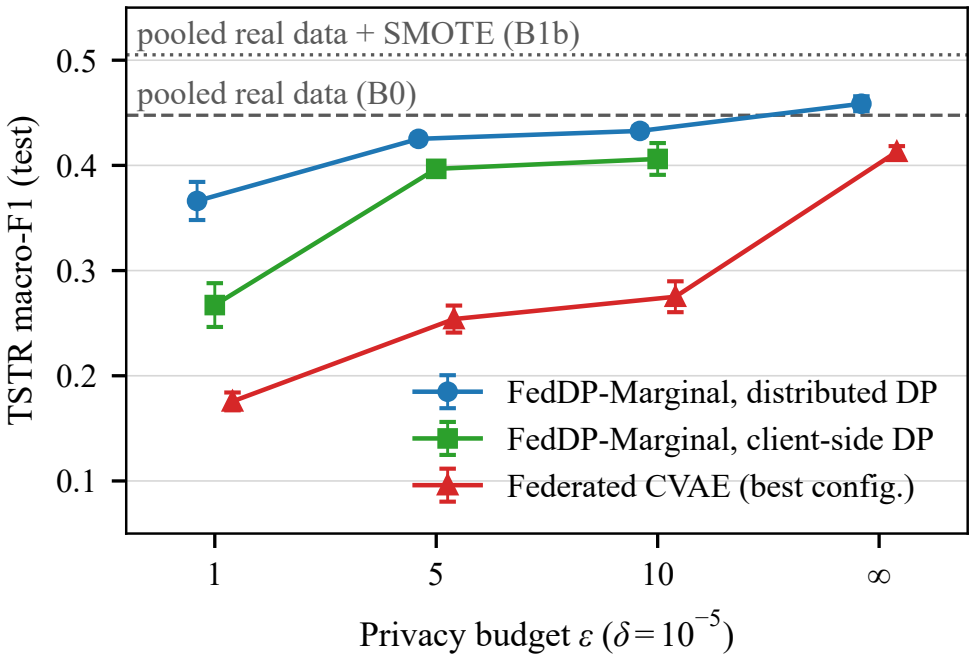


Figure 4.3: TSTR macro-F1 against the privacy budget ε for the main routes (mean over three seeds, with the seed standard deviation), with the reference levels on pooled real data.

4.3 Analysis of Results

4.3.1 RQ1: Utility of the Synthetic Data for the IDS

Synthetic Data Alone (TSTR)

Where the raw traffic cannot be pooled, the classifier has only the synthetic data. Table 4.4 shows what remains. Without DP, FedDP-Marginal reaches 0.459 and the CVAE route reaches 0.413. With DP, FedDP-Marginal keeps 0.366, 0.425 and 0.433 at $\varepsilon = 1, 5, 10$, close to the RF on pooled real data (0.448). The SMOTE and class-weight references on pooled real data stay higher, at 0.505 and 0.496.

A TSTR score at or above the B0 level does not mean that synthetic data are better than real data. Red flag F2 of the protocol (TSTR above TRTR) was triggered for the MLP, whose B0 reference is low because it is trained on the imbalanced split, while the synthetic set holds 5,000 rows of every class. Against a class-balanced real reference, the flag disappears: no TSTR score reaches B1b-RF (0.505) or B1b-MLP (0.477).

Adding Synthetic Rows to Pooled Real Data (TAug and TAugR)

Table 4.5 gives the gain from adding synthetic rows to the pooled real data. For the RF, 2 of the 13 generators pass the decision rule and the largest gain is $+0.0012$, while SMOTE gains $+0.0575$ and class weights $+0.0483$. For the MLP, 9 of the 13 generators pass, with a largest gain of $+0.0378$, against $+0.1339$ for SMOTE. The recall of the four rare classes under the RF moves by at most 0.007 for any generator and class, against $+0.070$ and $+0.096$ (mean over the four classes) for class weights and SMOTE.

The targeted variant does no better. In the scorecard, the gain of TAugR over B0 lies between -0.001 and $+0.001$ for every generator, with the ratio chosen on validation.

The CVAE Against the Simple Methods

TAug with the CVAE (B2 centralised, B3 federated) against class weights and SMOTE gives six comparisons, and all six are worse, with a macro-F1 difference between -0.1177 and -0.0471 and a rare-class recall difference between -0.167 and -0.068. When real data can be pooled, the two simple methods are the better choice in this study.

Table 4.5: TAug: difference of test macro-F1 against real data only (B0), with the 95% paired bootstrap interval

Configuration	RF: Δ [95% CI]	Verdict	MLP: Δ [95% CI]	Verdict
B1a (reference)	$+0.0483$ $[+0.0449, +0.0514]$	better	not applicable	
B1b (reference)	$+0.0575$ $[+0.0538, +0.0613]$	better	$+0.1339$ $[+0.1289, +0.1384]$	better
B2	$+0.0012$ $[+0.0001, +0.0024]$	better	$+0.0343$ $[+0.0319, +0.0365]$	better
B3	$+0.0011$ $[+0.0000, +0.0023]$	better †	$+0.0162$ $[+0.0140, +0.0184]$	better
M1-eps1	-0.0000 $[-0.0010, +0.0008]$	no effect	$+0.0049$ $[+0.0030, +0.0069]$	no effect
M1-eps5	$+0.0011$ $[+0.0002, +0.0020]$	no effect	$+0.0170$ $[+0.0153, +0.0186]$	no effect
M1-eps10	-0.0006 $[-0.0015, +0.0004]$	no effect	$+0.0223$ $[+0.0201, +0.0243]$	better
M2	$+0.0005$ $[-0.0005, +0.0016]$	no effect	$+0.0378$ $[+0.0353, +0.0406]$	better
M3-eps5	$+0.0005$ $[-0.0004, +0.0014]$	no effect	$+0.0114$ $[+0.0097, +0.0129]$	better
MG-eps1	-0.0010 $[-0.0020, +0.0001]$	no effect	$+0.0096$ $[+0.0079, +0.0112]$	no effect
MG-eps5	-0.0008 $[-0.0020, +0.0002]$	no effect	$+0.0101$ $[+0.0082, +0.0120]$	better
MG-eps10	$+0.0009$ $[-0.0002, +0.0021]$	no effect	$+0.0341$ $[+0.0313, +0.0368]$	better
MGl-eps1	-0.0004 $[-0.0014, +0.0006]$	no effect	$+0.0085$ $[+0.0064, +0.0107]$	no effect
MGl-eps5	-0.0012 $[-0.0022, -0.0001]$	worse	$+0.0205$ $[+0.0182, +0.0227]$	better
MGl-eps10	$+0.0011$ $[-0.0000, +0.0022]$	no effect	$+0.0256$ $[+0.0232, +0.0281]$	better

What Federation Costs a Non-Private CVAE

Moving from the centralised CVAE (B2) to five non-IID clients (B3) costs $+0.0257$ $[+0.0225, +0.0291]$ of macro-F1 under TSTR with the RF, which is a relative loss of 6.1%. Under TSTR with the MLP the cost is $+0.0129$ (3.0%), and under TAug with the MLP it is $+0.0181$ (4.8%). TAug with the RF shows no effect ($+0.0001$).

Local Augmentation at Each Gateway

Table 4.6 shows the setting where synthetic rows give the most. A gateway that has seen little of some classes trains its own RF and receives synthetic rows of the classes it holds fewer than 5,000 rows of. The mean macro-F1 of the 15 client models rises from 0.384 ± 0.055 to 0.423, 0.426 and 0.427 with the main FedDP-Marginal configurations. Those are gains of $+0.039$, $+0.042$ and $+0.043$, with 11, 13 and 13 of the 15 client models gaining. Losses are at most 0.005 and gains reach 0.139.

IDS-Side Options

Class-probability weights chosen on validation, and the use of all 20,000 synthetic rows per class, act on the classifier and leave the generator unchanged. Their effect depends on the

Table 4.6: Local augmentation at each gateway (RF, 15 client models over 5 clients and 3 seeds, mean ± standard deviation)

Training data of a client	Macro-F1	Rare recall	Gain over local	Clients better
Local real rows only	0.384 ± 0.055	0.297	n/a	n/a
Local + B3 (CVAE, no protection)	0.425 ± 0.044	0.311	$+0.041$	15 / 15
Local + M2 (CVAE, SecAgg$^+$)	0.423 ± 0.046	0.310	$+0.039$	15 / 15
Local + M1o-eps5 (CVAE, DP)	0.398 ± 0.053	0.295	$+0.014$	10 / 15
Local + MG-eps1	0.423 ± 0.045	0.308	$+0.039$	11 / 15
Local + MG-eps5	0.426 ± 0.044	0.311	$+0.042$	13 / 15
Local + MG-eps10	0.427 ± 0.043	0.313	$+0.043$	13 / 15
Local + MGl-eps5	0.422 ± 0.044	0.307	$+0.038$	12 / 15

generator. For MG-eps1, the RF rises from 0.365 to 0.406 and the rare-class recall from 0.235 to 0.311; the MLP rises from 0.344 to 0.414 with both options. For MG-eps5 and MG-eps10, the changes are small and mixed in sign. The weights correct the tendency of an IDS trained on synthetic rows to predict NORMAL too often. They need labelled real validation data, and they must be chosen again for another class mix, so they are reported as an option only.

The Alternative of Training the Detector Directly

A federation that needs only a detector can train the classifier itself by FedAvg, with no generator. Table 4.7 compares this route with the generators at the same protection. With the first group of settings (DP at each client), the direct MLP is above the CVAE route under DP. With distributed DP, where the noise is split over the gateways as in FedDP-Marginal, the direct MLP (DP-FedSGD) is not different from the framework at $\varepsilon = 1$ and below it at $\varepsilon = 5$ and $\varepsilon = 10$.

The two routes cost very different amounts of communication. DP-FedSGD needs 2,593 model rounds and about 7.7 GB in total over the clients (an estimate from the bytes per parameter measured for M2), against three rounds and 0.48 to 0.53 MB for FedDP-Marginal. Its epsilon with a single honest gateway grows to 3.1, 58.6 and 142.0, against 2.4, 14.2 and 33.1 for FedDP-Marginal. Without noise, the same recipe reaches 0.467 ± 0.008, against 0.459 for FedDP-Marginal without DP. That difference of $+0.008$ $[+0.004, +0.012]$ passes the rule. Federated SMOTE, with no generator and no protection, reaches 0.445 ± 0.016.

Table 4.7: The classifier trained directly by federated learning, compared with the generator routes at the same protection (TSTR with the MLP on the generator side)

Direct route and protection	Direct MLP	Generator route	Δ [95% CI]	Verdict
Against the CVAE route				
No protection	0.425 ± 0.026	0.413	$+0.012$ $[+0.008, +0.015]$	no effect
SecAgg^{+} (numerical emulation)	0.416 ± 0.021	0.419	-0.003 $[-0.006, +0.000]$	no effect
DP at each client, $\varepsilon = 1$	0.245 ± 0.008	0.176	$+0.069$ $[+0.066, +0.073]$	better
DP at each client, $\varepsilon = 5$	0.276 ± 0.022	0.178	$+0.098$ $[+0.093, +0.102]$	better
DP at each client, $\varepsilon = 10$	0.290 ± 0.024	0.189	$+0.101$ $[+0.097, +0.106]$	better
Against FedDP-Marginal, noise split over the gateways in both routes				
DP-FedSGD, $\varepsilon = 1$	0.350 ± 0.010	0.337	$+0.013$ $[+0.010, +0.016]$	no effect
DP-FedSGD, $\varepsilon = 5$	0.369 ± 0.001	0.425	-0.057 $[-0.060, -0.053]$	worse
DP-FedSGD, $\varepsilon = 10$	0.389 ± 0.011	0.433	-0.043 $[-0.047, -0.040]$	worse

Answer to RQ1

When real data can be pooled, synthetic data do not improve the IDS in any way that matters in this study, and class weights or SMOTE do better. When raw traffic cannot be pooled, the data from FedDP-Marginal keep a TSTR macro-F1 between 0.366 and 0.433 at ε from 1 to 10, which is 82% to 97% of the RF on pooled real data. The most useful role is local augmentation, where a gateway with few rows of some classes gains about 0.04 of macro-F1.

4.3.2 RQ2: FedDP-Marginal Against the Federated CVAE

Utility at the Same Budget

Table 4.8 applies the decision rule to the difference between FedDP-Marginal and the best CVAE route at each budget. All 12 comparisons are "better" under both the record and the stream bootstrap.

The local variant (MGl) gives the cleanest comparison, because it adds noise at each gateway exactly as the CVAE does, so the difference comes from the generator and not from the noise split. Moving to distributed noise then adds about 0.03 to 0.10 of macro-F1 at the same ε.

Table 4.8: FedDP-Marginal minus the best CVAE route with DP (TSTR on the real test split). All verdicts are "better" under both bootstraps.

ε	Comparison	Δ macro-F1 [95% CI]	Largest seed sd	Δ rare recall [95% CI]
1	MGl-eps1 − M1-eps1	$+0.092$ $[+0.088, +0.095]$	0.021	$+0.049$ $[+0.046, +0.053]$
1	MGs-eps1 − M1-eps1	$+0.191$ $[+0.187, +0.195]$	0.018	$+0.163$ $[+0.160, +0.168]$
5	MGl-eps5 − M3o-eps5	$+0.143$ $[+0.139, +0.147]$	0.013	$+0.147$ $[+0.143, +0.152]$
5	MGr-eps5 − M3o-eps5	$+0.171$ $[+0.167, +0.176]$	0.013	$+0.169$ $[+0.163, +0.174]$
10	MGl-eps10 − M1o-eps10	$+0.131$ $[+0.127, +0.135]$	0.015	$+0.131$ $[+0.126, +0.136]$
10	MGs-eps10 − M1o-eps10	$+0.158$ $[+0.154, +0.162]$	0.015	$+0.155$ $[+0.150, +0.160]$

What DP Costs Each Generator

DP costs the CVAE far more than it costs FedDP-Marginal. Against the CVAE without DP with the same plain decoder, M1 at $\varepsilon = 1, 5, 10$ is worse in 11 of 12 comparisons, and the loss in TSTR macro-F1 runs up to 58%. Against its own $\varepsilon = \infty$, FedDP-Marginal loses -0.076 (RF) and -0.122 (MLP) at $\varepsilon = 1$, -0.017 and -0.033 at $\varepsilon = 5$, and changes by $+0.012$ (RF) and -0.026 (MLP) at $\varepsilon = 10$.

Re-tuning the CVAE per budget at full federated scale (round O1) did not close this gap. It lifts the best valid CVAE route to 0.254 at $\varepsilon = 5$ and 0.275 at $\varepsilon = 10$, against 0.178 and 0.189 for the first search. At $\varepsilon = 1$, the setting chosen on validation does worse on the test split (0.167 for M1o-eps1 with the RF), so the baseline M1-eps1 remains the best CVAE route there.

Per-Class Behaviour and the Working Hypotheses

Table 4.9 and Figure 4.4 break the scores down by class.

H1 predicted that at $\varepsilon = 1$ the marginal generator would reach a higher class-level F1 on the classes defined by discrete header fields (INVALID, WILL). The point estimates support it: MGs-eps1 minus M1-eps1 is $+0.139$ for INVALID and $+0.222$ for WILL, and both exceed the largest seed standard deviation of the two configurations (0.086 and 0.073). **H2** predicted the reverse for the volumetric classes (BCF, SYN). The results do not support it: the difference is $+0.138$ for BCF and $+0.312$ for SYN, again in favour of FedDP-Marginal and above the largest seed standard deviation (0.047 for both). Per-class differences were compared with the seed spread only; the paired bootstrap of Section 3.3.7 was applied to the macro-F1 and to the mean rare-class recall.

Table 4.9: Test F1 per class (mean over three seeds; classifier chosen on validation)

Configuration	NORMAL	BCF	DELAYED	SYN	INVALID	WILL
B0, RF (real data only)	0.521	0.559	0.362	0.560	0.256	0.428
B1b, RF (SMOTE)	0.548	0.585	0.450	0.556	0.381	0.511
B3 (CVAE, no protection)	0.623	0.318	0.418	0.426	0.336	0.356
M1-eps1	0.339	0.319	0.102	0.095	0.042	0.156
M3o-eps5	0.510	0.391	0.125	0.115	0.096	0.287
M1o-eps10	0.499	0.408	0.119	0.238	0.199	0.188
MGs-eps1	0.523	0.457	0.250	0.407	0.181	0.378
MGr-eps5	0.530	0.459	0.311	0.445	0.377	0.429
MGs-eps10	0.539	0.482	0.321	0.444	0.419	0.391
MGs-nodp	0.585	0.485	0.379	0.429	0.432	0.440

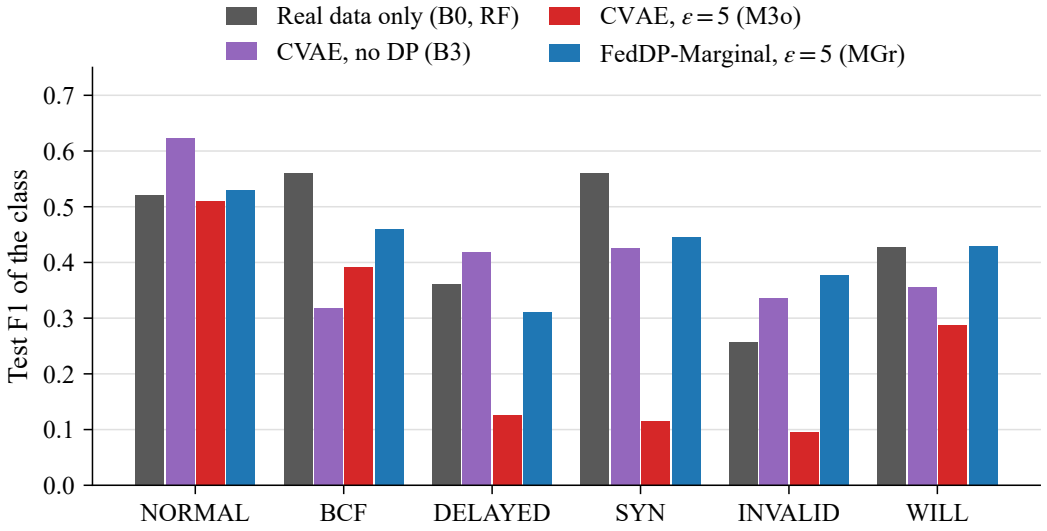


Figure 4.4: Test F1 per class for the real-data reference, the CVAE without DP, and the two generator routes at $\varepsilon = 5$.

Fidelity

Table 4.10 gives the fidelity measures. FedDP-Marginal is closer to the real numeric marginals than the CVAE with DP at the same budget: the Wasserstein distance at $\varepsilon = 5$ is 0.037 against 0.194, and at $\varepsilon = 10$ it is 0.049 against 0.176. The distance at $\varepsilon = 5$ is below the one at $\varepsilon = 10$; the two main configurations are not the same variant (MGr with fusion and 16 bins at $\varepsilon = 5$, MGs with 8 bins at $\varepsilon = 10$), so the two values are not points on one curve.

No generator produces rows that a classifier cannot tell from real ones. The C2ST AUC is between 0.9995 and 1.0000 in every row, DP or not, deep or marginal. Utility for a detector therefore does not imply that the rows look real.

Table 4.10: Fidelity of the synthetic sets (mean over classes and seeds)

Configuration	Wasserstein	Jensen–Shannon	Correlation distance	C2ST AUC
B3 (CVAE, no protection)	0.1029	0.0024	0.0633	0.9997
M1-eps1	0.2352	0.0159	0.1649	1.0000
M1-eps5	0.1944	0.0128	0.1390	1.0000
M1-eps10	0.1755	0.0103	0.1244	0.9999
MG-eps1	0.1941	0.0232	0.1691	0.9998
MG-eps5	0.0374	0.0003	0.0581	0.9997
MG-eps10	0.0489	0.0025	0.0740	0.9995
MGl-eps1	0.2691	0.0376	0.2191	0.9999
MGl-eps5	0.1132	0.0094	0.1244	0.9998
MGl-eps10	0.0722	0.0059	0.0995	0.9997

Answer to RQ2

At the same (ε, δ), FedDP-Marginal achieves a higher TSTR macro-F1 and a higher rare-class recall than the federated CVAE with DP at every tested budget, by $+0.092$ to $+0.191$ of macro-F1. It also has a higher score without DP ($+0.044$ RF, $+0.046$ MLP against B3). The remaining advantage of the non-private CVAE is the binary F1, 0.904 against 0.885 for FedDP-Marginal without DP.

4.3.3 RQ3: Overhead, Granularity and Honest Gateways

Cost of Secure Aggregation

Table 4.11 gives the overhead. Time is compute time on one machine and network latency is not measured. SecAgg$^+$ adds little to either route. Its effect on utility is negligible: for the CVAE route, 7 of 8 comparisons with the unprotected route show no effect and one is better (+0.0216, TAug with the MLP). For FedDP-Marginal the secure sum equals the exact sum, so the runs with and without SecAgg$^+$ release the same tables and the same synthetic data at every seed, and the TSTR difference is 0.000.

Table 4.11: Measured overhead of secure aggregation

Pair	Time	Volume per round	Ratios (time, bytes)
M2 / B3 (CVAE, no DP)	2.18 s vs 1.95 s	60.6 vs 40.0 bytes per parameter	1.12, 1.52
M3-eps5 / M1-eps5 (CVAE, DP)	3.83 s vs 3.76 s	64.7 vs 40.0 bytes per parameter	1.02, 1.62
MGs-nodp / MGp-nodp	21.6 s vs 20.4 s (run)	0.158 vs 0.114 MB	1.06, 1.39

DP-SGD itself is the expensive part of the CVAE route: its median time per round is 3.76 s, against 1.95 s for B3, a ratio of 1.93. Over a whole run with $K = 5$, FedDP-Marginal stays at or below 1.33 MB across all its configurations, while the CVAE routes take from 8.4 to 191 MB. Figure 4.5 shows the trade-off between total traffic and detection.

Protected Unit

Table 4.12 shows the cost of moving the guarantee from one packet to a larger unit. Protecting a TCP stream costs 0.03 to 0.05 of macro-F1, since 90% of the streams hold a single row. Protecting a whole capture is not usable on this data. The macro-F1 falls to 0.09 to 0.18, because only a few thousand rows can be kept (3,675 at $m = 25$).

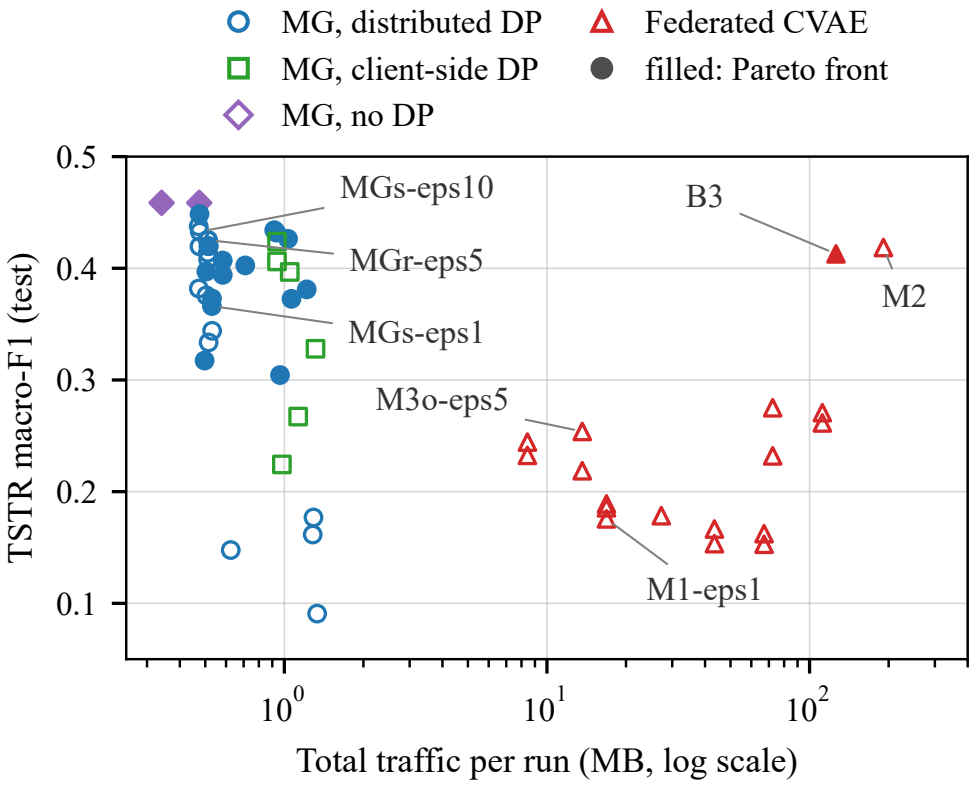


Figure 4.5: TSTR macro-F1 against the total traffic of a run. Filled markers belong to the Pareto front.

Table 4.12: TSTR macro-F1 by protected unit (rows kept in brackets when capped)

ε	**Packet**	**TCP stream**	**Capture**
1	0.366 ± 0.018	0.334 ± 0.025 ($m = 1$; 77,106)	0.091 ± 0.037 ($m = 25$; 3,675)
5	0.425 ± 0.004	0.376 ± 0.007 ($m = 1$; 77,106)	0.177 ± 0.020 ($m = 50$; 7,350)
10	0.433 ± 0.004	0.382 ± 0.004 ($m = 2$; 80,244)	0.148 ± 0.021 ($m = 100$; 14,000)

Honest Gateways

With distributed noise, the guarantee holds only if enough gateways add their share. Figure 4.6 draws the effective ε as a function of the number of honest gateways. It grows from 5 to 14.2 at the MGr-eps5 setting when one gateway alone remains honest. The trend worsens with more clients: at $\varepsilon = 5$ with one honest gateway, the value is 25.4 for $K = 10$ and 49.9 for $K = 20$.

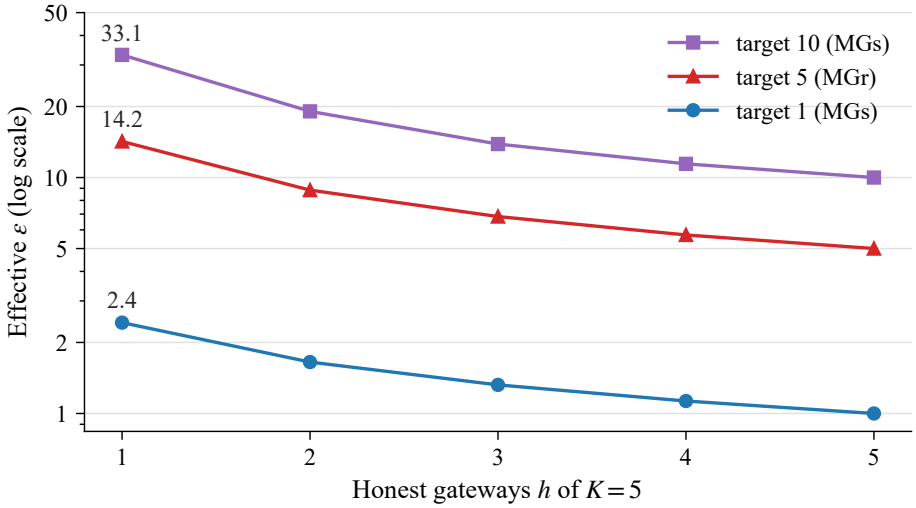


Figure 4.6: Effective ε as a function of the number of honest gateways h, for $K = 5$ and calibration at $t = K$.

Table 4.13 gives the effect of calibrating the noise for t honest gateways instead of K. Each gateway then adds $1/t$ of the noise. Changing t from 5 to 3 moves the macro-F1 by -0.005 to $+0.016$, and changing it to 2 moves it by -0.022 to $+0.000$, while the ε with every gateway honest falls by 24% to 44%. On this data, the extra noise of a smaller t is nearly free in utility. This holds for $K = 5$ only.

Empirical Privacy Checks

All duplicate rates are at most 0.0007, and all distance ratios lie between 0.85 and 0.93, inside the heuristic thresholds (at most 0.01 and at least 0.5). These two checks detect only near-copies.

Membership inference on the synthetic set stays between 0.488 and 0.494 for every generator, including B3 without DP. The attack did not pass its positive control on an over-fitted

Table 4.13: Effect of the honest-gateway threshold t ($K = 5$)

Target ε	t	TSTR macro-F1	ε (t honest)	ε (all honest)	ε (one honest)
1	5	0.366 ± 0.018	1.00	1.00	2.4
1	3	0.373 ± 0.029	1.00	0.76	1.8
1	2	0.344 ± 0.013	1.00	0.61	1.5
5	5	0.425 ± 0.004	5.00	5.00	14.2
5	3	0.420 ± 0.009	5.00	3.72	10.1
5	2	0.425 ± 0.011	5.00	2.94	7.6
10	5	0.433 ± 0.004	10.00	10.00	33.1
10	3	0.449 ± 0.005	10.00	7.14	22.3
10	2	0.420 ± 0.004	10.00	5.59	15.9

CVAE (AUC 0.510), so an AUC close to 0.5 here only says that no synthetic row is a near-copy of a training row. On a generator that copies its members, the same attack reaches 0.981 without noise and 0.526 with noise of 0.2.

A second attack with access to the released CVAE weights passes its positive control, with a calibrated AUC of 0.647 ± 0.011 on the CVAE over-fitted on 500 rows. Against the released federated models it gives 0.506 ± 0.003 for B3, 0.497 ± 0.002 for M1-eps10, 0.499 ± 0.002 for M1-eps5, and 0.502 ± 0.002 for M1-eps1. It detects nothing in any of them, including the unprotected B3.

The third attack, on the released tables of FedDP-Marginal, is the only one that separates the release without DP from the releases with DP. Table 4.14 and Figure 4.7 show the result. The positive control, on tables built from 500 records without noise, reaches 0.835. All three drops in AUC against the release without DP pass the bootstrap, and the absolute leakage remains small: 1.91% true positives at 1% false positives without DP.

Robustness and Sensitivity

Table 4.15 reports the branches of Table 3.5. Because the distributed variant releases only sums over the clients, its tables do not depend on how the rows are spread: at $\alpha = 0.1$ and $\alpha = 10$, the macro-F1 is the same, 0.398. Its ε with one honest gateway grows with K, while its macro-F1 stays between 0.400 and 0.415. The local variant, which pays the sum of K noises, loses utility as K grows.

Table 4.14: Membership inference on the released tables of FedDP-Marginal (calibrated attack, 1,000 bootstrap resamples)

Release	AUC [95% CI]	TPR at 1% FPR	Drop against no DP [95% CI]
No DP (MGs-nodp)	0.573 [0.568, 0.578]	1.91%	n/a
$\varepsilon = 10$ (MGs-eps10)	0.539 [0.534, 0.544]	1.39%	0.034 [0.029, 0.040]
$\varepsilon = 5$ (MGr-eps5)	0.525 [0.519, 0.530]	1.14%	0.048 [0.042, 0.055]
$\varepsilon = 1$ (MGs-eps1)	0.515 [0.510, 0.520]	1.14%	0.059 [0.052, 0.065]

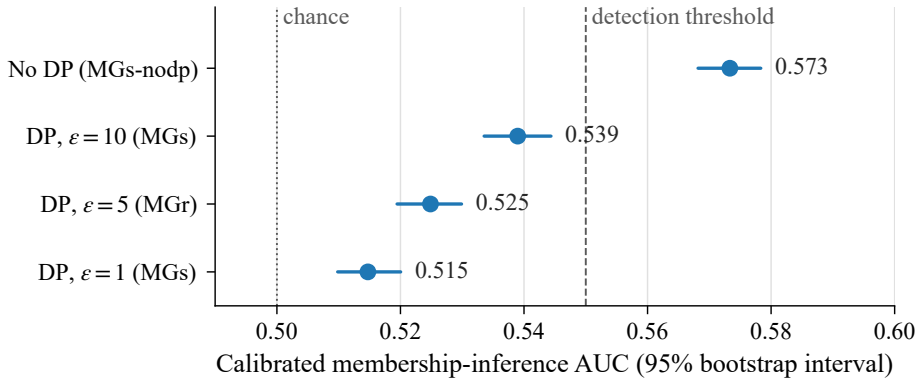


Figure 4.7: Calibrated membership-inference AUC on the released tables, with the 95% bootstrap interval, against the chance level and the heuristic detection threshold.

Table 4.16 repeats the study in five other worlds. Of the ten verdicts compared, six keep the same verdict in every world, and eight keep it across the two alternative choices of test groups (A4). The verdicts that change are mostly small differences close to the seed noise. The comparison between MG-eps5 and the CVAE without DP (B3) under the RF, for example, is "better" in the main study and in both A4 worlds, but "no effect" under the stream cap A5 and in both quota worlds.

Answer to RQ3

SecAgg$^+$ is cheap, at 1.02 to 1.12 times the time and 1.39 to 1.62 times the bytes, and it leaves utility intact. Distributed noise is the main source of the difference between MGs and MGl, but its guarantee depends on how many gateways stay honest. A smaller t gives protection against collusion at almost no cost in utility for $K = 5$. A TCP stream is an affordable unit of protection, whereas a whole capture is not usable on this data.

Table 4.15: Robustness of FedDP-Marginal at $\varepsilon = 5$, and the eleven-class mode

Setting	**TSTR macro-F1**	**ε (one honest)**	**Seeds**
MGs, main setting ($K = 5$, $\alpha = 0.5$)	0.408 ± 0.009	14.2	3
MGs, $\alpha = 0.1$	0.398	14.2	1
MGs, $\alpha = 10$	0.398	14.2	1
MGs, $K = 10$	0.400	25.4	1
MGs, $K = 20$	0.415	49.9	1
MGl, $K = 5$	0.397 ± 0.006	5.0	3
MGl, $K = 10$	0.362 ± 0.017	5.0	3
MGl, $K = 20$	0.364 ± 0.021	5.0	3
Eleven classes, B0, RF (real data only)	0.336 ± 0.000	n/a	3
Eleven classes, MGs-eps1	0.238 ± 0.015	2.4	3
Eleven classes, MGs-eps5	0.284 ± 0.003	14.2	3
Eleven classes, MGs-eps10	0.308 ± 0.004	33.1	3
Eleven classes, B3 (CVAE, no protection)	0.249 ± 0.018	n/a	3

Table 4.16: Sensitivity worlds: test macro-F1 (mean $\pm$ standard deviation over three seeds)

World	**B0, RF**	**B3, TSTR, MLP**	**MG-eps5, TSTR, RF**	**MG-eps5, TSTR, MLP**
Main study	0.448 ± 0.000	0.413 ± 0.005	0.425 ± 0.012	0.425 ± 0.004
A4: other test groups (seed 1)	0.457 ± 0.001	0.388 ± 0.021	0.419 ± 0.015	0.427 ± 0.009
A4: other test groups (seed 2)	0.462 ± 0.000	0.371 ± 0.028	0.416 ± 0.012	0.429 ± 0.010
A5: at most 20 packets per stream	0.442 ± 0.001	0.387 ± 0.019	0.394 ± 0.006	0.376 ± 0.006
A2: rare quotas $\times 6$	0.511 ± 0.000	0.428 ± 0.015	0.432 ± 0.007	0.446 ± 0.013
A2: rare quotas $\times 0.3$	0.405 ± 0.001	0.386 ± 0.038	0.388 ± 0.013	0.372 ± 0.004

4.3.4 RQ4: Configuration per Deployment Requirement

The rule of Section 3.1.5 takes each requirement of Table 3.6 and returns the configuration on its Pareto front with the best value on the requested priority. Table 4.17 gives the outcome. The front of the scorecard holds 18 configurations, 17 of which are FedDP-Marginal variants. The remaining one is B3, retained for its binary F1 and rare-class recall.

Table 4.17: Recommended configuration per deployment requirement. The value of ε is the one that holds under the trust assumed by the requirement.

Scenario	Recommended	Macro-F1	Rare recall	ε	MB/run
`lab`	MGp-nodp	0.459	0.367	∞	0.3
`hidden-tables`	MGs-nodp	0.459	0.367	∞	0.5
`cloud-aggregator`	MGd-eps5	0.426	0.325	5.00	1.0
`consortium`	MGr-t2-eps5	0.425	0.310	7.63	0.5
`strict-privacy`	MGs-t3-eps1	0.373	0.233	1.00	0.5
`low-bandwidth`	MGs-t3-eps10	0.449	0.346	10.00	0.5
`rare-attacks`	MGd-eps5	0.426	0.325	5.00	1.0
`honest-majority`	MGs-t3-eps10	0.449	0.346	10.00	0.5
`gateway-dropout`	MGr-t2-eps5	0.425	0.310	3.94	0.5
`session-privacy`	MGs-strm1-eps5	0.397	0.310	5.00 (stream)	0.5
`capture-privacy`	MGr-cap50-eps5	0.177	0.105	5.00 (capture)	1.3

Three remarks qualify the table. MGd is a comparator whose Gaussian noise and secure sum are simulated in one process, so a deployment that selects it needs a real implementation of the mechanism. The difference between MGd-eps5 and MGs-t3-eps5 (0.420) lies within the seed noise. The `capture-privacy` row is the best the data permit, and its macro-F1 of 0.177 does not make it a usable detector.

Answer to RQ4

The choice depends more on trust than on utility. No configuration is best everywhere. Where the operator trusts every party, the versions without DP suffice and give the best detection. Where the aggregator must not see any gateway's tables but the released data stay inside the company, secure aggregation alone meets the requirement. Where a formal guarantee is required, the distributed variants at $\varepsilon = 5$ to 10 give macro-F1 values of 0.408 to 0.449. A demand for $\varepsilon \leq 1$ costs about 0.05 to 0.08 in macro-F1 at the same protection level.

4.4 Interpretation of Results

Why FedDP-Marginal Withstands DP Better Than the CVAE

The privacy budget of the CVAE is consumed through many noisy optimisation steps: each client runs local epochs in each round, and every step composes. FedDP-Marginal spends the budget on three count releases, which are sums with bounded sensitivity. Noise therefore lands directly on what the generator needs. In addition, the CVAE has to learn a joint distribution over 59 columns from noisy gradients, whereas the tree needs only low-order tables. These differences are consistent with the results and with the literature on marginal synthesis, but the study did not isolate them experimentally.

The second structural factor is the noise split. Under secure addition, each gateway can add only $1/K$ of the noise, and the sum still carries the calibrated total. That is valid because each release is a single additive sum. It is not valid for multi-epoch DP-SGD, so SecAgg^{+} hides the updates of M3 in transit but does not change the noise it needs. This explains why distributed noise raises FedDP-Marginal by up to 0.10 of macro-F1 at the same ε, while M3 and M1 are the same within the noise.

Why the Curve Flattens Between $\varepsilon = 5$ and $\varepsilon = 10$

FedDP-Marginal without DP reaches 0.459, only 0.026 above MG-eps10. Beyond a moderate budget, the limiting factor is what a Chow–Liu tree over 37 attributes, sampled one packet at a time, can represent. The classifier is limited too: the real-data references show that per-packet separation of the rare classes is hard.

Why Binary F1 and Macro-F1 Tell Different Stories

Across the main configurations of Table 4.4, the binary F1 ranges from 0.75 to 0.90, and macro-F1 ranges from 0.18 to 0.46. Saying that a packet belongs to an attack is much easier than saying which attack. Most of the difficulty lies in separating the attack families, and part of the literature's high scores comes from reporting only the binary task (Section 4.5).

Why Synthetic Rows Add Nothing to Pooled Real Data

TAug adds synthetic rows to a classifier that already sees all the real rows of every class. A generator trained on those rows, and in particular one that models each packet independently, gives no new information about the boundary between classes. Local augmentation differs, since the client there receives classes it has rarely seen, and this accounts for its gain of about 0.04.

Why the Attacks Behave Differently

The attack on the synthetic set measures near-copies and sees none. The attack on the released CVAE sees nothing either, including in the unprotected B3, which suggests that the training regime of the federated CVAE leaves little membership signal for this attack family. The attack on the tables does see the release without DP, because a count table responds directly to the presence of a record, especially in the sparse cells of rare classes. DP then reduces the signal in a measurable way, and the ordering of the AUC follows ε. Secure aggregation alone does not affect this, since it hides each gateway's tables from the server but not what the released sum reveals.

What the Honest-Gateway Results Mean

With one honest gateway and calibration at $t = K$, the effective ε is 2.4 to 3.3 times the value claimed for full honesty. The guarantee at $\varepsilon = 10$ becomes 33.1, a value too large to mean much. The threshold t exposes this trade-off and makes it a design decision. It does not remove it.

Limits of These Interpretations

These explanations are consistent with the measured results, but only the comparisons in Section 4.3 were tested. The causal statements in this section were not tested separately, and they apply to one dataset, packet-level features and three seeds.

4.5 Comparison with Literature

4.5.1 Published Results on the Same Dataset

Published scores on this dataset are far above the macro-F1 of this study, but they answer other questions, so a score can be read only together with its task, input and split. A study therefore enters Table 4.18 only if it trains or tests on this dataset and its full text could be read at the source. Three studies meet both conditions. The search is not systematic: 103 works cite the dataset paper (OpenAlex, snapshot of 2026-11-09), 36 of them mention MQTT and detection, and only the most likely ones were opened.

Table 4.18: Published results on the DoS/DDoS-MQTT-IoT dataset, with the task, input and split of each study

Study	**Task**	**Input**	**Split**	**Reported result**
Alatram et al. [2]	Attack against normal	Packets; the 30-feature list contains source IP, stream index and time columns, and the paper does not say which columns the classifiers received	Not described; 3,654,560 records used	Accuracy of RF 92.69% to 100%; no F1
Zhang et al. [75]	Binary, each attack against benign (DDoS variants of BF, Delay, Sub, WILL)	Packet payload: keywords from 30-packet windows, hashed to 8 integers	Contiguous part of the captures, balanced by selection	F1 (RF) 0.993 to 1.000
Parra and Bayas [76]	Binary, normal against two flooding attacks	TCP flows in 1 s windows; 4 statistics; no payload	Stratified 70/30 and 5-fold; no separation by capture stated	F1 0.846, AUC 0.876 (hold-out)
This study	6 classes, per packet (and the binary tasks below)	Packets; 59 encoded features without source IP, absolute time or stream index	By capture or by block of a capture; 30,000 balanced test packets	Macro-F1: real 0.448, SMOTE 0.505, MG-eps10 0.433; binary F1: 0.786, 0.821, 0.843

On the closest binary tasks, the saved predictions are re-scored with every attack class

counted as "attack", with no retraining (Table 4.19). The tasks are close but not identical: the papers train a binary model, while this study reuses its six-class models.

Table 4.19: Binary F1 on the closest tasks (mean of three seeds)

Task	Published (study)	F1	Real data (B1a, RF)	Synthetic, $\varepsilon = 10$
Normal / BCF + DELAYED	Parra and Bayas [76]	0.846	0.837	0.832
Normal / BCF	Zhang et al. [75]	1.000	0.888	0.857
Normal / DELAYED	Zhang et al. [75]	1.000	0.738	0.723
Normal / INVALID	Zhang et al. [75]	0.993	0.786	0.728
Normal / WILL	Zhang et al. [75]	0.999	0.853	0.836

4.5.2 How Much the Protocol Alone Moves a Score

Table 4.20 measures this directly. The same RF is trained on the same 130,500 sampled packets, and only the split and the identifier columns change.

Table 4.20: Effect of the split and the identifier columns on the real-data RF (three seeds)

Split	Identifier columns added	Macro-F1	Binary F1
By capture or block (this study)	None (this study)	0.448 ± 0.000	0.786 ± 0.000
By capture or block	All of them	0.842 ± 0.003	0.974 ± 0.001
Random, row by row	None	0.464 ± 0.001	0.799 ± 0.001
Random, row by row	Source IP	0.496 ± 0.001	0.848 ± 0.001
Random, row by row	Absolute time, stream index, time since capture start	0.941 ± 0.003	0.989 ± 0.001
Random, row by row	All of them	0.939 ± 0.002	0.987 ± 0.001

The split alone explains little: going from capture to row by row moves the macro-F1 by $+0.017$. Source IP alone adds $+0.032$. The time and stream columns explain the bulk of the gap, adding $+0.476$, because each capture was recorded at its own time and absolute time can identify the capture, and hence the class. A deployed detector cannot rely on such a column, which is why the study removes them. A split by capture does not stop this leak (0.842 with all identifiers), likely because the test rows of 8 of the 11 sub-classes are a block of the same capture as their training rows. These measurements describe this classifier on this data. They

do not establish how the classifiers in the cited studies were trained, because those papers do not say, and the study does not claim that their scores are inflated. Differences in task (binary against six classes), input (payload against header features) and identifier columns are enough to make the scores incomparable.

4.5.3 Against Federated and Private Data Generation

None of the studies read combines federated learning, differential privacy and synthetic data on this dataset, so the protected results have no direct published reference. Marginal-based DP synthesis in federated settings already exists. FLAIM [36] adapts AIM [12] to the federated setting, and CaPS [37] relies on secure multi-party computation. As Chapter 2 notes, these studies evaluate workload error, negative log-likelihood, runtime and communication, on tabular benchmarks that are not intrusion-detection traffic. FLAIM and AIM themselves were not run here, since Private-PGM was not installed, so the study cannot rank FedDP-Marginal against them. To the best of our knowledge, based on a non-systematic search, the contribution here is an evaluation that puts these pieces together: exact distributed DP with Skellam noise [22] through a real SecAgg$^+$ implementation, measured byte counts, the ε per number of honest gateways, a comparison with a CVAE under the same federation, and a configuration guide per requirement. The study claims no algorithmic novelty.

4.6 Implications of the Results

For Operators of MQTT Deployments

A federation of gateways can train a detector without pooling raw traffic. On this data, three short releases of less than 1 MB in total give a TSTR macro-F1 between 0.366 and 0.433, with a formal guarantee at record level. The cost in bandwidth is low enough for constrained backhaul such as NB-IoT or metered 4G, although the times here are compute times and network behaviour was not measured.

For the Design of Privacy-Preserving IDS

- A marginal generator is the better default than a deep generator when a formal guarantee is required at small or moderate ε on tabular traffic data, in the setting tested.

- The ε must be reported with its unit and its trust assumption. A value of 5 per packet with all gateways honest is not the same guarantee as 14.2 with one honest gateway, or as 5 per TCP stream.
- A protected detector can be obtained by another route. If only the detector matters and traffic is not a constraint, the direct federated classifier is a competitor. Where a shareable dataset is needed, or where communication is scarce, the generator route is preferable.
- Secure aggregation hides individual gateways from the aggregator at modest cost and does not protect what the final tables reveal. Both mechanisms are needed for different reasons.

For Evaluation Practice

The score of an IDS on this dataset moves from 0.448 to 0.941 when only the protocol changes. Reports on this dataset should therefore state the task, the input columns, and the split. Generators should be judged with a positive control for every privacy attack, since an attack at chance says nothing when it cannot detect a deliberately exposed target.

What the Results Do Not Allow Us to Claim

The results do not show that synthetic data improve an IDS when real data can be pooled. They do not show that synthetic rows look real, nor that the framework would behave the same on other traffic. The empirical attacks measure one family of attacks at record level and do not replace the formal guarantee.

Chapter 5

Discussion

This chapter returns to the problem that motivated the study, gathers the main findings, sets them in a wider context, and states how far they can be trusted.

5.1 Restate the Research Problem or Objectives

IoT gateways that broker MQTT traffic see attacks that a single site observes rarely, or not at all. An intrusion detection system would need data from many sites, yet raw packet traces reveal devices, topics and behaviour, and their owners cannot always share them. Secure aggregation hides what each site sends. Differential privacy bounds what the output reveals. Each has a cost in utility, bandwidth or trust, and the literature offers few controlled measurements of what these costs are for a detector on MQTT traffic.

The study therefore set four objectives (Section 1.3): build a controlled evaluation on real MQTT traffic with leakage checks (O1), extend marginal-based private synthesis to the federated setting with distributed noise (O2), compare it with a federated CVAE under matched privacy budgets (O3), and turn the measurements into a configuration guide per deployment requirement (O4). Four research questions follow from them: how much IDS utility the synthetic data retain (RQ1), how the marginal and deep routes compare at the same budget (RQ2), what each protection costs and at what granularity it protects (RQ3), and which configuration suits which requirement (RQ4).

5.2 Summarize Key Findings

- **Utility without pooling.** An IDS trained only on the synthetic data of FedDP-Marginal reaches a test macro-F1 of 0.366, 0.425 and 0.433 at $\varepsilon = 1, 5, 10$, against 0.448 for a

Random Forest on pooled real data and 0.505 with SMOTE on pooled real data. Without DP, the generator reaches 0.459.

- **No gain when real data can be pooled.** Adding synthetic rows to pooled real data passes the decision rule for 2 of 13 generators with the RF, by at most $+0.001$, against $+0.057$ for SMOTE. The CVAE is worse than class weights and SMOTE in 6 of 6 comparisons.
- **A real gain at the edge.** A gateway that adds synthetic rows for the classes it lacks raises its macro-F1 from 0.384 to about 0.426, and 11 to 13 of 15 client models gain, depending on ε.
- **Marginal route ahead of the deep route.** At the same (ε, δ), FedDP-Marginal exceeds the best CVAE route by $+0.092$ to $+0.191$ of macro-F1 and $+0.049$ to $+0.169$ of rare-class recall, with all verdicts passing under both bootstraps.
- **A fraction of the traffic.** FedDP-Marginal needs three rounds and at most 1.33 MB in a run, against 8.4 to 191 MB for the CVAE routes.
- **A measurable privacy effect.** The membership attack on the released tables detects the release without DP (AUC 0.573) and detects those with DP less (0.515 to 0.539), and it passes its positive control. Attacks on the CVAE find nothing, the unprotected model included.
- **A guarantee that depends on trust and unit.** At $\varepsilon = 5$, one honest gateway alone gives an effective ε of 14.2. Protecting a TCP stream costs 0.03 to 0.05 of macro-F1. A whole capture is not usable on this data.
- **No universal configuration.** Eleven requirements map to eight distinct configurations, 17 of the 18 configurations on the Pareto front being variants of FedDP-Marginal.

5.3 Interpretation in a Broader Context

Choosing a Generator Is Choosing a Privacy Cost Structure

The results suggest that the architecture of a generator determines how the privacy budget is spent. A model trained over many noisy steps pays for every step. A generator whose entire

interaction with the data is a small number of bounded count releases pays only for those. Under strict budgets the second design has a clear advantage on this tabular data, and it fits secure aggregation by construction, because what it sends is a sum.

Different Protections Answer Different Threats

A recurring confusion in this area is to treat secure aggregation and differential privacy as substitutes. Here they answer different questions. SecAgg^{+} prevents the server from seeing an individual gateway's tables and changes nothing in what the final tables reveal, as the unchanged attack AUC of 0.573 shows. DP bounds what the released output reveals. A deployment must therefore decide which adversary it fears, which is why the requirement-driven guide lets the choice depend on trust.

Evaluation Protocol as a Result in Itself

On one dataset and one classifier, the macro-F1 moved from 0.448 to 0.941 by changing only the split and the identifier columns. The lesson is broader than this study: reported scores on network traffic data are difficult to compare without their protocol, and a high figure may partly reflect identifying columns. Earlier work had warned about such pitfalls in security datasets [59, 60], and these measurements quantify them on this dataset.

The Value of Generated Data Depends on Who Lacks What

The same synthetic data are of no use to a classifier that already sees every real row, and of real use to a gateway that has barely seen some classes. Their value is a function of what the recipient lacks, which suggests that the question "do synthetic data help?" needs the qualifier "for whom, and instead of what?"

5.4 Working Hypotheses and Research Questions

Table 5.1 collects the answers. H1 and H2 are the working hypotheses of Section 2.1.3.

Table 5.1: Status of the working hypotheses and research questions

Item	Finding	Status
H1	At $\varepsilon = 1$, the marginal generator has a higher F1 on INVALID ($+0.139$) and WILL ($+0.222$)	Supported (point estimates, above the seed spread)
H2	The CVAE was expected to lead on BCF and SYN; instead, the marginal generator leads by $+0.138$ and $+0.312$	Not supported
RQ1	Macro-F1 of 0.366 to 0.433 under TSTR with DP; no meaningful gain from TAug; about $+0.04$ from local augmentation	Answered
RQ2	FedDP-Marginal exceeds the CVAE under DP on macro-F1 and rare-class recall at every budget	Answered, in favour of FedDP-Marginal
RQ3	SecAgg$^+$ is cheap; a TCP stream is an affordable unit; a capture is not; the guarantee degrades with fewer honest gateways	Answered
RQ4	Eleven requirements are mapped through a transparent rule	Answered, as a rule on measured values

5.5 Why the CVAE Was Studied and Why It Remains the Baseline

A reader may ask why this thesis keeps a generator whose results are negative. The reason lies in how the study was built.

The premise. The study began from a specific premise: that a label-conditional CVAE trained by federated learning under non-IID clients, and protected by DP-SGD, secure aggregation or both, could generate data that balance the classes of an IDS. The CVAE was the premise of the study and was not selected by a comparison between generators. RQ1 and RQ2 are the test of that premise.

What the numbers say. When real data can be pooled, the premise fails on utility. TAug with the CVAE is worse than class weights and SMOTE in 6 of 6 comparisons, by -0.047 to -0.118 of macro-F1. Generators pass the decision rule against real data only in 2 of 13 cases with the RF (at most $+0.001$) and in 9 of 13 with the MLP (at most $+0.038$, against $+0.134$ for SMOTE). DP lowers the synthetic-only score of the CVAE by 12% to 58%, and re-tuning

at full scale did not close the gap.

The case that remains. All of TRTR, TAug and TAugR need the pooled real data that the threat model excludes. Where raw data must stay at the gateways, the CVAE route without DP reaches a TSTR macro-F1 of 0.413 and has the highest binary F1 of the study, 0.904. It therefore remains a meaningful reference for the case without a formal guarantee, and its loss under DP is what motivated the move to FedDP-Marginal. It also keeps the comparison honest: without a deep baseline under the same federation, DP and SecAgg^{+} conditions, the advantage of the marginal route would have no measured point of comparison.

What was not tested. Other generators (WGAN-GP, TVAE or CTGAN-type models, diffusion models, and FLAIM or AIM) were not run. Other ways of using synthetic data, beyond topping classes up and local augmentation, were not studied either. The results therefore compare one CVAE with one marginal generator on one dataset, and they do not show that the CVAE family is inferior in general.

5.6 Limitations

Data and Features

All results come from one dataset, so nothing here shows that they carry over to other traffic or networks. Records are single packets generated independently, so the temporal structure of a session is not reproduced. Eight of the eleven sub-classes come from a single capture file, so their validation and test rows are consecutive blocks of that capture. The DoS and DDoS variants are merged in the main matrix. The test split comes from 11 capture groups only, and the two alternative choices in A4 cover this only in part.

Privacy Scope

The default ε protects one packet. Packets of one stream or capture are correlated, so what is protected about a whole session is weaker. Labels, class counts and client sizes are outside the guarantee. Normalisation statistics are computed centrally on the training split, as if each client had shared its data once, and a real federation would have to compute them securely or fix them beforehand. Hyper-parameter selection used the real validation split and is not covered by ε: 12 candidates per budget for FedDP-Marginal, and at $\varepsilon = 5$ and $\varepsilon = 10$ the two best

candidates differ by less than the weight of one validation packet. Distributed DP assumes that enough gateways add their noise share, and malicious gateways are outside the threat model.

Generators

FedDP-Marginal models each class with a Chow–Liu tree, so every attribute depends on one parent, and the numeric values inside a fine bin are drawn uniformly. Second-order structure was tried and did not help on this data, but it may on others. Synthetic rows remain easy to tell from real ones, with a C2ST AUC of at least 0.9995 for every generator. The recall of the rare classes stays below that of the non-private federated CVAE for most protected configurations. The MGd variant simulates its secure sum in process, so it has no measured deployment cost.

Evaluation and Statistics

Three seeds give only a rough estimate of the spread, and federated training is sensitive to very small perturbations. The bootstrap intervals reflect the sampling of test rows or whole streams, and they do not cover the choice of capture groups, the hyper-parameters or the data sampling. The thresholds of the evaluation (membership AUC 0.55, $\varepsilon \leq 5$, overhead 3 times) are heuristics. The empirical attacks test one family at record level and do not test group inference. The class-level analysis of H1 and H2 is based on point estimates and the seed spread.

Execution Platform

The federation is simulated on one CPU machine with Ray actors, so times are compute times, and network latency, packet loss and constrained-device behaviour were not measured. Byte counts are measured, except for the local variant, where they are estimated from the number of cells.

Comparison with Published Work

The literature comparison rests on three studies whose full text could be read, from a non-systematic search. The closest binary tasks are close but not identical to the published ones.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis studied how several MQTT gateways can jointly produce data for training an intrusion detection system without releasing their raw packets, and what each protection costs. It built PP-FedData, a framework evaluated on the DoS/DDoS-MQTT-IoT dataset under a split by capture, with five leakage checks, a locked test set and a paired-bootstrap decision rule. Two generator routes were compared under the same federation: FedDP-Marginal, which releases noisy count tables in three rounds and samples from a Chow–Liu tree, and a federated conditional variational autoencoder trained with DP-SGD.

Three conclusions stand out.

First, for protected generation on this data, the marginal route is the better choice. At the same privacy budget it keeps a higher macro-F1 and a higher rare-class recall than the CVAE, by $+0.092$ to $+0.191$ and $+0.049$ to $+0.169$, and it uses a run of at most 1.33 MB in three rounds in place of 8.4 to 191 MB over 15 to 50 rounds. Its releases are sums, so the noise can be split across gateways through secure aggregation with exact accounting.

Second, generated data have a specific place. They do not improve an IDS when real data can be pooled, and class weights or SMOTE are preferable there. They are valuable where pooling is impossible, with a synthetic-only macro-F1 of 0.366 to 0.433 at ε from 1 to 10 against 0.448 for real data only, and they are valuable to a gateway that lacks some classes, which gains about 0.04 of macro-F1.

Third, a privacy claim is meaningful only together with its unit and its trust assumption. Protecting a TCP stream is affordable, at 0.03 to 0.05 of macro-F1, and protecting a whole capture is not usable on this data. With one honest gateway out of five, an ε of 5 becomes 14.2. A configuration should therefore be selected by requirement, and the study provides a rule that

does so from measured values for eleven deployment scenarios.

These conclusions apply to one dataset, packet-level records, five gateways simulated on one machine, and three seeds. Marginal-based federated synthesis is not new, and this work claims no algorithmic novelty. Its contribution is the measured evaluation, the exact distributed accounting through a real secure-aggregation implementation, and the requirement-driven guide.

6.2 Future Work

- **Stronger generators under the same conditions.** Run FLAIM, AIM and other marginal mechanisms, together with DP-CTGAN or diffusion-based models, under the same federation, DP accounting and SecAgg^{+}, so that FedDP-Marginal can be ranked against them.
- **Richer structure.** Study higher-order or class-specific graphical structures with private selection, to raise utility beyond the current plateau at moderate budgets.
- **Session-level modelling.** Generate sequences of packets or flow-level records, so that temporal attack patterns are represented and capture-level protection becomes feasible.
- **Private preprocessing and selection.** Compute normalisation and category statistics securely, or fix them in advance, and add a private selection step so that the tuning falls inside the privacy accounting.
- **Real deployment measurements.** Run the protocol on real gateways over constrained links such as NB-IoT or LoRa, to measure latency, packet loss, dropouts and energy use.
- **Other datasets and larger federations.** Repeat the study on other MQTT and IoT traffic, with a wider range of K and with client dropout, so that the effect of t can be studied beyond $K = 5$.
- **Malicious participants.** Extend the threat model to poisoning and Byzantine behaviour, including robust aggregation compatible with distributed noise.
- **Group-level attacks.** Develop membership and attribute attacks at the level of a TCP stream or capture, to test what the group bounds protect in practice.

- **Hybrid detection.** Combine a privately trained detector with local augmentation by generated data, and measure whether the two routes complement each other on the rare classes.

Appendix A

Complete Scorecard

Table A.1 lists every configuration of the scorecard. It is generated from `results/scorecard.json` by `scripts/make_thesis_tables.py`. A configuration lies on the Pareto front if no valid configuration dominates it on the axes of Section 3.1.5. The columns are the same as in Table 4.4, with three additions: the gain of TAug over B0 with the same classifier, the value of ε with a single honest gateway, and the presence of SecAgg^{+}.

Table A.1: Scorecard of all configurations (TSTR on the real test split, classifier chosen on validation). Configurations marked with a dagger use statistics outside ε and are excluded from the Pareto front.

Configuration	Clf.	Front	Macro-F1	Binary F1	Rare recall	TAug gain	ε	ε (h=1)	SecAgg^{+}	MB/round	Rounds	MB/run
B3	MLP	yes	0.413 ± 0.005	0.904	0.361	+0.001	∞	∞	no	4.20	30	126.06
M1-eps1	MLP	no	0.176 ± 0.008	0.750	0.068	0.000	1.00	1.00	no	0.56	30	16.85
M1-eps5	RF	no	0.186 ± 0.009	0.761	0.055	+0.001	5.00	5.00	no	0.56	30	16.85
M1-eps10	MLP	no	0.189 ± 0.021	0.775	0.061	−0.001	10.00	10.00	no	0.56	30	16.85
M2	MLP	no	0.419 ± 0.008	0.896	0.361	+0.001	∞	∞	yes	6.37	30	191.07
M3-eps5	MLP	no	0.178 ± 0.010	0.771	0.059	+0.001	5.00	5.00	yes	0.91	30	27.26
M1-eps1 (residual noise) †	MLP	no	0.233 ± 0.036	0.782	0.105	+0.001	1.00	1.00	no	0.56	30	16.85
M1-eps5 (residual noise) †	MLP	no	0.269 ± 0.008	0.860	0.163	0.000	5.00	5.00	no	0.56	30	16.85
M1-eps10 (residual noise) †	MLP	no	0.260 ± 0.015	0.860	0.131	+0.001	10.00	10.00	no	0.56	30	16.85
M3-eps5 (residual noise) †	MLP	no	0.263 ± 0.016	0.844	0.141	+0.001	5.00	5.00	yes	0.91	30	27.26
M1o-eps5	MLP	no	0.244 ± 0.012	0.810	0.121	+0.001	5.00	5.00	no	0.56	15	8.43
M1o-eps5 (plain)	MLP	no	0.233 ± 0.038	0.817	0.127	0.000	5.00	5.00	no	0.56	15	8.43
M1o-eps10	MLP	no	0.275 ± 0.015	0.799	0.146	0.000	10.00	10.00	no	1.45	50	72.37
M1o-eps10 (plain)	MLP	no	0.232 ± 0.015	0.820	0.139	0.000	10.00	10.00	no	1.45	50	72.37
M1o-eps1	RF	no	0.167 ± 0.038	0.676	0.042	0.000	1.00	1.00	no	1.45	30	43.42
M1o-eps1 (plain)	MLP	no	0.153 ± 0.019	0.724	0.020	0.000	1.00	1.00	no	1.45	30	43.42
M3o-eps5	MLP	no	0.254 ± 0.013	0.814	0.132	+0.001	5.00	5.00	yes	0.91	15	13.63
M3o-eps5 (plain)	MLP	no	0.219 ± 0.024	0.816	0.121	0.000	5.00	5.00	yes	0.91	15	13.63
M3o-eps10	MLP	no	0.271 ± 0.017	0.806	0.122	+0.001	10.00	10.00	yes	2.24	50	111.86
M3o-eps10 (plain)	MLP	no	0.262 ± 0.045	0.817	0.154	0.000	10.00	10.00	yes	2.24	50	111.86
M3o-eps1	RF	no	0.162 ± 0.033	0.677	0.041	0.000	1.00	1.00	yes	2.24	30	67.12
M3o-eps1 (plain)	RF	no	0.153 ± 0.007	0.691	0.020	0.000	1.00	1.00	yes	2.24	30	67.12
MGb-eps5	RF	yes	0.403 ± 0.003	0.830	0.277	−0.001	5.00	14.22	yes	0.24	3	0.71
MGd-eps0.5	RF	yes	0.304 ± 0.005	0.791	0.160	0.000	0.50	1.13	yes	0.32	3	0.96
MGd-eps1	RF	yes	0.373 ± 0.023	0.819	0.230	−0.001	1.00	2.29	yes	0.35	3	1.06
MGd-eps10	MLP	yes	0.432 ± 0.001	0.859	0.305	0.000	10.00	26.65	yes	0.31	3	0.93

Configuration	Clf.	Front	Macro-F1	Binary F1	Rare recall	TAug gain	ε	ε (h=1)	SecAgg^{+}	MB/round	Rounds	MB/run
MGd-eps2	RF	yes	0.381 ± 0.011	0.826	0.247	0.000	2.00	4.69	yes	0.41	3	1.22
MGd-eps20	MLP	yes	0.434 ± 0.004	0.847	0.317	+0.001	20.00	58.35	yes	0.31	3	0.92
MGd-eps5	RF	yes	0.426 ± 0.006	0.849	0.325	−0.001	5.00	12.42	yes	0.34	3	1.03
MGl-eps0.5	RF	no	0.224 ± 0.025	0.729	0.077	0.000	0.50	0.50	no	0.33	3	0.98
MGl-eps1	RF	no	0.267 ± 0.021	0.774	0.117	0.000	1.00	1.00	no	0.38	3	1.13
MGl-eps10	MLP	no	0.406 ± 0.015	0.828	0.277	+0.001	10.00	10.00	no	0.31	3	0.94
MGl-eps2	MLP	no	0.328 ± 0.011	0.765	0.183	0.000	2.00	2.00	no	0.44	3	1.31
MGl-eps20	MLP	no	0.424 ± 0.015	0.843	0.291	+0.001	20.00	20.00	no	0.31	3	0.94
MGl-eps5	RF	no	0.397 ± 0.006	0.825	0.279	−0.001	5.00	5.00	no	0.35	3	1.05
MGp-nodp	MLP	yes	0.459 ± 0.007	0.885	0.367	+0.002	∞	∞	no	0.11	3	0.34
MGr-cap50-eps5	RF	no	0.177 ± 0.020	0.721	0.105	+0.001	5.00	13.04	yes	0.43	3	1.29
MGr-eps2	RF	yes	0.394 ± 0.017	0.825	0.285	−0.001	2.00	5.02	yes	0.19	3	0.58
MGr-eps5	MLP	no	0.425 ± 0.004	0.824	0.301	−0.001	5.00	14.22	yes	0.17	3	0.51
MGr-strm1-eps5	MLP	no	0.376 ± 0.007	0.787	0.254	0.000	5.00	14.22	yes	0.17	3	0.50
MGr-t2-eps5	MLP	no	0.425 ± 0.011	0.832	0.310	−0.001	5.00	7.63	yes	0.17	3	0.51
MGr-t3-eps5	MLP	no	0.420 ± 0.009	0.823	0.306	−0.002	5.00	10.10	yes	0.17	3	0.51
MGs-cap100-eps10	RF	no	0.148 ± 0.021	0.694	0.012	0.000	10.00	27.45	yes	0.21	3	0.63
MGs-cap25-eps1	RF	no	0.091 ± 0.037	0.446	0.042	0.000	1.00	2.42	yes	0.44	3	1.33
MGs-cap25-eps5	RF	no	0.162 ± 0.022	0.726	0.035	0.000	5.00	13.04	yes	0.43	3	1.28
MGs-eps0.5	RF	yes	0.317 ± 0.004	0.790	0.178	0.000	0.50	1.20	yes	0.17	3	0.50
MGs-eps1	RF	yes	0.366 ± 0.018	0.813	0.231	−0.001	1.00	2.43	yes	0.18	3	0.53
MGs-eps10	MLP	no	0.433 ± 0.004	0.843	0.301	+0.001	10.00	33.10	yes	0.16	3	0.48
MGs-eps2	RF	yes	0.407 ± 0.013	0.824	0.283	−0.001	2.00	5.02	yes	0.19	3	0.58
MGs-eps20	MLP	no	0.438 ± 0.001	0.862	0.319	+0.001	20.00	99.61	yes	0.16	3	0.47
MGs-eps5	MLP	no	0.408 ± 0.009	0.824	0.262	−0.001	5.00	14.22	yes	0.17	3	0.51
MGs-nodp	MLP	yes	0.459 ± 0.007	0.885	0.367	+0.002	∞	∞	yes	0.16	3	0.47
MGs-strm1-eps1	RF	no	0.334 ± 0.025	0.791	0.220	−0.001	1.00	2.43	yes	0.17	3	0.51
MGs-strm1-eps5	RF	yes	0.397 ± 0.010	0.829	0.310	+0.001	5.00	14.22	yes	0.17	3	0.50
MGs-strm2-eps10	RF	no	0.382 ± 0.004	0.799	0.291	+0.001	10.00	28.46	yes	0.16	3	0.47

Configuration	Clf.	Front	Macro-F1	Binary F1	Rare recall	TAug gain	ε	ε (h=1)	SecAgg^{+}	MB/round	Rounds	MB/run
MGs-t2-eps1	RF	no	0.344 ± 0.013	0.799	0.204	0.000	1.00	1.46	yes	0.18	3	0.53
MGs-t2-eps10	MLP	no	0.420 ± 0.004	0.846	0.282	0.000	10.00	15.89	yes	0.16	3	0.48
MGs-t2-eps5	MLP	no	0.414 ± 0.004	0.830	0.269	−0.001	5.00	7.63	yes	0.17	3	0.51
MGs-t3-eps1	RF	yes	0.373 ± 0.029	0.814	0.233	0.000	1.00	1.83	yes	0.18	3	0.53
MGs-t3-eps10	RF	yes	0.449 ± 0.005	0.892	0.346	+0.001	10.00	22.31	yes	0.16	3	0.48
MGs-t3-eps5	RF	yes	0.420 ± 0.011	0.854	0.308	−0.001	5.00	10.10	yes	0.17	3	0.51

Appendix B

Effective Privacy Budget by Honest Gateways

Table B.1 gives the effective ε of the distributed configurations when only h of the $K = 5$ gateways add their noise share. Configurations with a suffix `-t2` or `-t3` are calibrated for 2 or 3 honest gateways, and the suffixes `-strm` and `-cap` refer to a TCP stream or a capture as the protected unit.

Table B.1: Effective ε of the distributed configurations as a function of the number of honest gateways h ($K = 5$)

Configuration	$h = 1$	$h = 2$	$h = 3$	$h = 4$	$h = 5$
MGb-eps5	14.22	8.84	6.83	5.71	5.00
MGr-cap50-eps5	13.04	8.58	6.75	5.69	5.00
MGr-eps2	5.02	3.35	2.66	2.27	2.00
MGr-eps5	14.22	8.84	6.83	5.71	5.00
MGr-strm1-eps5	14.22	8.84	6.83	5.71	5.00
MGr-t2-eps5	7.63	5.00	3.94	3.33	2.94
MGr-t3-eps5	10.10	6.43	5.00	4.21	3.72
MGs-cap100-eps10	27.45	17.80	13.47	11.30	10.00
MGs-cap25-eps1	2.42	1.65	1.32	1.13	1.00
MGs-cap25-eps5	13.04	8.58	6.75	5.69	5.00
MGs-eps0.5	1.20	0.82	0.66	0.56	0.50
MGs-eps1	2.43	1.65	1.32	1.13	1.00

Configuration	$h=1$	$h=2$	$h=3$	$h=4$	$h=5$
MGs-eps10	33.10	19.07	13.86	11.43	10.00
MGs-eps2	5.02	3.35	2.66	2.27	2.00
MGs-eps20	99.61	43.27	29.65	23.80	20.00
MGs-eps5	14.22	8.84	6.83	5.71	5.00
MGs-strm1-eps1	2.43	1.65	1.32	1.13	1.00
MGs-strm1-eps5	14.22	8.84	6.83	5.71	5.00
MGs-strm2-eps10	28.46	18.04	13.54	11.32	10.00
MGs-t2-eps1	1.46	1.00	0.80	0.69	0.61
MGs-t2-eps10	15.89	10.00	7.61	6.44	5.59
MGs-t2-eps5	7.63	5.00	3.94	3.33	2.94
MGs-t3-eps1	1.83	1.25	1.00	0.86	0.76
MGs-t3-eps10	22.31	12.88	10.00	8.20	7.14
MGs-t3-eps5	10.10	6.43	5.00	4.21	3.72

Appendix C

Reproducing the Results

The code and the result files are in the project repository, https://github.com/Khanh1916/PP-FedData. The raw data are not part of the repository and must be obtained from the Kaggle release of the dataset [56]. Table C.1 lists the commands that produce the material of Chapter 4. Each is run as `python -m ppfeddata.cli <command>`.

Table C.1: Commands that produce the results of Chapter 4

Command	**Output**
`extract-normal`, `check-extraction`	Normal traffic extracted with the authors' settings, and the provenance report
`check`	Leakage report C1–C5
`aggregate`	Final report and tables of the main matrix
`scorecard`, `recommend`	Scorecard, Pareto front and recommendations per requirement
`privacy-units`	Protected units and honest-gateway thresholds
`robustness`, `sensitivity`	Robustness branches and worlds A2, A4 and A5
`ids-followup`	IDS-side options and local augmentation
`mg-nodp`	FedDP-Marginal without DP, plain and with SecAgg^{+}
`fed-baseline`	The classifier trained directly by federated learning
`mia`	Membership inference with access to the released CVAE
`compare-published`	Comparison with published results on the dataset
`accept`	Definition-of-done checklist

Two scripts in the thesis repository draw on these results. The script `make_thesis_figures.py` produces the figures of Chapter 4 that are not already available in the results folder, and `make_thesis_tables.py` produces Tables A.1 and B.1.

References

[1] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[2] A. Alatram, L. F. Sikos, M. Johnstone, P. Szewczyk, and J. J. Kang, "DoS/DDoS-MQTT-IoT: A dataset for evaluating intrusions in IoT networks using the MQTT protocol," *Computer Networks*, vol. 231, p. 109809, 2023.

[3] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[4] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. PMLR, 2017, pp. 1273–1282.

[5] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[6] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, "Federated learning for resource-constrained IoT devices: Panoramas and state-of-the-art," *Machine Learning and Knowledge Extraction*, 2022. [Online]. Available: https://arxiv.org/abs/2002.10610

[7] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/60a6c4002cc7b29142def8871531281a-Paper.pdf

[8] M. R. Behera, S. Upadhyay, S. Shetty, S. Priyadarshini, P. Patel, and K. F. Lee, "FedSyn: Synthetic data generation using federated learning," *arXiv preprint arXiv:2203.05931*, 2022. [Online]. Available: https://arxiv.org/abs/2203.05931

[9] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "PrivBayes: Private data release via Bayesian networks," *ACM Transactions on Database Systems*, vol. 42, no. 4, pp. 25:1–25:41, 2017.

[10] C. K. Chow and C. N. Liu, "Approximating discrete probability distributions with dependence trees," *IEEE Transactions on Information Theory*, vol. 14, no. 3, pp. 462–467, 1968.

[11] R. McKenna, D. Sheldon, and G. Miklau, "Graphical-model based estimation and inference for differential privacy," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 4435–4444. [Online]. Available: https://arxiv.org/abs/1901.09136

[12] R. McKenna, B. Mullins, D. Sheldon, and G. Miklau, "AIM: An adaptive and iterative mechanism for differentially private synthetic data," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2599–2612, 2022.

[13] A. Vyas, S. Balamurugan, P. Kumar, and K. Kotecha, "Privacy-preserving federated learning for intrusion detection in IoT environments: A survey," *IEEE Access*, vol. 12, pp. 130 452–130 478, 2024.

[14] R. Manzano, M. Zaman, D. Upadhyay, N. Goel, and S. Sampalli, "Federated learning framework based on distributed storage and diffusion model for intrusion detection on IoT networks," *IEEE Access*, vol. 13, pp. 79 571–79 595, 2025.

[15] J. Nakayiza and J. H. Park, "PureFL: Trust-weighted federated learning with noise-resilient homomorphic encryption for blockchain-based IoV networks," *IEEE Internet of Things Journal*, 2026.

[16] P. Ruzafa-Alcázar, P. Fernández-Maimó, F. Gómez Mármol, and G. Martínez Pérez, "Intrusion detection based on privacy-preserving federated learning for the industrial IoT," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 1145–1154, 2023.

[17] C. Bessaa, M. R. Senouci, and S. Bouzefrane, "FL-SQMPC: Secure quantized aggregation for mitigating gradient inversion attacks in federated IoT intrusion detection," *IEEE Journal on Selected Areas in Sensors*, 2026.

[18] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly)logarithmic overhead," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS '20)*. ACM, 2020, pp. 1253–1269.

[19] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. ACM, 2017, pp. 1175–1191.

[20] C. Gutti, R. Kumar, and S. Sharma, "Federated learning for distributed IoT security: A privacy-preserving approach to intrusion detection," *IEEE Access*, vol. 13, pp. 25 410–25 425, 2025.

[21] P. Kairouz, Z. Liu, and T. Steinke, "The distributed discrete Gaussian mechanism for federated learning with secure aggregation," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139. PMLR, 2021, pp. 5201–5212. [Online]. Available: https://arxiv.org/abs/2102.06387

[22] N. Agarwal, P. Kairouz, and Z. Liu, "The Skellam mechanism for differentially private federated learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34. Curran Associates, Inc., 2021, pp. 5052–5064. [Online]. Available: https://arxiv.org/abs/2110.04995

[23] Z. Deng, A. Torim, S. Ben Yahia, and H. Bahsi, "Generative AI in intrusion detection systems for internet of things: A systematic literature review," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 4689–4717, 2025.

[24] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[25] J. Yang, V. Govindarajan, L. Y. Por, Z. A. Shaikh, Q. Xin, P. Bhattacharya, A. A. Khan, and Y. Wang, "DDoS attack detection in consumer IoT-based healthcare systems using improved off-policy proximal policy optimization and generative adversarial network," *IEEE Transactions on Consumer Electronics*, vol. 71, no. 4, pp. 11 662–11 675, Nov. 2025.

[26] S. L. Jacob and H. P. Sultana, "GBOA_SR-ShuffleNet: An explainable federated learning framework for privacy-preserving intrusion detection in IoT," *Scientific Reports*, vol. 16, p. 5621, 2026.

[27] R. Anandhi and G. U. Srikanth, "Federated sparse-learning framework for detecting low-rate stealth attacks in resource-constrained IoT devices," *Scientific Reports*, vol. 16, p. 21298, 2026.

[28] S. Durga, M. G. Shankar, E. Daniel, and B. G. Varghese R., "Enhancing security in IoMT using federated TinyGAN for lightweight and accurate malware detection," *Scientific Reports*, vol. 16, p. 7116, 2026.

[29] H. N. Cunha Neto, J. Hribar, I. Dusparic, D. M. F. Mattos, and N. C. Fernandes, "A survey on securing federated learning: Analysis of applications, attacks, challenges, and trends," *IEEE Access*, vol. 11, pp. 41 928–41 953, 2023.

[30] W. Yao, H. Zhao, and H. Shi, "Privacy-preserving collaborative intrusion detection in edge of Internet of Things: A robust and efficient deep generative learning approach," *IEEE Internet of Things Journal*, vol. 11, no. 9, pp. 15 704–15 722, 2024.

[31] X. He, Q. Chen, L. Tang, W. Wang, and T. Liu, "CGAN-based collaborative intrusion detection for UAV networks: A blockchain-empowered distributed federated learning approach," *IEEE Internet of Things Journal*, vol. 10, no. 1, pp. 120–132, 2023.

[32] S. Salim, N. Moustafa, and A. Almorjan, "Responsible deep-federated-learning-based threat detection for satellite communications," *IEEE Internet of Things Journal*, vol. 12, no. 5, pp. 4807–4819, 2025.

[33] S. Jiao, J. Wang, Z. Tong, Z. Wang, L. Tan, and X. Zhang, "Foundation-model-based federated learning for intrusion detection in drone-aided industrial IoT," *IEEE Internet of Things Journal*, vol. 12, no. 22, pp. 46 889–46 901, 2025.

[34] R. Shettar, R. Coelho, and S. S. Veerasagar, "SYNAPSE: Synthetic network-traffic agent pipeline for federated edge security," *IEEE Access*, vol. 14, pp. 116 318–116 335, 2026.

[35] D. Hamouda, M. A. Ferrag, N. Benhamida, H. Seridi, and M. C. Ghanem, "Revolutionizing intrusion detection in industrial IoT with distributed learning and deep generative techniques," *Internet of Things*, vol. 26, p. 101149, 2024.

[36] S. Maddock, G. Cormode, and C. Maple, "FLAIM: AIM-based synthetic data generation in the federated setting," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. ACM, 2024, pp. 2165–2176.

[37] S. Pentyala, M. Pereira, and M. De Cock, "CaPS: Collaborative and private synthetic data generation from distributed sources," in *Proceedings of the 41st International Conference on Machine Learning (ICML '24)*, ser. Proceedings of Machine Learning Research, vol. 279. PMLR, 2024, pp. 173–198.

[38] X. Li, F. Gao, X. Wei, T. Wang, C. Shen, and J. Yang, "HeteroFedSyn: Differentially private tabular data synthesis for heterogeneous federated settings," *arXiv preprint arXiv:2603.08832*, 2026.

[39] S. Duan, C. Liu, P. Han, X. Jin, X. Zhang, T. He, H. Pan, and X. Xiang, "HT-Fed-GAN: Federated generative model for decentralized tabular data synthesis," *Entropy*, vol. 25, no. 1, p. 88, 2023.

[40] R. McKenna, G. Miklau, and D. Sheldon, "Winning the NIST contest: A scalable and general approach to differentially private synthetic data," *Journal of Privacy and Confidentiality*, vol. 11, no. 3, pp. 1–35, 2021.

[41] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28. Curran Associates, Inc., 2015, pp. 3483–3491. [Online]. Available: https://proceedings.neurips.cc/paper/2015/hash/8d55a249e6baa5c0677fbc52b95b178b-Abstract.html

[42] I. Mironov, "Rényi differential privacy," in *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*. IEEE, 2017, pp. 263–275.

[43] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. ACM, 2016, pp. 308–318.

[44] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.

[45] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[46] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. Li, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[47] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Tian, N. Christofides, R. Gu, M. Ling, H. Yu, B. Wang, Z. Gu *et al.*, "Opacus: User-friendly differential privacy library in PyTorch," *arXiv preprint arXiv:2109.12298*, 2021.

[48] O. Goldreich, *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge, U.K.: Cambridge University Press, 2004.

[49] S. P. Kasiviswanathan, H. K. Lee, K. Nissim, S. Raskhodnikova, and A. Smith, "What can we learn privately?" *SIAM Journal on Computing*, vol. 40, no. 3, pp. 793–826, 2011.

[50] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography Conference (TCC)*. Springer, 2006, pp. 265–284.

[51] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[52] W. E. Deming and F. F. Stephan, "On a least squares adjustment of a sampled frequency table when the expected marginal totals are known," *The Annals of Mathematical Statistics*, vol. 11, no. 4, pp. 427–444, 1940.

[53] M. Bun and T. Steinke, "Concentrated differential privacy: Simplifications, extensions, and lower bounds," in *Theory of Cryptography Conference (TCC)*. Springer, 2016, pp. 635–658.

[54] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*. ACM, 2019, pp. 2623–2631.

[55] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011, pp. 2546–2554.

[56] A. Alatram, L. F. Sikos, M. Johnstone, P. Szewczyk, and J. J. Kang, "DoS/DDoS-MQTT-IoT dataset," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/alaaalatram/dosddos-mqtt-iot

[57] A. Banks and R. Gupta, "MQTT version 3.1.1," OASIS Standard, Oct. 2014. [Online]. Available: http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[58] Wireshark Foundation, "Wireshark: Network protocol analyzer (TShark and capinfos)," Version 4.4.7, 2025. [Online]. Available: https://www.wireshark.org

[59] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[60] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proceedings of the 31st USENIX Security Symposium*. USENIX Association, 2022, pp. 3971–3988.

[61] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical Society*, vol. 7, no. 1, pp. 48–50, 1956.

[62] G. A. Miller, "Note on the bias of information estimates," in *Information Theory in Psychology: Problems and Methods*, H. Quastler, Ed. Glencoe, IL, USA: Free Press, 1955, pp. 95–100.

[63] I. Csiszár, "I-divergence geometry of probability distributions and minimization problems," *The Annals of Probability*, vol. 3, no. 1, pp. 146–158, 1975.

[64] B. Balle, G. Barthe, M. Gaboardi, and J. Geumlek, "Hypothesis testing interpretations and Rényi differential privacy," in *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 108. PMLR, 2020, pp. 2496–2506.

[65] J. G. Skellam, "The frequency distribution of the difference between two Poisson variates belonging to different populations," *Journal of the Royal Statistical Society: Series A (General)*, vol. 109, no. 3, p. 296, 1946.

[66] I. Mironov, K. Talwar, and L. Zhang, "Rényi differential privacy of the sampled Gaussian mechanism," *arXiv preprint arXiv:1908.10530*, 2019.

[67] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2014, arXiv:1312.6114.

[68] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, "beta-VAE: Learning basic visual concepts with a constrained variational framework," in *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.

[69] F. McSherry and K. Talwar, "Mechanism design via differential privacy," in *Proceedings of the 48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2007, pp. 94–103.

[70] J. Liu and K. Talwar, "Private selection from private candidates," in *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC)*. ACM, 2019, pp. 298–309.

[71] N. Papernot and T. Steinke, "Hyperparameter tuning with Rényi differential privacy," in *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022.

[72] P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912.

[73] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.

[74] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall/CRC, 1993.

[75] F. Zhang, Y. Cui, G. Cui, L. Huang, Q. Li, K. Shi, and Y. Lan, "Adaptive DDoS attack detection via packet payload feature selection," *Cybersecurity*, vol. 9, no. 1, p. 65, 2026.

[76] A. M. Parra and M. M. Bayas, "Neuro-fuzzy approach for detecting DDoS attacks in IoT environments applied to biosignal monitoring," *Technologies*, vol. 14, no. 5, p. 253, 2026.